

**Algebraic Multigrid Based Multilevel Decomposition for
Unstructured Scientific Data: Application to Data
Compression and Machine Learning**

by

Jeffrey Tyler Hadley

B.Sc.E., Mechanical Engineering, University of New Brunswick, 2019

Computer Science minor, University of New Brunswick, 2019

A thesis submitted to the
Faculty of the Graduate School of the
University of Colorado in partial fulfillment
of the requirements for the degree of
Doctor of Philosophy
Department of Aerospace Engineering Sciences

2026

Committee Members:

Kenneth Jansen, Chair

John A. Evans

Alireza Doostan

Stephen Becker

Jed Brown

Hadley, Jeffrey Tyler (Ph.D., Aerospace Engineering Sciences)

Algebraic Multigrid Based Multilevel Decomposition for Unstructured Scientific Data: Application to Data Compression and Machine Learning

Thesis directed by Prof. Kenneth Jansen and Prof. John A. Evans

Computational simulations of complex engineering problems generate immense volumes of scientific data on highly unstructured meshes, yet the majority of existing data compression and machine learning methods are fundamentally limited to structured or block-structured discretizations. This dissertation addresses this gap through the development of a novel algebraic multigrid based multilevel decomposition method for truly unstructured scientific data, referred to as A-MGARD, and demonstrates its utility across two distinct application domains: lossy scientific data compression and neural operator machine learning.

A-MGARD is derived by amalgamating the multilevel decomposition framework of Ainsworth et al. with algebraic multigrid coarsening methods, enabling the construction of a hierarchy of function spaces and interpolation operators directly from a mesh's weighted graph Laplacian, without assumptions on mesh structure or generation methodology. This distinguishes A-MGARD from its geometric counterpart MGARD, which is restricted to meshes generated through congruent element refinements, and renders it applicable to structured, unstructured, and mixed-element meshes alike.

The decomposition is first applied as a substitute for the decomposition step in a standard lossy compression pipeline and benchmarked against state-of-the-art compressors MGARD and SZ3 on turbulent channel flow data across a series of increasingly complex and unstructured mesh configurations. On meshes where both methods are applicable, A-MGARD demonstrates performance parity with geometric MGARD while exhibiting greater robustness on highly anisotropic configurations. On truly unstructured meshes where geometric MGARD is no longer applicable, A-MGARD achieves PSNR gains of up to 23 dB over

SZ3 in worst-case configurations, with compression factor improvements ranging from 1.8x to 4.6x across all tested data sets, confirming that A-MGARD’s mesh-informed decomposition produces a significantly more compressible multiscale representation of unstructured scientific data.

The decomposition is then integrated as the core of a novel neural operator architecture, the Multilevel Neural Operator (MNO), for learning surrogate models of complex PDEs on unstructured domains. By replacing the Fourier and wavelet decompositions used in FNO and WNO with A-MGARD, the MNO inherits the expressive multiscale function space learning approach of those methods while relieving their structured grid restrictions. Validation on structured advection wave and Navier-Stokes problems demonstrates that MNO architectures approach state-of-the-art performance on structured domains despite being designed for the more general unstructured setting. When applied to a vortex shedding cylinder in cross flow on an unstructured body-fitted mesh, a problem where current practical neural operators are not applicable, the MNO successfully learns surrogate models capturing dominant flow physics with lift and drag force prediction error norms of 3.5% and 1.15% respectively.

Together, these contributions demonstrate that A-MGARD is a broadly applicable mathematical tool whose utility extends well beyond its original purpose. The ability to construct meaningful hierarchical function spaces from any unstructured mesh, without geometric assumptions, enables substantial improvements in both unstructured data compression and machine learning on the complex geometries encountered in real engineering applications.

Dedication

To Britt, Ellie, and Piper

Acknowledgements

A special thank you to my co-advisors, Dr. Kenneth Jansen and Dr. John A. Evans, whose guidance and mentorship were invaluable throughout this research. Your ability to know when to let me wrestle with new concepts before stepping in was a gift. Those moments of productive struggle were where I grew the most, and ultimately where I developed a deeper and more genuine understanding of this complex work at the intersection of engineering physics, mathematics, and scientific computing. I am a better researcher for it.

Thank you to Dr. Alireza Doostan, Dr. Stephen Becker, and Dr. Jed Brown for serving on my committee and providing insightful discussions, and pointing us towards the correct avenues to explore in the early days of this research.

Thank you to Dr. James Wright III, for allowing me to implement my data compression work within the HONEE framework, and for collaborating on said implementation when I was deep in the weeds trying to figure out the best way to put all the puzzle pieces together.

Thank you to the many PHASTA, CMGLab, ASCR, and general FSM group members, present and past, for help and guidance. There's far too many to list, but you all know who you are. Many side discussions and 'voids' have led to meaningful implementation improvements, debugging successes, workflow speed-ups, and generally being able to look at my research from a different perspective. I've learned far beyond what is presented in this dissertation, and for that I am forever grateful for you all.

Finally, thank you to the c-suite at Spudson Farms: Duncan Bark, Kalvin Monroe, and Mitchell Wall, for graciously taking care of Ellie and Piper when Britt and I took some much needed and regenerative vacations.

Contents

Chapter

1	Introduction	1
2	Derivation of AMG based multilevel decomposition	4
2.1	Multilevel decomposition background	5
2.2	Algebraic multigrid (AMG) coarsening background	11
2.2.1	Algebraic Multigrid Coarsening Operators	12
2.3	Choice of system matrix, $\mathbf{A}$	14
2.4	AMG based multilevel decomposition for engineering applications	15
2.4.1	Pre-computing function space hierarchies and their operators with AMG	16
2.4.2	A-MGARD - Incorporating AMG into multilevel decomposition . . .	17
2.4.3	Practical implementation of A-MGARD	18
2.5	AMG process study: coarsening methods and prolongation operator smoothing	22
2.5.1	Strength of connection measures	24
2.5.2	Coarse/Fine splitting and interpolation method	25
2.5.3	Interpolation operator smoothing	26
2.5.4	Choice of strength of connection and smoothing methods testing . . .	28
2.6	Lessons learned, choice of $\mathbf{A}$ matrix	34
2.7	Software contributions	40
2.7.1	Python Implementation	40

2.7.2	PETSc x HONEE implementation	41
2.8	Summary	43
2.9	Future Work	45
3	Application of AMG based multilevel decomposition to data compression	46
3.1	Background and Motivation	46
3.2	Investigation of AMG based multilevel decomposition compression performance	48
3.2.1	General compression testing set-up and procedure	49
3.2.2	Source data and original mesh configuration	49
3.2.3	Modifying original meshes for induced unstructuredness	50
3.2.4	Measuring induced error from compression	51
3.3	Testing effect of mesh on SZ3 errors	52
3.3.1	Testing approach	53
3.3.2	Mesh effect results	53
3.4	Effect of decomposition methods and mesh type on compression errors	56
3.4.1	Decomposition method testing approach	56
3.4.2	A-MGARD vs G-MGARD on original JHTDB hex mesh	58
3.4.3	A-MGARD applied to perturbed hexagonal mesh	61
3.4.4	A-MGARD applied to unstructured tetrahedrals	64
3.5	Compressed solution visualizations	73
3.6	Storage considerations in the context of data compression	76
3.7	Summary and conclusions	76
3.8	Future Work	77
4	Application of AMG based multilevel decomposition to Neural Operators	79
4.1	Neural operator background and motivation	79
4.2	Development of Multilevel Neural Operator	83
4.2.1	MNO Architecture description	84

4.2.2	Defining decomposition transform $\mathcal{M}$ with A-MGARD	87
4.2.3	Parameterization in Multilevel Space	88
4.2.4	Unstructured Domain Considerations for $P(\cdot)$ Lifting	93
4.2.5	Embedding periodicity in the MNO	94
4.2.6	Consideration for Convolution Branch, $W(\cdot)$	95
4.3	Periodicity and Convolution Branch Study	96
4.3.1	Approximation model architecture set-ups	98
4.3.2	Effects of embedding periodicity in $K(\cdot)$ and $W(\cdot)$	99
4.3.3	Convolution Branch, GNN vs CNN	103
4.4	MNO for approximation models on unstructured domains	107
4.4.1	MNO model and training set-up	108
4.4.2	MNO model training results - Velocity magnitude	109
4.4.3	MNO model training results - Pressure	114
4.5	Software Implementation	121
4.6	Summary and Conclusions	122
4.7	Future Work	124
4.7.1	Proof of concept results of using MNO as a PDE solver initial-guess	
	generator	125
5	Dissertation summary and final thoughts	128
	Bibliography	131

Appendix

Tables

Table

4.1	1D Advection Wave problem - network architecture test configurations	. . .	98
4.2	2D time-dependent N-S problem - network architecture test configurations	.	98
4.3	Problem specific optimizer settings used in combination with models from		
	tables 4.1 and 4.2		99
4.4	Pressure lift and drag prediction statistics		116
4.5	Comparison of solver iteration counts between ‘Same $\mathbf{Y}$ ’ and MNO prediction.		
	listed are linear and non-linear iteration counts, residual norms, and total		
	linear iteration counts for each respective method.		127

Figures

Figure

2.1	Example of nested refinements required by original MGARD multilevel decomposition. The final mesh is obtained by starting with a coarse unstructured mesh (left-most) and applying nested geometric element refinement. The gold elements highlights the nested refinements of an initial coarse element. The final refined mesh (right-most) is thus block-structured, and not a truly unstructured mesh.	6
2.2	Example of hierarchical spaces utilizing hat functions, arbitrary 1D example. Nodes with green overlays represent $\mathcal{N}_\ell^*$ for that respective level. The dashed basis functions are associated to nodes shared between relative coarse and fine levels; the $\bar{\Delta}_\ell u(x)$ at those nodes are exactly 0 due to interpolatory requirements. Therefore, the dashed basis functions can be omitted in the hierarchical decomposition, resulting in restricted function spaces V_ℓ^* composed of the remaining solid line basis functions.	8
2.3	Components of multilevel decomposition, arbitrary 1D example.	10
2.4	Comparison of AMG prolongation schemes for u_x , with global error plotted against compression factor.	32
2.5	Comparison of AMG prolongation schemes for u_y , with global error plotted against compression factor.	33

2.6	Comparison of AMG prolongation schemes for u_z , with global error plotted against compression factor.	33
2.7	Uniform, unit hex element. Green node represents node i , while blue nodes are immediate neighbor nodes j . The element stiffness matrix entries associated with the green-blue node pairs are 0.0.	35
2.8	Examples of checker-boarding patterns emerging in increasing levels of compression when using a Poisson stiffness matrix for a uniform hex mesh. Compression is increased from 2x (top left) to 10x (bottom right)	37
2.9	Comparison of AMG coarse/fine splitting patterns resulting from mass weighted Laplacian A matrix (left) and Poisson stiffness A matrix (right). Bottom row figures show a cut-through the center of the domain to better expose the "slabs" of fine nodes in the Poisson stiffness matrix splitting case (right).	39
3.1	Full JHTDB Channel Flow domain, x velocity at time step 3100. Green box indicates the approximate location of the data cutouts for the data compression tests.	50
3.2	Comparison of mesh configuration effects on SZ3 compression performance. PSNR is plotted against bit rate. High PSNR and low bit rate is best. a) 129x129x65 node meshes b) 257x257x17 node meshes	55
3.3	Comparison of hexahedral mesh configurations.	57
3.4	Comparison of compression methods on 2D slices of original hexahedral mesh. PSNR is plotted against bit rate. a) Average of 65 129×129 slices, b) Average of 17 257×257 slices	59
3.5	Comparison of compression methods on original hexahedral meshes. PSNR is plotted against bit rate. a) 129×129×65, b) 257×257×17	60
3.6	Node perturbation effects on A-MGARD applied to hexahedral meshes. PSNR is plotted against bit rate. a) 129×129×65, b) 257×257×17	63

3.7 Comparison of tetrahedral mesh variations.	66
3.8 Effect of Tet mesh on A-MGARD. PSNR is plotted against bit rate. a)	
129×129×65, b) 257×257×17	68
3.9 Comparison of u_x velocity field compression results between a) Block and b)	
Slab tetrahedral meshes.	70
3.10 Comparison of u_y velocity field compression results between a) Block and b)	
Slab tetrahedral meshes.	71
3.11 Comparison of u_z velocity field compression results between a) Block and b)	
Slab tetrahedral meshes.	72
3.12 Visualization of A-MGARD decomposition compressed fields vs original, non-	
compressed field (u_y).	74
3.13 Visualization of A-MGARD decomposition compressed fields vs original, non-	
compressed field (u_z).	75
4.1 Visual example of MNO Architecture for a single MNO layer. Practical im-	
plementations have > 1 MNO layers.	84
4.2 Arbitrary periodic 1D domain. Node N is connected back to node 1 to embed	
periodicity.	94
4.3 12 testing errors vs epoch for 1D advection. a) Results for various combinations	
of embedding periodicity in $K(\cdot)$ and $W(\cdot)$ branches. b) Results for MNO style	
models compared to GNN and WNO. Superscript * denotes non-periodic for	
that branch. Inlays are close-ups of epochs 450 to 500	101
4.4 top: positional RMSE across samples. middle: positional mean error $\pm$ Std	
across samples. bottom: median wave shape and amplitude of the data sam-	
ple set at $t=5$, as well as the 95th and 5th percentile bounds of the wave	
amplitudes (area under curve). Dashed vertical lines are to identify trouble	
regions pre, mid, and post square wave.	102

4.5	Approximation model errors for 2D Navier-Stokes problem. a) l2 test error and RMSE per predicted timestep. b) RMSE across samples and position vs predicted timestep. Superscript * denotes non-periodic for that branch. . . .	104
4.6	Evolution of model predictions over time for the 2D Navier-Stokes problem. Results are presented in a grid, each column shows results for a given time step of the model. Each row, from top to bottom, represents the true solution, predicted solution from the trained MNO model, and absolute error over the time steps. Note that color scales vary by time, yet are consistent between truth and prediction for a given time	106
4.7	3D unstructured mesh used in vortex shedding case.	107
4.8	Approximation model errors for velocity magnitude, $ \vec{u} $, of the 2D Navier-Stokes problem. a) l2 test over epochs. b) RMSE across samples and position vs predicted timestep. Superscript * denotes non-periodic for that branch. .	111
4.9	Lomb-Scargle power density analysis of velocity magnitude. Comparison of M-GNN and GNN to truth.	112
4.10	Top to bottom: Model predictions compared to truth; $error = prediction - truth$, with truth data as visual reference; relative error for all predicted time steps.	113
4.11	Signed time-averaged relative error of pressure field predictions on surface of cylinder.	115
4.12	Lomb-Scargle power density analysis of lift (top) and drag (bottom). Predicted and truth.	117
4.13	Integrated pressure lift forces from modeled pressure field. Top to bottom: Model predictions compared to truth; $error = prediction - truth$, with truth data as visual reference; relative error for all predicted time steps.	119

4.14 Integrated pressure drag forces from modeled pressure field. Top to bottom:

Model predictions compared to truth; $error = prediction - truth$, with truth

data as visual reference; relative error for all predicted time steps. 120

Chapter 1

Introduction

As the world continually aims to improve its engineering capabilities, computational simulations of complex engineering problems have proven a valuable alternative to iterative design, build, test methodologies of old. The shift to a digital engineering age has brought forth revolutionary advancements in safety, productivity, and human achievements once thought beyond the realm of possibility. Entire automobile chassis, for example, can now be designed, virtually tested for a multitude of performance characteristics, and iterated upon prior to building costly physical prototypes. While indisputably an overwhelming net benefit to engineering and society as a whole, the shift to digital technologies for engineering has not been without its self-induced complexities. The convenience of virtual or digital engineering comes with the trade-off of generating immense amounts of data, through highly computationally intensive methods.

This data generation, at the scale of modern computational engineering, has fostered an environment where the rate of scientific data creation has far surpassed the ability to stream or store it sustainably, such that it can be meaningfully analyzed. These large volumes of scientific data produced by modern computing systems must be carefully reduced to pass through comparatively small data I/O bandwidths, while maintaining accuracy and precision to the original data. A systematic reduction in data size, achievable through data compression, permits storage of a greater range of meaningful temporal and spatial data at the same cost as what is currently possible.

Many data compression methods have been developed over the past few decades with variable levels of success depending on the nature of the data. Great advancements have been made in compression of highly structured data, such as images, videos, and scientific floating-point data on structured grids, ex: [39] [41] [20]. However, many practical engineering geometries such as airfoils, vehicle suspension components, robotics components, etc, often require highly unstructured body-conforming meshes for finite element or finite volume analysis. These meshes are often created with multiple disconnected regions of element refinement, mixed element types, and wall-normal element scaling, resulting in a breadth of element sizes and highly unstructured vertex connectivity schemes. Geometric structuredness is essentially lost for these types of problems, rendering the majority of existing lossy spatial compressors ineffective at achieving substantial compression for unstructured data, as will be shown in this dissertation. A crucial gap therefore exists for efficient lossy compression methods applicable to truly unstructured data.

Bridging this gap motivates the core methodological contribution of this dissertation: a novel multilevel decomposition method for truly unstructured meshes, derived from algebraic multigrid (AMG) principles, referred to hereafter as A-MGARD. Unlike existing methods such as MGARD [22, 1, 4, 3, 2], which is restricted to geometrically nested meshes obtained through congruent element refinements, A-MGARD imposes no assumptions on how the mesh was created, making it applicable to structured, unstructured, and mixed-element meshes alike.

The remainder of this dissertation is organized around three successive contributions, each building on the last. Chapter 2 presents the derivation of A-MGARD and its practical implementation leveraging existing computational frameworks. While the decomposition is motivated by and presented in the context of unstructured data compression, it is a versatile tool whose applicability extends well beyond this original use case, as the proceeding chapters will demonstrate.

Chapter 3 applies A-MGARD as a substitute for the decomposition step in a typical

decomposition, quantization, encoding, and lossless compression pipeline. To establish baseline performance, the method is first applied to compression problems where state-of-the-art methods such as MGARD and SZ3 [39] remain applicable, before being applied to increasingly complex unstructured discretizations where existing methods fall short or are no longer applicable.

To demonstrate the breadth of applicability of the decomposition beyond compression, Chapter 4 integrates A-MGARD as the foundation of a novel neural operator architecture, the Multilevel Neural Operator (MNO), for learning surrogate models of non-linear PDEs on unstructured domains. By replacing the Fourier and wavelet decompositions used in existing neural operators like FNO [35] and WNO [56], A-MGARD relieves the structured grid restrictions that limit those methods, enabling practical surrogate modeling on the complex unstructured geometries that define real engineering problems.

Each chapter presents its own background and motivations, research products and results, conclusions, and directions for future work. Finally, Chapter 5 provides an overall summary of the dissertation and final closing thoughts.

Together, these contributions demonstrate that a decomposition method, initially developed out of necessity for unstructured data compression, proves to be a broadly applicable mathematical tool, ultimately opening the door to practical machine learning on the complex unstructured geometries encountered in real engineering problems.

Chapter 2

Derivation of AMG based multilevel decomposition

The goal of lossy data compression methods is to efficiently compress data while maintaining acceptable levels of error in the compressed representation of the original solution. The original solution data is assumed to be given on an unstructured mesh with vertex (node) locations and element connectivity information known ahead of time. The mesh is a discretization of a spatial domain, Ω , where $\Omega \subset \mathbb{R}^d$, $d = 1, 2, 3$. The nodes of the mesh, x , are defined as $x = \{x_i\}_{i=1}^{n_L}$, where n_L is the total number of mesh nodes. The solution data, $u(x)$ is then stored as coefficients on respective mesh nodes. No other assumptions are made about the mesh or data. To achieve compression of a given solution $u(x)$, a representation of the solution must be established that utilizes a reduced amount of bits. This reduced representation is assumed to be obtained via a typical decomposition, quantization, encoding, and lossless compression pipeline, and will be denoted as $\tilde{u}(x)$. The decomposition step of the data compression pipeline aims to transform the original data $u(x)$ into a more manageable format for the proceeding steps. Ideally, this decomposition should return a systematic arrangement of coefficients, from large to small, for which quantization, encoding and lossless compression is effective. The remainder of this work will focus on the decomposition of unstructured data such that it can be integrated into the typical compression pipeline.

The remainder of this chapter is organized as follows: Section [2.1](#) will provide an overview of multilevel decomposition and is presented as a skeleton framework for decomposition of truly unstructured data. Sections [2.2](#) and [2.3](#) then provide an overview of AMG

methods to help complete the skeleton framework. Section 2.4 presents the amalgamation of AMG methods with multilevel decomposition to create the proposed method: A-MGARD, an AMG based multilevel decomposition of unstructured scientific data. Sections 2.5 and 2.6 presents some early development studies and results which influenced the design of the decomposition method presented in the preceding sections. Section 2.7 presents software implementation contributions for the novel method. Finally sections 2.8 and 2.9 will present a summary of this chapter and future work for the A-MGARD method, respectively.

2.1 Multilevel decomposition background

Multilevel decomposition, presented in [1, 4, 3, 2], performs recursive orthogonal projections of functions, u , living in a relative-fine discretized domain onto a relative-coarse discretized domain, while retaining the difference between each successive level's projections as 'multilevel coefficients', u_{mc} .

Due to the use of projections, the multilevel coefficients inherit orthogonality properties which provide numerical stability when modifications are made to the coefficients. Additionally, as they are hierarchical in nature, the multilevel coefficients decay as they approach the levels associated with the finest projections, analogous to the higher frequency coefficients of Fourier transforms. Originally created for data compression purposes, these multilevel coefficients thus encapsulate a richer multiscale representation of an original function u .

While this is incredibly powerful, a key limitation of the original MGARD multilevel decomposition is the requirement that the original function u is discretized on a domain which was obtained through nested geometric element refinements. An example of such a refinement is shown in fig. 2.1, where an initial coarse domain is refined in a nested manner, and the solution u would be discretized on the right-most domain.

Thus MGARD relies on an embedded structure within an 'unstructured' domain, and as such is only applicable to block-structured discretizations. This assumption thus precludes MGARD from operating on truly unstructured meshes. The following work removes this

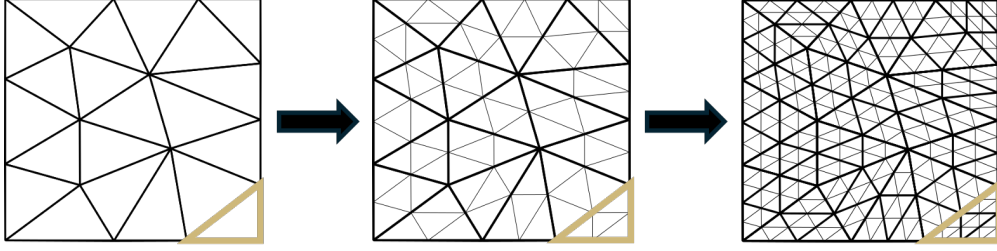


Figure 2.1: Example of nested refinements required by original MGARD multilevel decomposition. The final mesh is obtained by starting with a coarse unstructured mesh (left-most) and applying nested geometric element refinement. The gold elements highlights the nested refinements of an initial coarse element. The final refined mesh (right-most) is thus block-structured, and not a truly unstructured mesh.

block-structured restriction from the multilevel decomposition process, and develops a novel decomposition method tailored for truly unstructured meshes. To see how we can leverage the multilevel decomposition process for truly unstructured domains, it is best presented as a modified version of classical hierarchical decomposition, and the remainder of this section takes that approach.

The decomposition of data is achieved by first considering the data as a function $u(x) \in V_L$, where V_L is a function space spanned by linear Lagrange basis functions, $\mathcal{B}_L = \{\psi_i^L\}_{i=1}^{n_L}$, and nodes (coefficients), $\mathcal{N}_L = \{x_j^L\}_{j=1}^{n_L}$. These nodes are split into a hierarchy of node sets $\mathcal{N}_\ell = \{x_j^\ell\}_{j=1}^{n_\ell}$, $\mathcal{N}_0 \subset \dots \subset \mathcal{N}_L$. For each node set in the hierarchy, we define function spaces V_ℓ such that $V_0 \subset \dots \subset V_L$, $0 \leq \ell \leq L$, where L is relatively the finest level. We require that the functions spaces are interpolatory and observe a partition of unity. These function spaces are related by interpolation operators, Π_ℓ , such that $\Pi_\ell : V_L \rightarrow V_\ell$. These interpolation operators hold the following identity: $\Pi_{\ell-1}\Pi_\ell v = \Pi_{\ell-1}v$. Additionally, denote $\mathcal{N}_\ell^* = \mathcal{N}_\ell \setminus \mathcal{N}_{\ell-1}$ as the set of fine nodes that are not also in the set of coarse level nodes. Note that the union of all $\mathcal{N}_\ell^*$ recovers all the nodes with no overlap: $\cup_{\ell=0}^L \mathcal{N}_\ell^* = \mathcal{N}_L$ and $\mathcal{N}_\ell^* \cap \mathcal{N}_j^* = \emptyset \forall \ell \neq j$.

As a starting point, fig. [2.2](#) demonstrates the basis functions for a 1D example of a hierarchical decomposition consisting of 3 levels. The classical hierarchical decomposition

can be written as a sum of hierarchical components, $\bar{\Delta}_\ell u(x)$, defined as:

$$\bar{\Delta}_\ell u(x) = (\Pi_\ell - \Pi_{\ell-1})u(x) \quad (2.1)$$

which in turn can be written as the sum of hierarchical coefficients, $\bar{\Delta}_\ell u(x_i)$, multiplied by their corresponding basis functions:

$$\bar{\Delta}_\ell u(x) = \sum_{i \in \mathcal{N}_\ell} \bar{\Delta}_\ell u(x_i) \psi_i^\ell \quad (2.2)$$

Due to the interpolatory requirement, the difference of the solution interpolated onto neighboring levels is exactly 0 at nodes which are shared between neighboring levels. Thus, the hierarchical components can be represented on restricted subspaces of their respective level: $V_\ell^* = \text{span}\{\phi_i^\ell : i \in \mathcal{N}_\ell^*\}$. The direct sum of these restricted subspaces is the space that $u(x)$ lives in: $\bigoplus_{\ell=0}^L V_\ell^* = V_L$. Thus, representing the solution in terms of these hierarchical components forms a unique decomposition:

$$u(x) = \sum_{\ell=0}^L \bar{\Delta}_\ell u(x) = \sum_{\ell=0}^L \sum_{i \in \mathcal{N}_\ell^*} \bar{\Delta}_\ell u(x_i) \psi_i^\ell \quad (2.3)$$

Diverging from classical hierarchical decomposition, multilevel decomposition makes use of a hierarchy of orthogonal $L^2(\Omega)$ projections, $Q_\ell : V_L \rightarrow V_\ell$ along with nodal interpolation operators Π_ℓ to represent a given function u onto level V_ℓ . To see the connection between hierarchical and multilevel, first apply the identity $\Pi_{\ell-1}u(x) = \Pi_{\ell-1}\Pi_\ell u(x)$ onto eq. (2.1) to obtain:

$$\bar{\Delta}_\ell u(x) = (I - \Pi_{\ell-1})\Pi_\ell u(x) \quad (2.4)$$

Then, replace interpolants Π_ℓ with orthogonal projections Q_ℓ to get the multilevel components:

$$\Delta_\ell u(x) = (I - \Pi_{\ell-1})Q_\ell u(x) \quad (2.5)$$

The multilevel components, a sum of multilevel coefficients multiplied with basis functions, are simply the difference between the relative fine level projections, and the interpolation of those projections onto the relative coarse space. The multilevel coefficients are

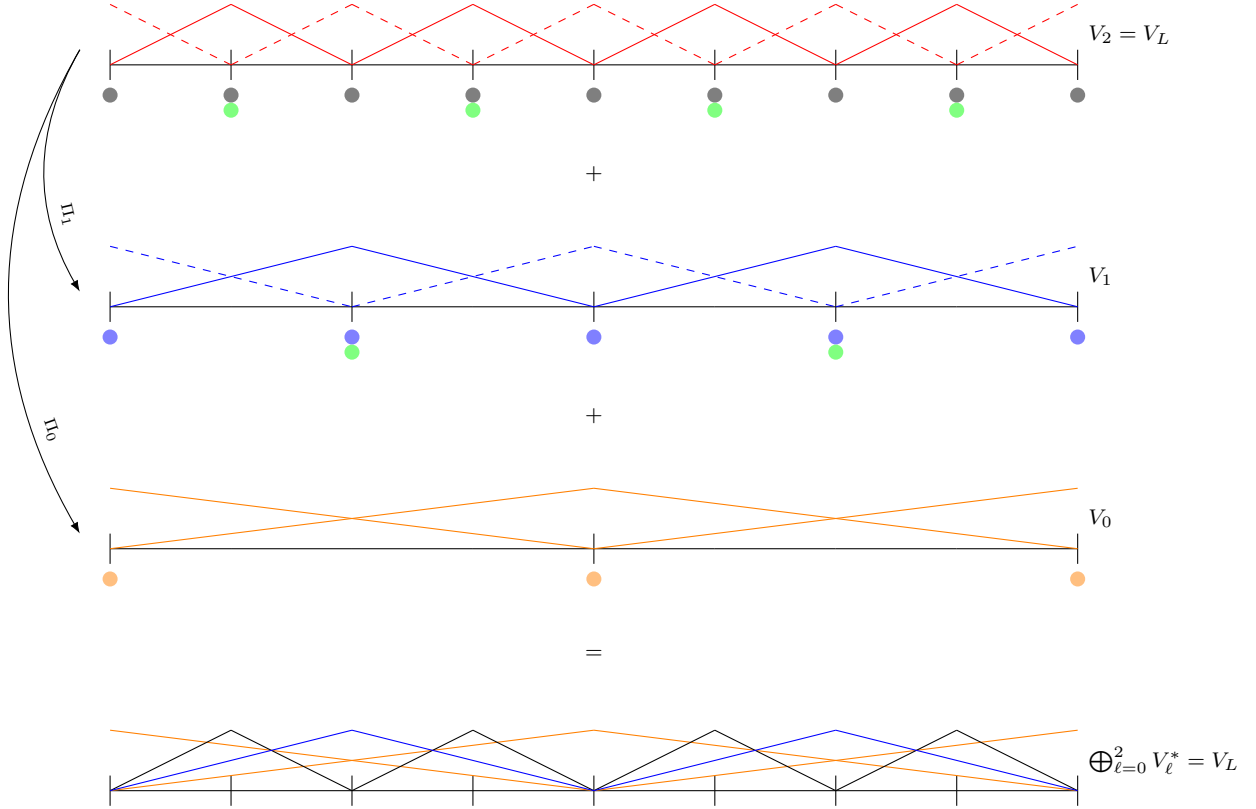


Figure 2.2: Example of hierarchical spaces utilizing hat functions, arbitrary 1D example. Nodes with green overlays represent $\mathcal{N}_\ell^*$ for that respective level. The dashed basis functions are associated to nodes shared between relative coarse and fine levels; the $\bar{\Delta}_\ell u(x)$ at those nodes are exactly 0 due to interpolatory requirements. Therefore, the dashed basis functions can be omitted in the hierarchical decomposition, resulting in restricted function spaces V_ℓ^* composed of the remaining solid line basis functions.

defined as $u_{mc}(x_i) \equiv \Delta_\ell u(x_i)$. A representation of this multilevel decomposition is visualized in fig. [2.3](#)

The original solution, u , is thus decomposed into the discretized multilevel coefficients, u_{mc} by:

$$u = \sum_{\ell=0}^L \sum_{i \in \mathcal{N}_\ell^*} u_{mc}(x_i) (I - Q_{\ell-1}) \psi_i^\ell \quad (2.6)$$

Analogously to the hierarchical decomposition, the difference of the relative fine projected solutions and their interpolation onto a relative coarse levels is exactly 0 at nodes which are shared between those levels. This permits to once again operate on restricted subspaces V_ℓ^* .

Interestingly, the operation of $(I - Q_{\ell-1}) \psi_i^\ell$ forms ‘out-of-level’ orthogonal basis functions, denoted $\tilde{\psi}_i^\ell$. These basis functions of level ℓ are orthogonal to all other basis functions which are not on that same level:

$$\int_{\Omega} \tilde{\psi}_i^\ell \tilde{\psi}_j^g = 0 \quad \forall i \in \mathcal{N}_\ell^*, j \in \mathcal{N}_g^*, \text{ and } \ell \neq g$$

‘Out-of-level’ basis functions $\tilde{\psi}_i^\ell$ are in essence wavelet bases created in an ad hoc manner at each level of the decomposition. As multilevel decomposition makes use of orthogonal projections, it inherits superior numerical stability properties leading to improved compression performance over hierarchical decompositions, as shown in [\[4\]](#) [\[3\]](#).

In a naive implementation, obtaining a projection $Q_\ell u$ onto coarse space V_ℓ , directly from the finest space V_L , is cost prohibitive. However, Ainsworth et. al. presented that these projections can be achieved in a more efficient, recursive manner, by relating the current level projection to the next coarse level projection with:

$$Q_{\ell-1} \mathbf{u} = Q_\ell \mathbf{u} - \Delta_\ell \mathbf{u} + (\mathbf{M}_{\ell-1})^{-1} (\mathbf{R}_\ell \mathbf{M}_\ell \Delta_\ell \mathbf{u}) \quad (2.7)$$

where $\mathbf{M}_\ell$ are the discretized mass matrices formed by the basis functions, and $\mathbf{R}_\ell$ is a discretized ‘restriction’ operator which will be defined in section [2.2](#). The finest level

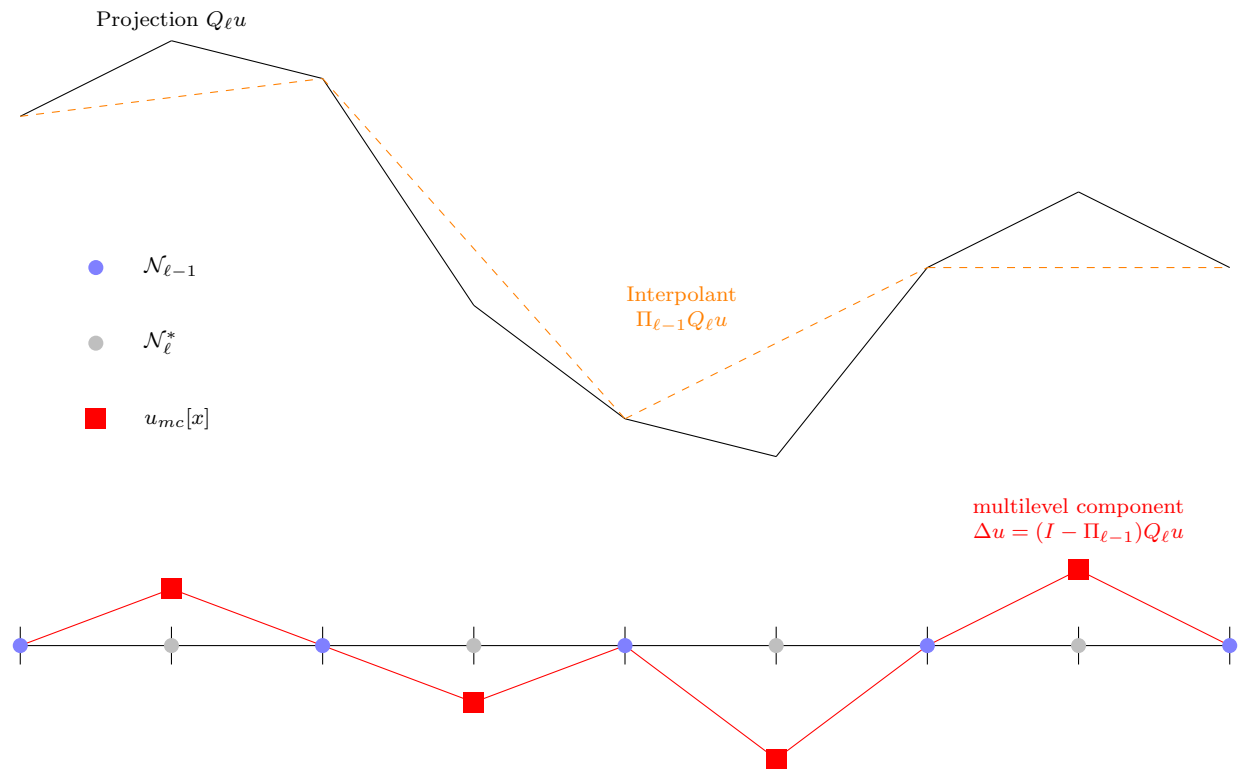


Figure 2.3: Components of multilevel decomposition, arbitrary 1D example.

mass matrix, $\mathbf{M}_L$, of an unstructured mesh can be obtained from a common finite element discretization of the following formulation, using linear Lagrange basis functions, ψ_n :

$$m(\psi_a, \psi_b) = \int_{\Omega} \psi_a \psi_b d\Omega \quad (2.8)$$

Noting that the discretized finest level projection $Q_L \mathbf{u}(x) = \mathbf{u}(x)$, the entire hierarchy of projections and resulting multilevel coefficients $\mathbf{u}_{mc}$ can be determined from original data and an appropriate hierarchy of function spaces and interpolation operators. Once the multilevel decomposition is obtained, the resulting multilevel coefficients $\mathbf{u}_{mc}$ can be passed to the remaining quantization, encoding, and lossless compression steps of the compression pipeline.

Additional details regarding the original multilevel decomposition framework derivation can be referenced in the series of papers [1, 4, 3, 2] by Ainsworth et al. The original framework assumed a priori knowledge of the function spaces, interpolation operators, and mass matrices which resulted from nested stenciled mesh refinements during the creation the of data $\mathbf{u}$'s computational domain Ω . In order to work with truly unstructured meshes, where the nested refinement assumption is no longer applicable, we must determine the V_ℓ function spaces and their associated operators $\mathbf{M}_\ell$, $\Pi_{\ell-1}$ from the original data mesh. To accomplish this, we now turn to algebraic multigrid.

2.2 Algebraic multigrid (AMG) coarsening background

Algebraic multigrid has long been used as an efficient method for solving large sparse systems of equations arising from the discretization of partial differential equations on unstructured meshes. To solve these systems, the AMG method decomposes the system into a series of related sub-systems, called coarsened levels, each containing fewer degrees-of-freedom than their predecessor. For a simple 2 level decomposition, the solution at the fine level is transferred onto the coarse level space, solved, and then transferred back to the fine level space. Ultimately, this process can be performed in a recursive manner over multiple

levels of a hierarchy of coarsened spaces. Importantly, the AMG method is what determines the hierarchy, associated transfer operators, and coarsened level operators without requiring a priori knowledge of an underlying function space hierarchy, or even mesh information [57]. These properties make AMG an ideal candidate in determining function spaces and transfer operators (interpolants) of truly unstructured meshes to be used in multilevel decomposition for data compression purposes.

In order to use the hierarchy of functions spaces and transfer operators from AMG for multilevel decomposition, we need to enforce certain properties. The interpolation operators produced by AMG must enforce the following properties for the basis function spaces:

- (1) The basis functions are interpolatory: $\psi_i^\ell(x_j^\ell) = \delta_{ij}$.
- (2) The basis functions form a partition of unity: $\sum_{i=1}^{n_\ell} \psi_i^\ell = 1$.
- (3) The basis functions are locally supported.

Properties 1 and 2 enable the method to accurately decompose and recompose the solution data u such that the method is consistent, i.e the original function u is recovered exactly from it's multilevel coefficients u_{mc} . The locally supported property produces function spaces with limited support; i.e. the regions of support of the basis functions overlap as little as possible. This permits the interpolation operators to remain sparse, which improves computational performance of the method, and makes them storage efficient.

A further explanation of the AMG method, its resulting interpolation operators, and their application in the context of unstructured scientific data compression is provided below.

2.2.1 Algebraic Multigrid Coarsening Operators

A variety of AMG coarsening methods have been developed over the years to solve a linear system of equations, $\mathbf{A}\mathbf{u} = \mathbf{b}$. Typically, this is completed through a cycling process, utilizing relaxation-correction schemes on increasingly coarse space representations of the

original system. The solution solve for the system occurs on the coarsest discretization and the solution gets interpolated upwards through the hierarchy by the relaxation-correction scheme. The discretized relative-coarse system of equations in the hierarchy are defined as $\mathbf{A}_{\ell-1}\mathbf{u}_{\ell-1} = \mathbf{b}_{\ell-1}$. These relative-coarse systems are obtained through the application of interpolatory transfer operators from the fine space to the coarse space: $\mathbf{A}_{\ell-1} = \mathbf{R}_\ell\mathbf{A}_\ell\mathbf{P}_\ell$. Interpolatory operators $\mathbf{R}_\ell$ and $\mathbf{P}_\ell$ are referred to as the restriction and prolongation operators, respectively, where $\mathbf{R}_\ell : V_\ell \rightarrow V_{\ell-1}$ and $\mathbf{P}_\ell : V_{\ell-1} \rightarrow V_\ell$. $n_{\ell-1}$ is denoted as the number of basis functions in the coarse space $V_{\ell-1}$, and n_ℓ as the number of basis functions in the fine space V_ℓ . Thus, $\mathbf{A}_\ell$ is $n_\ell \times n_\ell$, $\mathbf{P}_\ell$ is $n_\ell \times n_{\ell-1}$, $\mathbf{R}_\ell$ is $n_{\ell-1} \times n_\ell$, and resultant $\mathbf{A}_{\ell-1}$ is $n_{\ell-1} \times n_{\ell-1}$. Following standard procedure for AMG methods, this work proceeds with the Galerkin approximation of $\mathbf{R}_\ell \equiv \mathbf{P}_\ell^T$, thus only requiring an appropriate prolongation operator for every level [53].

The prolongation operator $\mathbf{P}_\ell$ defines $V_{\ell-1}$ as the linear combination of relative fine level basis functions $\{\phi_i^\ell\}_{i \in \mathcal{N}_\ell}$. Each entry in the columns of $\mathbf{P}_\ell$ defines weights, ω , used to combine the fine level bases to form the coarse level bases, $\mathcal{B}_{\ell-1}(\mathbf{P}_\ell) = \{\phi_j : \phi_j = \sum_{i=1}^{n_\ell} p_{ij}\phi_i^\ell\}$. The interpolatory and partition of unity constraints placed on the basis functions can then be imposed on the prolongation operators, assuming that the finest level basis functions are linear Lagrange, and all relative-fine basis functions stem from these. Therefore, to ensure partition of unity, we enforce that the row sums of $\mathbf{P}_\ell$ are equal to 1. To ensure an interpolatory property, we enforce that all entries of fine level rows i corresponding to a coarse level node j of $\mathbf{P}_\ell$ are 0, unless fine node i directly corresponds to coarse node j . The prolongation operator $\mathbf{P}_\ell$ is then constructed following:

$$\mathbf{P}_{i,j} = \begin{cases} 1, & \text{if } x_i^\ell = x_j^{\ell-1} \\ \omega, & \text{if } x_i^\ell \in \mathcal{N}_\ell^* \\ 0, & \text{if } x_i^\ell \neq x_j^{\ell-1}, x_i^\ell \in \mathcal{N}_{\ell-1} \end{cases} \quad s.t. \sum_{j=1}^{n_{\ell-1}} \mathbf{P}_{i,j} = 1, \forall i \quad (2.9)$$

The values of ω for each entry in $\mathbf{P}_\ell$ are determined by AMG through algebraic mea-

tures of each respective level's system matrix $\mathbf{A}_\ell$.

Assuming that AMG produces a hierarchy $\{\mathbf{P}_\ell\}_{\ell=1}^L$ abiding by our set constraints, we can obtain our required spaces and operators V_ℓ , Π_ℓ , $\mathbf{M}_\ell$, and $\mathbf{R}_\ell$ to be used at all levels in multilevel decomposition. To obtain the interpolation operator from multilevel decomposition, we use:

$$\Pi_{\ell-1} = \mathbf{P}_\ell [\mathbf{P}_\ell^T] \quad (2.10)$$

where $[\mathbf{P}_\ell^T]$ are the pure restriction operators which only selects the fine grid nodes on level ℓ corresponding to coarse grid nodes on level $\ell - 1$, i.e. $\mathcal{N}_{\ell-1} \cap \mathcal{N}_\ell$. Recalling that mass matrix $\mathbf{M}_\ell$ from multilevel decomposition is determined from the basis functions used to define V_ℓ , and that our relative coarse basis functions are simply linear combinations of the relative fine basis through $\mathbf{P}_\ell$, we can directly obtain the relative coarse mass matrices from the relative fine mass matrices: $\mathbf{M}_{\ell-1} = \mathbf{P}_\ell^T \mathbf{M}_\ell \mathbf{P}_\ell$.

Yet, a question still remains: what exactly is the matrix $\mathbf{A}_L$ that is passed to AMG to obtain these operators?

2.3 Choice of system matrix, $\mathbf{A}$

The AMG method was historically developed for application to a system matrix $\mathbf{A}$ that is a positive definite square *M-matrix* arising from a finite element discretization for a second-order elliptic problem [53, 10]. As the goal of utilizing AMG in the proposed decomposition method is to determine a hierarchy of function spaces and interpolation operators, rather than solving a system of equations, we can chose any arbitrary problem which results in an M-matrix.

It may seem logical to use the same mass matrix from the multilevel decomposition, however this matrix is not an *M-matrix* and often results in poor quality hierarchies of function spaces and interpolants. A simple, mesh informed M-matrix, is that of a weighted

graph Laplacian matrix of the following form:

$$\mathbf{A}_{i,j} = \begin{cases} \sum_{k,k \neq i} \int_{\Omega} \psi_i \psi_k d\Omega, & \text{if } i = j \\ - \int_{\Omega} \psi_i \psi_j d\Omega, & \text{if } i \neq j \end{cases}$$

Conveniently, this weighted Laplacian matrix is easily obtained through simple manipulations of the mass matrix created in the multilevel decomposition of section 2.1. $\mathbf{A}_{\ell}$ can be obtained by replacing the diagonal entries in $\mathbf{M}_{\ell}$ with it's off-diagonal row sums, and then multiplying those same $\mathbf{M}_{\ell}$ off-diagonals by -1. The function spaces and interpolation operators are then determined from these resulting weighted Laplacian matrices $\mathbf{A}_{\ell}$. It is important to note that the relative-coarse level graph Laplacian $\mathbf{A}_{\ell-1} \neq \mathbf{P}_{\ell}^T \mathbf{A}_{\ell} \mathbf{P}_{\ell}$, but rather should be obtained from first determining $\mathbf{M}_{\ell-1} = \mathbf{P}_{\ell}^T \mathbf{M}_{\ell} \mathbf{P}_{\ell}$ and then forming $\mathbf{A}_{\ell-1}$ from that coarsened mass matrix. Conveniently, modifying the mass matrix also permits us to store a single finest-level sparse matrix and recover the entire hierarchy from that matrix.

A general explanation of the creation of the algebraically smooth interpolatory transfer operators $\mathbf{P}_{\ell}$ from $\mathbf{A}_{\ell}$, as it was originally developed for AMG, can be referenced in [10]. The following section will describe the creation of interpolation operators and resulting algebraic multigrid hierarchy in the context of performing multilevel decomposition for data compression.

2.4 AMG based multilevel decomposition for engineering applications

We now describe how to incorporate the hierarchy of function spaces and their respective operators obtained from section 2.2 into the multilevel decomposition from section 2.1. In short, AMG determines the basis spaces V_{ℓ} and their relative interpolants, while the multilevel decomposition uses those tools to transform discrete function $\mathbf{u}$ into the decomposed function, $\mathbf{u}_{mc}$, which can be of use in various engineering context. For data compression, the coefficients can be passed to post-decomposition steps of compression pipelines as will be demonstrated in chapter 3. For machine learning, the coefficients can be used to learn

data-driven operators in a more expressive basis as will be demonstrated in chapter [4](#)

2.4.1 Pre-computing function space hierarchies and their operators with AMG

The first step in incorporating AMG with multilevel decomposition is to pre-compute appropriate prolongation (interpolation) operators and the resulting multigrid hierarchy from the finest level's mesh weighted Laplacian $\mathbf{A}_L$. The process of obtaining these operators for each coarsened partition level can be summarized as follows, starting with $\mathbf{A}_\ell = \mathbf{A}_L$:

- (1) From $\mathbf{A}_\ell$, determine a coarse and fine nodal splitting, C/F of the current level node set $\mathcal{N}_\ell$.
- (2) From $\mathbf{A}_\ell$ and C/F , determine a tentative prolongation operator $\mathbf{P}_\ell^0$.
- (3) Perform prolongation smoothing on $\mathbf{P}_\ell^0$ to obtain $\mathbf{P}_\ell$ such that the smoothing ensures a partition of unity.
- (4) Obtain coarse level mass operator, $\mathbf{M}_{\ell-1} = \mathbf{P}_\ell^T \mathbf{M}_\ell \mathbf{P}_\ell$.
- (5) Obtain coarse level weighted Laplacian $\mathbf{A}_{\ell-1}$ from $\mathbf{M}_{\ell-1}$ as described in section [2.2.1](#).
- (6) Repeat from 1, $\mathbf{A}_\ell \leftarrow \mathbf{A}_{\ell-1}$, until a desired amount of algebraically coarsened levels is achieved or a minimum number of nodes on the coarsest domain is reached.
- (7) Retain smoothed prolongation operator hierarchy $\{\mathbf{P}_1, \dots, \mathbf{P}_L\}$ for use in multilevel decomposition.

An assortment of algorithms and computational schemes with varying computational complexities have been developed over the years to determine coarsened function spaces and their corresponding interpolation operators, each aiming to improve AMG's PDE solver capabilities [\[53, 10, 9, 64, 19\]](#). As we are primarily concerned with determining function space hierarchies, Steps 1 and 2 are performed by utilizing what is commonly referred to as 'classical Ruge-Stuben' strength of connection measures and interpolation schemes [\[53, 10, 19\]](#).

These schemes are relatively computationally inexpensive and produce sparse interpolation operators as desired. The choice of utilizing these schemes was informed by a study during the development of the method, as presented in section [2.5](#).

The resultant interpolation from the classical Ruge-Stuben schemes are smoothed with a row sum scaling scheme (see section [2.5.3](#)) at each level of the hierarchy to ensure that the resulting coarse function spaces have an interpolating partition of unity. Once the smoothed prolongation operator is obtained (step 3 above), the coarse mass and Laplacian operators, $\mathbf{M}_{\ell-1}$ and $\mathbf{A}_{\ell-1}$, can be obtained, and steps 1 through 6 can be repeated. The resulting prolongation operator hierarchy $\{\mathbf{P}_\ell\}_{\ell=1}^L$ can now be stored in a sparse format to be used in the multilevel decomposition method.

2.4.2 A-MGARD - Incorporating AMG into multilevel decomposition

With a hierarchy of function spaces and their corresponding operators known from AMG, the proposed method can now proceed with performing multilevel decomposition using those operators. The coarsened level mass matrices used in multilevel decomposition can now be obtained via recursive applications of $\mathbf{M}_{\ell-1} = \mathbf{P}_\ell^T \mathbf{M}_\ell \mathbf{P}_\ell$. The interpolation operators $\Pi_{\ell-1}$ are formed with $\mathbf{P}_\ell[\mathbf{P}_\ell^T]$, and restriction operators $\mathbf{R}_\ell$ are $\mathbf{P}_\ell^T$. Substituting these AMG operators into an expanded version eq. [\(2.7\)](#), the relative coarse level projection can be written as:

$$Q_{\ell-1}\mathbf{u} = Q_\ell\mathbf{u} - (I - \mathbf{P}_\ell[\mathbf{P}_\ell^T])Q_\ell\mathbf{u} + (\mathbf{P}_\ell^T \mathbf{M}_\ell \mathbf{P}_\ell)^{-1} \mathbf{P}_\ell^T \mathbf{M}_\ell (I - \mathbf{P}_\ell[\mathbf{P}_\ell^T])Q_\ell\mathbf{u} \quad (2.11)$$

Recalling that the finest level projection is simply the original data, $Q_L\mathbf{u}(x) = \mathbf{u}(x)$, the entire hierarchy of solution $Q_\ell\mathbf{u}$ and multilevel components $\mathbf{u}_{mc}$ can be determined using the hierarchy of prolongation operators obtained from section [2.2](#)

Thus, the overall process of first pre-processing the solution level mass matrix $\mathbf{M}_L$ with AMG to discover the multilevel hierarchy, and then employing the resulting operators in a multilevel decomposition to obtain multilevel coefficients $\mathbf{u}_{mc}$, is the novel decomposition

method which we have named **A-MGARD**.

A-MGARD, short for Algebraic MGARD, is a respectful nod to the original MGARD method developed by Ainsworth et al., which directly inspired this work. We want to emphasize that, while it was inspired by MGARD, A-MGARD takes an entirely different approach to defining the decomposition. MGARD required that the final solution mesh was obtained from nested geometric element refinements, and thus their function space hierarchy was constructed in an *a priori* manner during the meshing process; MGARD requires that solution space V_L be constructed from coarsest space V_0 .

A-MGARD removes this restriction, and instead operates on **any** solution mesh, regardless of how it was obtained. A-MGARD **discovers** the function space hierarchy based on the algebraic information found in the system matrix representing the given mesh. Therefore, A-MGARD operates directly from solution space V_L , making it applicable to a much more general range of solution domains.

2.4.3 Practical implementation of A-MGARD

The A-MGARD method is best performed as a two step procedure: a **pre-compute** step to determine the hierarchy of function spaces and store the relevant operators. This pre-compute step only requires to be performed once for a given mesh. Pre-compute is then followed by a **compute** step which performs the multilevel decomposition using the AMG derived function space hierarchy.

With a hierarchy of prolongation operators $\mathbf{P}_\ell$ and the finest level mass matrix $\mathbf{M}_L$ available, the decomposition of the data $\mathbf{u}$ can proceed by following algorithm [1](#). This algorithm will decompose solution $\mathbf{u}$ into its multilevel coefficients $\mathbf{u}_{mc}$ for each respective level of the hierarchy. It starts with the finest level projection, and recursively determines the next coarse level projection until all levels of the hierarchy have been completed.

The resultant nodal $\mathbf{u}_{mc}$ values at each level replace their respective original nodal values in $\mathbf{u}$, i.e practically, the original memory for vector $\mathbf{u}$ takes on the values of $\mathbf{u}_{mc}$, thus

Algorithm 1 Algebraic Coarsening Multilevel Decomposition

Require: Hierarchy of interpolation operators $\{\mathbf{P}_\ell : 1 \leq \ell \leq L\}$ from Algebraic Multigrid

function AMGMULTILEVELCOEFFDECOMPOSE(data $\mathbf{u}$, Prolongation Operators $\{\mathbf{P}_\ell : 1 \leq \ell \leq L\}$, Mesh mass Matrix $\mathbf{M}_L$)

$\mathbf{u}_{mc} \leftarrow \mathbf{u}$

$\mathbf{M}_\ell \leftarrow \mathbf{M}_L$

for $\ell = L, \dots, 1$ **do**

for $x \in \mathcal{N}_\ell^*$ **do**

$\mathbf{u}_{mc}[x] \leftarrow \mathbf{u}_{mc}[x] - \mathbf{P}_\ell[\mathbf{P}_\ell^T]Q_\ell\mathbf{u}(x)$

end for $\triangleright \mathbf{u}_{mc}[\mathcal{N}_\ell^*]$ holds non-zero's of $(I - \mathbf{P}_\ell[\mathbf{P}_\ell^T])Q_\ell\mathbf{u}$, $\mathbf{u}_{mc}[\mathcal{N}_{\ell-1}]$ holds

$\mathbf{P}_\ell[\mathbf{P}_\ell^T]Q_\ell\mathbf{u}$

$\mathbf{M}_{\ell-1} \leftarrow \mathbf{P}_\ell^T\mathbf{M}_\ell\mathbf{P}_\ell$

$\boldsymbol{\alpha}_\ell \leftarrow \mathbf{u}_{mc}[\mathcal{N}_\ell] - [\mathbf{P}_\ell][\mathbf{P}_\ell^T]\mathbf{u}_{mc}[\mathcal{N}_\ell]$

$\mathbf{f}_{\ell-1} \leftarrow \mathbf{P}_\ell^T\mathbf{M}_\ell\boldsymbol{\alpha}_\ell$

 solve $\mathbf{M}_{\ell-1}\mathbf{z}_{\ell-1} = \mathbf{f}_{\ell-1}$ for $\mathbf{z}_{\ell-1}$

for $x \in \mathcal{N}_{\ell-1}$ **do**

$\mathbf{u}_{mc}[x] \leftarrow \mathbf{u}_{mc}[x] + \mathbf{z}_{\ell-1}[x]$ $\triangleright \mathbf{u}_{mc}[\mathcal{N}_{\ell-1}]$ now represents $Q_{\ell-1}\mathbf{u}$

end for

end for

return $\mathbf{u}_{mc}$

end function

the method does not require any additional storage to store the multilevel coefficients. The resultant $\mathbf{u}_{mc}$ coefficients vector from this decomposition can then be passed as inputs to their engineering application of interest.

As an example, in the data compression context, the multilevel coefficients can be passed through a compression pipeline, where the outputted compressed coefficients $\tilde{\mathbf{u}}_{mc}$ can be stored until the compressed version of the original solution $\tilde{\mathbf{u}}$ is desired. At that time, the decomposition can be reversed with algorithm 2 to obtain the solution in the original function space. The decomposition and recombination of these functions can simply be thought of as transform operations, analogous to the actions of forward and inverse wavelet transforms.

In the general application, algorithm 2 obtains the reconstructed version of the solution data, $\tilde{\mathbf{u}}$, by taking in multilevel coefficients $\mathbf{u}_{mc}$ and the same hierarchy of operators as in algorithm 1. Algorithm 2 essentially performs the decomposition in reverse, starting with the coarsest multilevel coefficients, recursively determining the next fine level projection, and ending with reconstructed original domain solution $\tilde{\mathbf{u}}$. If no modifications are made to $\mathbf{u}_{mc}$, the reconstruction recovers the original solution within machine precision.

Note that the AMG-based multilevel method requires storage space for the L sparse prolongation operators, as well as the sparse mass matrix of the original mesh in order to perform decomposition and recombination of the solution. We consider these storage costs minimal, as A-MGARD decomposition is envisioned to be used in a multi time-step setting, such as CFD solvers, where these data structures can be reused for every single time-step solution field which requires to be decomposed. In the context of data compression for CFD simulations, the data savings obtained from A-MGARD decomposition over multiple time-steps will far outweigh the comparatively minimal, yet necessary, storage costs of the decomposition method.

It should also be noted that the inverse, $(\mathbf{M}_{\ell-1})^{-1}$, required to solve for $\mathbf{z}_{\ell-1}$ in algorithms 1 and 2 is dense. Thus, performing a direct inverse becomes extremely costly as the domain size grows. Therefore, exploiting efficient iterative linear solvers such as those avail-

Algorithm 2 Algebraic Coarsening Multilevel Recomposition

Require: Hierarchy of interpolation operators $\{\mathbf{P}_\ell : 1 \leq \ell \leq L\}$ from Algebraic Multigrid

function AMGMULTILEVELCOEFFRECOMPOSE(compressed multilevel data $\tilde{\mathbf{u}}_{mc}$, Prolongation Operators $\{\mathbf{P}_\ell : 1 \leq \ell \leq L\}$, Mesh mass Matrix $\mathbf{M}_L$)

for $\ell = 1, \dots, L$ **do**

 Determine $\mathbf{M}_\ell$ and $\mathbf{M}_{\ell-1}$ from $\mathbf{P}_\ell$ hierarchy and $\mathbf{M}_L$

$\boldsymbol{\alpha}_\ell \leftarrow \tilde{\mathbf{u}}_{mc}[\mathcal{N}_\ell] - [\mathbf{P}_\ell][\mathbf{P}_\ell^T]\tilde{\mathbf{u}}_{mc}[\mathcal{N}_\ell]$

$\mathbf{f}_{\ell-1} \leftarrow \mathbf{P}_\ell^T \mathbf{M}_\ell \boldsymbol{\alpha}_\ell$

 solve $\mathbf{M}_{\ell-1} \mathbf{z}_{\ell-1} = \mathbf{f}_{\ell-1}$ for $\mathbf{z}_{\ell-1}$

for $x \in \mathcal{N}_{\ell-1}$ **do**

$\tilde{\mathbf{u}}_{mc}[x] \leftarrow \tilde{\mathbf{u}}_{mc}[x] - \mathbf{z}_{\ell-1}[x]$

end for

for $x \in \mathcal{N}_\ell^*$ **do**

 compute $\mathbf{P}_\ell[\mathbf{P}_\ell^T]Q_\ell \mathbf{u}(x)$ $\triangleright$ if $\ell = 0$, $\mathbf{P}_\ell[\mathbf{P}_\ell^T]Q_\ell \mathbf{u}(x) = 0$

$\tilde{\mathbf{u}}_{mc}[x] \leftarrow \tilde{\mathbf{u}}_{mc}[x] + \mathbf{P}_\ell[\mathbf{P}_\ell^T]Q_\ell \mathbf{u}(x)$

end for $\triangleright \tilde{\mathbf{u}}_{mc}[\mathcal{N}_\ell^*]$ now represents $(I - \mathbf{P}_\ell[\mathbf{P}_\ell^T])Q_\ell \tilde{\mathbf{u}}$, and $\tilde{\mathbf{u}}_{mc}[\mathcal{N}_{\ell-1}]$ now represents $\mathbf{P}_\ell[\mathbf{P}_\ell^T]Q_\ell \tilde{\mathbf{u}}$

end for

$\tilde{\mathbf{u}} \leftarrow \tilde{\mathbf{u}}_{mc}$

return $\tilde{\mathbf{u}}$

end function

able in PETSc [5] is recommended when minimizing overhead data structure storage is of utmost concern, such as in a data compression setting. Alternatively, if a user can afford to compute and store some additional supporting data structures, this inverse operation can be performed with a much faster LU decomposition. The user can pre-determine the LU factorization of the sparse $\mathbf{M}_{\ell-1}$ operators, and store them such that they are readily available for each inverse operation in the decomposition. This stored LU factorization option is highly beneficial when the decomposition is to be applied repeatedly within a single iteration of a given process, such as in learning epochs of a machine learning optimization problem, which will be presented in chapter 4. In our practical implementations of A-MGARD, solving for $\mathbf{z}_{\ell-1}$ using the LU factorization was significantly faster than iterative solver methods, often providing up to 10x computational speedups for the same problems.

2.5 AMG process study: coarsening methods and prolongation operator smoothing

We presented in section 2.2 that AMG was originally developed to be a solver. It is important to recognize that we are not using AMG for its solver capabilities, but rather for its ability to create function space hierarchies from an initial function space. Consequently, we are primarily interested in using AMG configurations which produce decomposition coefficients suitable for data compression purposes. What is typically beneficial for AMG solver purposes may not provide the same benefits for decomposition, and this is what this section seeks to present.

As shown in section 2.4.1, the first step in obtaining appropriate prolongation operators for decomposition is to determine a set of relative coarse and fine nodes from current level node set $\mathcal{N}_{\ell}$. This coarsening can be split into 2 procedures, first a measure of the strength of connection between the domain nodes, and then a coarse/fine splitting operation based on these strength of connection measures. The strength of connection step evaluates the influence of each nodal point in a system of equations $Au = b$ on the other points in the

system, denoted by a set of values S_i . There are two popular methods to determine the strength of connection of your system, forward and backward sense.

Historically, the strength of connection of a system of equations for AMG solver purposes was measured in a ‘classical’ forward sense, i.e., measuring the effects on source vector b at point i due to changes in solution u at point j . The forward measurement is appropriate for diagonally dominant semi-positive definite *M-Matrices* [53, 9, 47], such as those arising from the A system matrix used in the method.

While intuitive, forward strength of connection measurements degrade AMG solvers performance when the operator from the system of equations can no longer be classified as an *M-Matrix*, as was often the case when AMG methods were applied to broader ranges of partial differential equation problems [10, 9]. Researchers later found it beneficial to measure strength of connection in an ‘energy-based’ backward sense. This is achieved by measuring the effect on the solution vector u at point j due to a change in source vector b at point i . This provides a measure that is not restricted to *M-Matrices* and can be applied to more generalizable sets of problems, while alleviating the forward measure performance setbacks on non *M-Matrices* [59, 9].

While providing better results in the traditional AMG setting, the backward ‘energy-based’ measurement is more computationally intensive than the forward ‘classical’ measurement. Furthermore, its generalizability may not be required for decomposition purposes, as the system matrices used method produces an *M-matrice* for which the forward measurement was developed. The choice of utilizing ‘classical’ or ‘energy-based’ strength of connection measures for data decomposition purposes will be the subject of section 2.5.1. An overview of both the forward ‘classical’ and backwards ‘energy based’ methods are now presented, and further details can be found in [53, 10] and [60, 9], respectively.

2.5.1 Strength of connection measures

As stated, ‘classical’ forward AMG strength of connection measurements determine how much a change in the solution u at point j affects the source vector b at point i , which are related by:

$$a_{ij}u_j = b_i$$

Thus, the set of points j said to strongly influence point i is defined as:

$$S_i = \left\{ j : |a_{ij}| \geq \theta \max_{k \neq i} \{|a_{ik}|\} \right\} \quad (2.12)$$

with $0 < \theta \leq 1$. Typically, $\theta \approx 0.25$ for 2D systems, while 3D systems typically require larger theta values in the range of $0.5 \leq \theta \leq 0.6$ [64]. If point j strongly influences i then $S_{i,j} = |a_{ij}|$, and $S_{i,j} = 0$ otherwise. The ‘classical’ measurement is simply a relative measure of the off-diagonal entries of A .

Comparatively, the ‘energy-based’ backward measurement quantifies how much a change in the source vector b at point j affects the solution u at point i . A better representation of the system of equations for the backwards measure is then:

$$A^{-1}b = u \quad \text{or} \quad A^{-1}r = e \quad (2.13)$$

The ‘energy-based’ measurement is then quantifying the entries in A^{-1} . This quantification is in terms of the energy norms of A , as AMG algorithms seek to minimize the A energy norm of the relaxation errors, e , for each level [9][10].

Denote the i -th column vector $G^{(i)}$ with entries $(G^{(i)})_j = (A^{-1})_{ij}$, such that $AG = I$, and thus $AG^{(i)} = I^{(i)}$, where $I^{(i)}$ is the i -th canonical unit vector. The contribution of node j to the energy of this vector is taken to be the measure of the influence node j has on node i , and thus a measure of the strength of connection between i and j . As the direct inverse A^{-1} , and by proxy $G^{(i)}$, are impractical to obtain, the strength measurement in Equation (2.14) is achieved using relaxation-based approximations of the columns of A^{-1} . $G^{(i)}$ is then approximated by performing μ relaxations on $AG^{(i)} = I^{(i)}$ with the same relaxation scheme as

the AMG method is using for it's solve. The entries of strength of connection matrix, $\mathbf{S}$, measured in the A energy norm, can then be found by:

$$\mathbf{S}_{ij} = \frac{\|G^{(i)} - (G^{(i)})_j I^{(j)}\|_A}{\|G^{(i)}\|_A} \quad (2.14)$$

The matrix is then passed through a thresholding stage to determine the strongest connections. As in [9], a node i is said to strongly depend on nodes j in the set defined by:

$$S_i = \left\{ j : \mathbf{S}_{ij} - 1 \geq \theta \max_{k \neq i} \{\mathbf{S}_{ik} - 1\} \right\} \quad (2.15)$$

Again the measurement is typically performed using $\theta = 0.25$ for 2D and $\theta = 0.60$ for 3D.

The resulting strength of connection matrices from both the ‘classical’ forward measurement and backward ‘energy-based’ are sparsely populated by construction, thus subsequent operators resulting from these matrices will also be sparse. These strength of connection matrices are then used to inform the coarse-fine splitting and resulting interpolation schemes in the multigrid cycles, as nodes with the strongest connections are considered as the best coarse node candidates.

2.5.2 Coarse/Fine splitting and interpolation method

Once the strength of connection matrix $\mathbf{S}_\ell$ and strong connection node sets S_i of the system on the current level is known, the next step is to identify the coarse and fine node splitting sets, C and F , respectively. C and F node splittings for a level are analogous to relative coarse and fine level node sets $\mathcal{N}_{\ell-1}$ and $\mathcal{N}_\ell^*$.

These splittings are determined with a classical absolute-value coarsening method [53, 19]. This coarsening method evaluates the values in the $\mathbf{S}_\ell$ matrix and identifies the strongest connected nodes in the fine level $\mathcal{N}_\ell$ to comprise the coarse level node set, C ($\mathcal{N}_{\ell-1}$). The remaining fine nodes not selected to be coarse nodes are labeled as F nodes ($\mathcal{N}_\ell^*$) and are the nodes whose values must be interpolated from their strongly connected coarse nodes.

Upon completion of the C/F splitting procedure, the method yields following nodes sets, for each node i :

$N_i = \{j \mid \mathbf{S}_{ij} \neq 0\}$, the set of non-zero values in row i of strength of connection

$S_i = \{j \in N_i \mid i \text{ strongly depends on } j\}$

$$F_i^s = F \cap S_i$$

$$C_i^s = C \cap S_i$$

$N_i^w = N_i \setminus (F_i^s \cap C_i^s)$, non-zero entries that are not in strongly connected coarse or fine nodes.

With the strength of connection matrix S_ℓ and the above sets known, a tentative prolongation operator, P^0 , can be determined by employing a classical Ruge-Stuben Interpolation method [64, 19]. The interpolation method aims to determine tentative interpolation values, P_{ij}^0 , associated to coarse nodes j for each fine node i , based on the determined node sets and strengths of connections from previous steps. The interpolation values of the tentative prolongation operator are determined by:

$$P_{ij}^0 = -\frac{1}{a_{ii} + \sum_{k \in N_i^w} a_{ik}} \left(a_{ij} + \sum_{k \in F_i^s} \frac{a_{ik} a_{kj}}{\sum_{m \in C_i^s} a_{km}} \right), j \in C_i^s$$

Note that the interpolation values are calculated using the values in the A matrix (a_{ij}), while continuing to abide by the sparsity pattern resulting from the coarse-fine splitting sets. The interpolation values from this method do not guarantee a partition of unity, and so we must apply "smoothing" in order to ensure our coarse bases have a partition of unity. Two smoothing methods are considered, a simple prolongation operator row sum scaling, and a prolongation energy minimization method, as presented in the following section

2.5.3 Interpolation operator smoothing

As the basis functions on coarsened spaces $V_{\ell-1}$ are linear combinations of the basis functions from their relative fine space V_ℓ , an interpolating partition of unity is achieved by

ensuring that the final prolongation operators' row-wise coefficients sum to a constant:

$$\sum_{j=1}^{n_{\ell-1}} \mathbf{P}_{ij} = 1, \forall i$$

A simple method of achieving this requirement is to perform a row sum scaling on the tentative prolongation operator $\mathbf{P}^0$, as shown in algorithm [3](#)

Algorithm 3 Prolongation Row Sum Scaling

Require: Tentative Prolongation Operator $\mathbf{P}^0$ from Classical Interpolation

function ROWSUMSCALING($\mathbf{P}^0$)

for $i = 1, \dots, n$ **do**

$a \leftarrow \sum_{j \in \mathbf{P}_{ij}^0 \neq 0} \mathbf{P}_{ij}^0$

for $j \in \mathbf{P}_{ij}^0 \neq 0$ **do**

$\mathbf{P}_{ij} \leftarrow \mathbf{P}_{ij}^0 / a$

end for

end for

return $\mathbf{P}$

end function

This algorithm makes use of the sparsity pattern of $\mathbf{P}^0$ to perform $\mathbf{P} = \text{diag}(\mathbf{P}^0 \mathbf{1})^{-1} \mathbf{P}^0$, where $\mathbf{1}$ is a column vector of 1's. This scaling maintains the ratios of coarse node contributions to each fine node i , while ensuring partition of unity if the i -th row sum of $\mathbf{P}^0$ is not already equal to 1.

A more robust prolongation operator smoothing method is that of energy-minimized basis functions, as developed in [\[59, 47, 50\]](#). Energy-minimized prolongation smoothing has the useful property of identically satisfying a partition of unity for the data compression problem at hand. As in [2.2.1](#), $P_\ell : V_{\ell-1} \mapsto V_\ell$, thus the vectors $P_\ell I^{(j)}$ are the discrete coarse basis functions of the coarse space. As before, $I^{(j)}$ is the j -th canonical unit vector. The energy of the j -th discrete coarse basis function is written as:

$$(API^{(j)}, PI^{(j)}) = p_{*j}^T A p_{*j}$$

Prolongation operator smoothing seeks to minimize the total energy, J , of these coarse

discrete basis functions:

$$J(P) = \frac{1}{2} \sum_{j=1}^{n_{\ell-1}} p_{*j}^T A p_{*j} = \frac{1}{2} (P, A P)$$

and uses a general optimization constraint of the form:

$$P B_{l-1} = B_l$$

where B_{l-1} and B_l are column-based matrices of the null-space modes of the system for the coarse and fine levels respectively.

Recalling the resulting A of the system from section [2.3](#) the null-space mode of such system is identically a single vector of 1's. For this null-space characteristic to hold on both the fine and coarse space, $B_\ell = \mathbf{1}$ and $B_{\ell-1} = \mathbf{1}$, therefore the prolongation operator must then preserve a partition of unity by construction. The energy minimized prolongation operators are then the solution to the minimization problem:

$$\min_P J(P) = \frac{1}{2} (P, A P)$$

subject to,

$$\sum_{j=1}^{n_{l-1}} P_{ij} = 1, \forall i$$

Note that the quadratic form $(P, A P)$ is strictly convex due to A being symmetric-positive-definite, thus this problem has a solvable minimum which can be obtained from a projected steepest descent method or other efficient minimization algorithms.

The resulting smoothed prolongation operators hierarchy from either the ‘classical’ and ‘energy-based’ AMG prolongation methods can now be stored in a sparse format to be used in the multilevel decomposition method.

2.5.4 Choice of strength of connection and smoothing methods testing

As described in section [2.4.1](#) the hierarchy of algebraically coarsened domains can be obtained by applying prolongation operators to system matrices. The prolongation operators

are the direct result of a combination of various strength of connection and prolongation operator smoothing methods. We will refer to these combinations as prolongation operator schemes.

Following previous discussions, the prolongation schemes can then be combinations of ‘classical’ and ‘energy-based’ methods. While ‘energy-based’ methods are applicable to more generalizable systems of equations in a traditional AMG solver sense, the additional computational costs incurred by these methods must be justified by outperforming ‘classical’ AMG methods in terms of data compression.

To determine their feasibility, we performed data compression tests utilizing coefficients resulting from the decomposition of various prolongation scheme combinations. These compression tests utilized 4 different schemes to determine whether the additional computational costs of energy-based AMG methods is justified. The following decomposition schemes are set by the choice of method for strength of connection measure and prolongation smoothing, as described in sections [2.5.1](#) and [2.5.3](#) respectively:

Energy Strength of Connection with Energy Minimization Smoothing (ESoC-EMS) utilized the ‘energy-based’ strength of connection measure, classical Ruge-Stuben Coarse-Fine splitting and Interpolation, followed by Energy Minimized prolongation smoothing.

Energy Strength of Connection with Row Sum Scaling (ESoC-RSS) utilized the ‘energy-based’ strength of connection measure, classical Ruge-Stuben Coarse-Fine splitting and Interpolation, followed by Row Sum Scaling prolongation smoothing.

Classical Strength of Connection with Energy Minimization Smoothing (CSoC-EMS) utilized the ‘classical’ strength of connection measure, classical Ruge-Stuben Coarse-Fine splitting and Interpolation, followed by Energy Minimized prolongation smoothing.

Classical Strength of Connection with Row Sum Scaling (CSoC-RSS) utilized the ‘classical’ strength of connection measure, classical Ruge-Stuben Coarse-Fine splitting

and Interpolation, followed by Row Sum Scaling prolongation smoothing.

2.5.4.1 Prolongation operator scheme testing approach

A compression percentage sweep test was performed on a 2D cutout of the JHTDB DNS turbulent flow channel data [33] [23] for each of the schemes described above. The selected data cutout was obtained by taking a 257 x 257 node x-y plane (Cutout indices: $x = [0, 256]$, $y = [0, 256]$, $z = 24$) at time step 3100. The cutout's rectangular domain is of non-dimensional size $0 \leq x \leq \Pi$ and $-1 \leq y \leq 0.0030795$. The data consists of 66049 data points, each with u_x , u_y , and u_z velocities data fields. It should be noted that the original JHTDB Channel Flow mesh has a wall-normal Δy growth to account for the flow's boundary layer, thus providing better resolution in this region. As such, the JHTDB mesh is structured in x and semi-structured in y, with highly anisotropic elements in the near wall boundary layer region.

Each test made use of a hybrid Delaunay modified version of the JHTDB mesh cutout, and the x , y , and z velocity fields were individually compressed for each scheme. The hybrid Delaunay mesh was obtained by splitting the original domain of the JHTDB cutout into structured and unstructured mesh regions. The structured region, $y < -0.6666$, is formed by splitting each original quad element into 2 upper and lower triangles. The unstructured region has a Delaunay triangulation applied to perturbed nodes locations. The data at each of the new node locations was interpolated from original JHTDB data using a cubic spline interpolation method.

Variable amounts of compression was achieved by retaining only the $\tilde{n}$ largest magnitude coefficients and reconstructing the solution fields from the reduced set of compressed coefficients. The number of multilevel coefficients retained, $\tilde{n}$, is determined by:

$$\tilde{n} = \frac{n_L}{CR}$$

where n_L is the total amount of original data points and CR is the standard compression ratio,

where $CR = 10$ means you retain 1/10th of the original amount of data. For testing, this was practically achieved by setting non-retained coefficients' values to 0.0 prior to recomposition.

The global error introduced by compression was then determined with following formula:

$$\%Error^{global} = 100\% * \frac{\|\tilde{u} - u^o\|_{l^2}}{\|u^o\|_{l^2}} = 100\% * \frac{rmse}{\|u^o\|_{l^2}} * \sqrt{n_L}$$

where $\tilde{u}$ is the reconstructed data set for the prescribed compression ratio, and u^o is the original uncompressed data set. Compression error was reported against compression ratio for each of the strength of connection and prolongation smoothing method combinations.

2.5.4.2 Effect of strength of connection and smoothing methods results

Observing the resulting global errors curves in Figures [2.4](#) to [2.6](#) it is evident that the additional computational cost of 'energy-based' methods do not provide significant improvements over 'classical' methods, if any for decomposition and compression purposes. The least computationally expensive scheme, CSoC-RSS, marginally outperforms the most expensive scheme, ESoC-EMS, in both the u_x and u_z fields while matching performance in the u_y field. Accordingly, the classical schemes provide successful interpolation operators while incurring the least amount of computational expenses. The least expensive, CSoC-RSS, which is classical Ruge-Stuben strength of connection measures paired with row-sum scaling to ensure partition of unity, will be the default prolongation operator creation scheme used for the A-MGARD method.

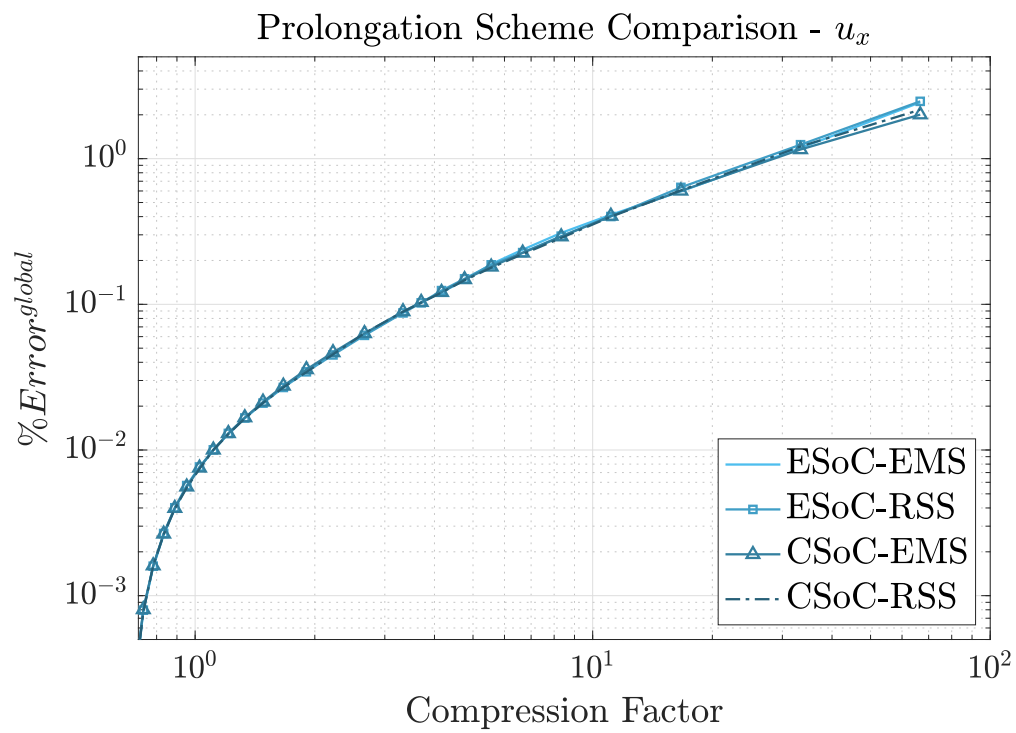


Figure 2.4: Comparison of AMG prolongation schemes for u_x , with global error plotted against compression factor.

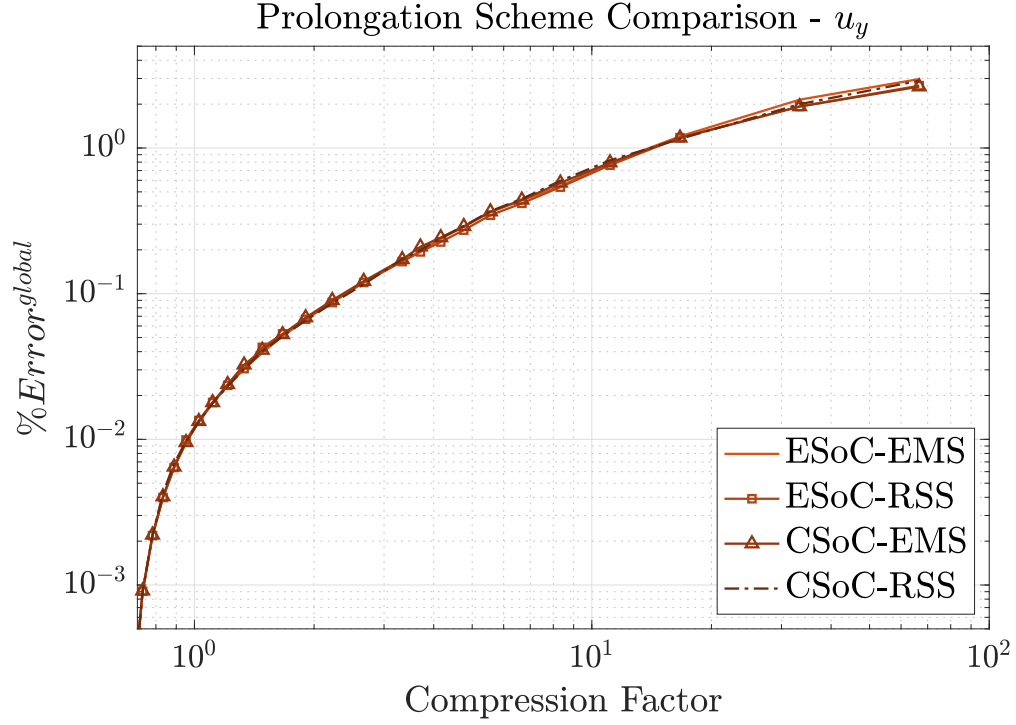


Figure 2.5: Comparison of AMG prolongation schemes for u_y , with global error plotted against compression factor.

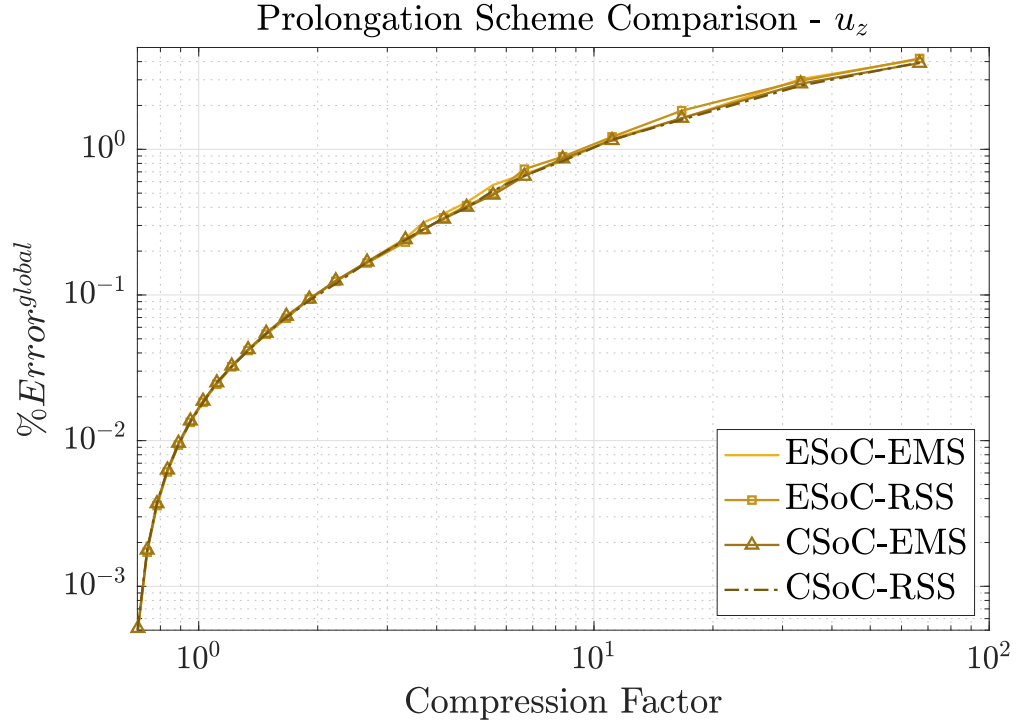


Figure 2.6: Comparison of AMG prolongation schemes for u_z , with global error plotted against compression factor.

2.6 Lessons learned, choice of A matrix

The choice of utilizing a mass weighted graph-Laplacian is the result of a lengthy debugging process which began when we first attempted to apply the A-MGARD decomposition to 3D domains. In the early development of the A-MGARD decomposition method, the A_L system matrix used in the process of creating the AMG hierarchy was the stiffness matrix resulting from the discretization of the Poisson problem on the same domain discretization as the data to be compressed:

$$k(\nabla\phi, \nabla\phi) = \int_{\Omega} \nabla\phi \cdot \nabla\phi d\Omega \quad (2.16)$$

We initially opted to use the Poisson stiffness matrix as it produces a matrix with a positive diagonal and negative off-diagonal entries, thus an *M-Matrix*. The stiffness matrix also embedded useful gradient information into the creation of the hierarchy. Additionally, as the A-MGARD method was envisioned to be used in conjunction with FEM numerical solvers, the stiffness matrix could easily be obtained in the pre-processing steps.

The AMG process appeared to work very well with a Poisson stiffness matrix for the 2 dimension examples we had used for early testing. However, issues began to appear with the resulting function space hierarchies and decomposition coefficients when we moved onto testing the method with 3 dimensional domains. Specifically, issues were presenting themselves with what should have been the "easiest" test cases, structured, uniform hexahedral meshes. When applied in a compression context, these structured uniform hex cases produced severe artifacts, specifically checker-boarding and striped patterns in the reconstruction of compressed solutions.

Unfortunately, the use of a Poisson stiffness matrix was at the root of undesired behavior in the resulting function spaces and interpolation operators. As was presented in section [2.5](#) the strength of connection measures from the first step in the AMG process informs the coarse/fine splitting step, where the coarsened domain nodes are determined. The measure of strength of connection between nodes makes use of a relative measure of

the non-zero off-diagonal entries in the target node's respective row of the A matrix. The Poisson stiffness matrix for a uniform hex mesh is especially problematic, as 3 off-diagonal entries of a given row, which would normally have non-zero values for non-uniform meshes, are identically 0.0 by construction.

This causes the AMG algorithms to entirely ignore these nodes, as a value of 0.0 in the system matrix indicates that these nodes do not influence each other. In actuality, these are the most strongly connected nodes from a topological standpoint as they directly share an edge. As demonstrated below, these problematic zero entries correspond to topologically connected nodes and arise during the standard finite element discretization when evaluating the stiffness integral (eq. (2.16)) over the reference element.

The hex single element stiffness matrix, A_{ij}^e , can be written in terms of an integral over the the parent element coordinates, ξ_k , as follows:

$$A_{ij}^e = \frac{kh}{2} \int_{-1}^1 \int_{-1}^1 \int_{-1}^1 \left(\frac{\partial \phi_i}{\partial \xi_1} \frac{\partial \phi_j}{\partial \xi_1} + \frac{\partial \phi_i}{\partial \xi_2} \frac{\partial \phi_j}{\partial \xi_2} + \frac{\partial \phi_i}{\partial \xi_3} \frac{\partial \phi_j}{\partial \xi_3} \right) d\xi_3 d\xi_2 d\xi_1 \quad (2.17)$$

where k is the stiffness coefficient and h is the element size for a single dimension of the **uniform** element (volume of h^3). If we consider the green node in fig. 2.7 to be node i ,

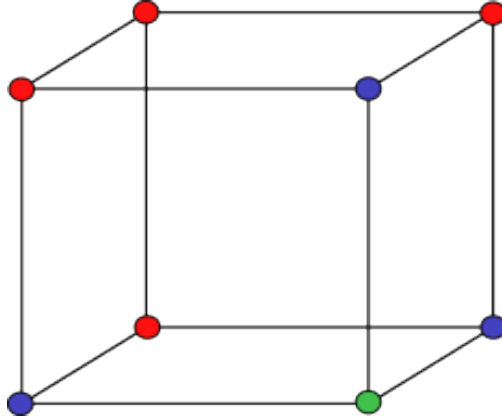


Figure 2.7: Uniform, unit hex element. Green node represents node i , while blue nodes are immediate neighbor nodes j . The element stiffness matrix entries associated with the green-blue node pairs are 0.0

with parent element coordinates $(-1, -1, -1)$, and any of the blue nodes to be node j with

coordinates $(1, -1, -1)$, the respective basis functions associated with nodes i and j are as follows:

$$\begin{aligned}\phi_i &= \frac{1}{8}(1 - \xi_1)(1 - \xi_2)(1 - \xi_3) \\ \phi_j &= \frac{1}{8}(1 + \xi_1)(1 - \xi_2)(1 - \xi_3)\end{aligned}$$

If we take the respective directional derivative of these basis functions, $\frac{\partial \phi_l}{\partial \xi_k}$, and split up the sums from eq. (2.17), we find that:

$$\begin{aligned}\int_{-1}^1 \int_{-1}^1 \int_{-1}^1 \left(\frac{-1}{8}(1 - \xi_2)(1 - \xi_3)\right) \left(\frac{1}{8}(1 - \xi_2)(1 - \xi_3)\right) d\xi_3 d\xi_2 d\xi_1 &= \frac{-2}{9} \\ \int_{-1}^1 \int_{-1}^1 \int_{-1}^1 \left(\frac{-1}{8}(1 - \xi_1)(1 - \xi_3)\right) \left(\frac{-1}{8}(1 + \xi_1)(1 - \xi_3)\right) d\xi_3 d\xi_2 d\xi_1 &= \frac{1}{9} \\ \int_{-1}^1 \int_{-1}^1 \int_{-1}^1 \left(\frac{-1}{8}(1 - \xi_1)(1 - \xi_2)\right) \left(\frac{-1}{8}(1 + \xi_1)(1 - \xi_2)\right) d\xi_3 d\xi_2 d\xi_1 &= \frac{1}{9}\end{aligned}\quad (2.18)$$

Notice that the sum of these three integrals amounts to 0.0. Thus, the nodes which are intuitively most strongly connected in a topological sense are completely ignored during the coarse/fine splitting creation process, as the AMG algorithms only selects non-zero of the system matrices.

The zero sum for edge-neighboring nodes is an issue as the nature of the coarse/fine splitting algorithm creates what is best described as 2D "slabs" of $\mathcal{N}_\ell^*$ nodes spanning nearly the entirety of the domain on the first pass of coarsening finest level L . These "slabs" of fine-level only nodes, which must be interpolated from coarse nodes, in-turn caused uni-directional interpolation schemes.

In a data compression context, fair amounts of compression was still achievable with low errors, however the uni-directional interpolation essentially introduced isolated regions with relatively very low error, and other regions with relatively high error (i.e. the fine node "slabs"). These two regions caused an acceptable average error, however the isolated regions of high error caused pronounced artifacts such as stripes and checker-boarding within the solution reconstruction. Examples of the checker-boarding artifacts can be seen in fig. 2.8

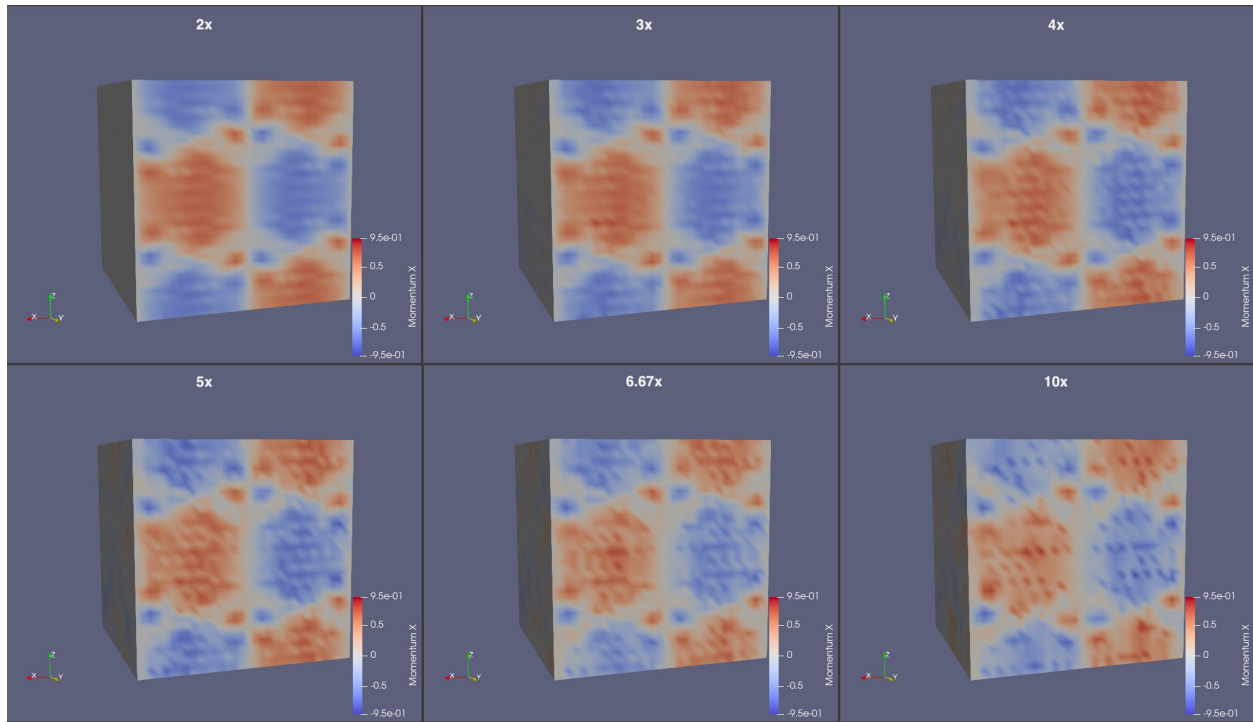


Figure 2.8: Examples of checker-boarding patterns emerging in increasing levels of compression when using a Poisson stiffness matrix for a uniform hex mesh. Compression is increased from 2x (top left) to 10x (bottom right)

Figure 2.9 shows the resulting coarse fine splittings for both the appropriate graph Laplacian based A matrix (left figures), and the use of the Poisson stiffness A matrix (right figures) on a uniform hex mesh. An example of a "slab" of fine nodes (blue color, splitting value of 0) is best seen in the bottom right figure

Note that once any form of anisotropy is introduced to the mesh, eq. (2.17) can no longer be written in simplified form with the $h/2$ term pulled outside of the integrals as each dimension is now different. The different edge lengths carry into the gradients, and the 0 sum issue for edge-connected neighboring nodes disappears. While the use of a Poisson stiffness matrix would work for all non-uniform meshes, this 0 sum pitfall is best avoided by selecting a different matrix for the AMG pre-compute steps.

The use of mass matrix M to derive the weighted Laplacian A avoids this potential 0 sum issue, as the integrals of the linear Lagrange basis function multiplications from eq. (2.8) always yield positive values. Therefore, all potentially influencing nodes are considered by the AMG algorithms as their values will be non-zero in the A matrix. Additionally, the mass matrix embeds mesh anisotropy information, similar to how the stiffness matrix embedded gradient information, which leads to better interpolation in regions of strongly anisotropic elements.

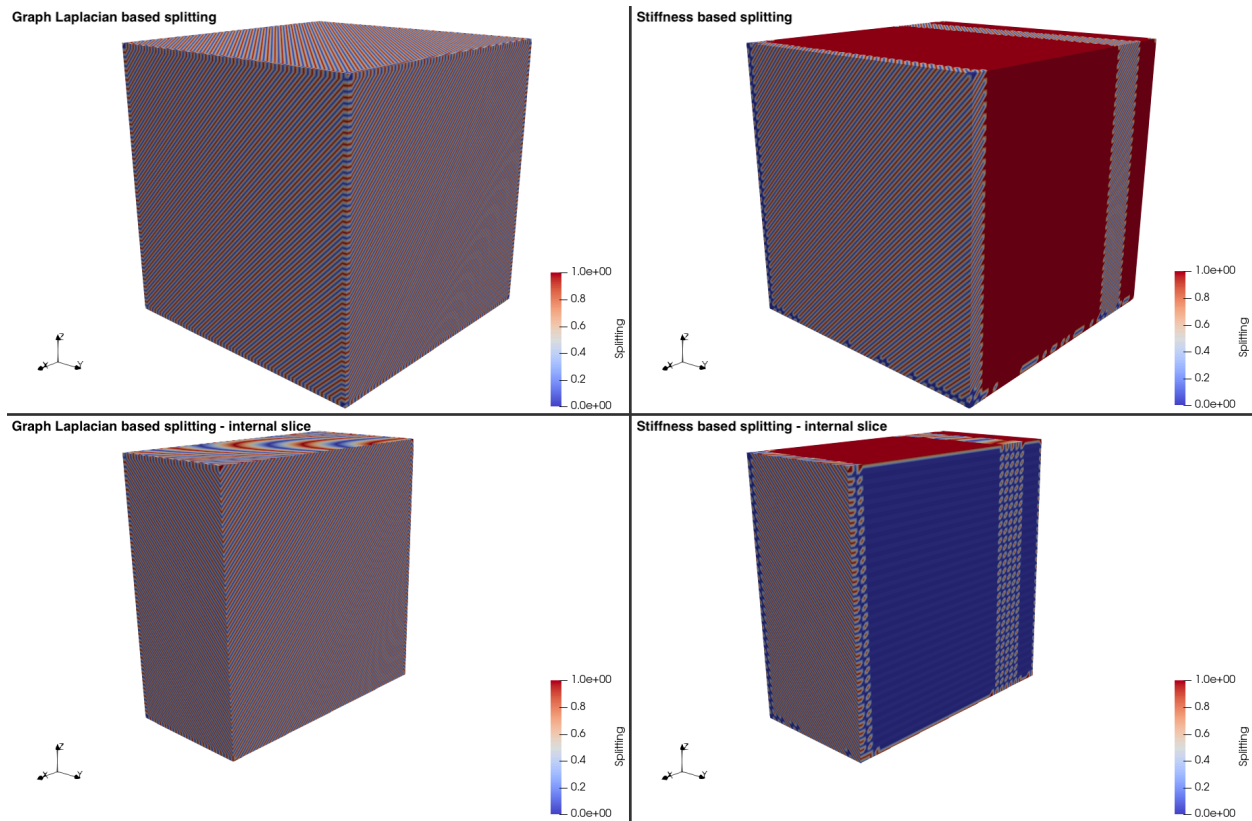


Figure 2.9: Comparison of AMG coarse/fine splitting patterns resulting from mass weighted Laplacian A matrix (left) and Poisson stiffness A matrix (right). Bottom row figures show a cut-through the center of the domain to better expose the "slabs" of fine nodes in the Poisson stiffness matrix splitting case (right).

2.7 Software contributions

The research presented in chapters 3 and 4 stemming from utilizing the AMG multi-level decomposition, required multiple coding language implementations of the decomposition method. A Python implementation was developed to support the machine learning work in chapter 4, while a parallel C implementation within HONEE was developed to support the data compression work in chapter 3.

As mentioned previously, A-MGARD is performed in two steps, a pre-compute step where the hierarchy of function spaces and corresponding operators are determined, and the compute step where those operators are utilized to perform the decomposition. The pre-compute step requires making use of, and modifying, functions from existing AMG solver software packages to extract the interpolation operators, while the multilevel decomposition and recomposition functions were freshly implemented to perform the actions in algorithms 1 and 2.

2.7.1 Python Implementation

An example Python implementation of the full A-MGARD pre-compute and compute steps is available in https://github.com/Jeff-Hadley/MNO_Toolbox.git. This specific implementation makes use of PyTorch [51] tensors for its data structures, however the supporting functions can all be modified to make use of standard Numpy arrays, should future applications require these data structures. The pre-compute step in this implementation is accomplished with a custom hierarchy creation function composed of PyAMG [6] functions. The compute step, which performs the multilevel decomposition and recomposition, was implemented such that it is not limited to PyAMG operators. It can take any hierarchy of prolongation operator matrices, a mass matrix, and original data as inputs, and returns a vector of multilevel coefficients.

The following lists the high level classes or functions in the `MNO_Toolbox` required to

perform the pre-compute and compute steps of the A-MGARD decomposition:

`AMGOperators` class (Pre-Compute), found in `NeuralNetwork_Functions_Optimized.py`.

This class, when an instance is initialized, takes in the fine level mass matrix and builds and stores the entire hierarchy of prolongation, restriction, and Laplacian operators required to perform the multilevel decomposition and recomposition in the compute steps.

`decompose` function (Compute), found in `NeuralNetwork_Functions_Optimized.py`, takes in the stored operator hierarchies in the `AMGOperator` class and the respective data to transform, and outputs a vector of the multilevel coefficients. It applies algorithm [1](#)

`recompose` function (Compute), found in `NeuralNetwork_Functions_Optimized.py` takes in the stored operator hierarchies in the `AMGOperator` class and the multilevel coefficient vector. It reverses the decomposition transform and recomposes the solution in original function space. It applies algorithm [2](#).

All other functions in the `NeuralNetwork_Functions_Optimized.py` are supporting functions for the above 3.

2.7.2 PETSc x HONEE implementation

A parallel capable implementation of A-MGARD decomposition and recomposition is available in the `jhadley/DataCompression` branch of HONEE¹ [\[29\]](#). This implementation makes use of the libCEED [\[11\]](#) and PETSc [\[5\]](#) ecosystems to enable parallel computing. The implementation is embedded into PDE solver HONEE, and serves as an example of how to perform A-MGARD decomposition on state solutions within a running computational simulation. The pre-compute step in this implementation is accomplished by custom PETSc

¹ HONEE source code can be obtained at https://gitlab.com/phypid/honee/-/tree/jhadley/DataCompression?ref_type=heads

functions which hook into BoomerAMG’s [64] hierarchical function space creation processes to extract function space and prolongation operator hierarchies. In short, these custom PETSc functions convert BoomerAMG data structures, which were not natively exposed, to be compatible with HONEE. The multilevel decomposition and recomposition computation steps were implemented with PETSc operators such that they can be executed in a parallel fashion given any assembled mass matrix, prolongation matrix hierarchy, and original data vector.

The following lists the high level functions in the HONEE implementation required to perform the pre-compute and compute steps of the A-MGARD decomposition. All functions listed are found in `src/DataCompression.c`:

DataCompSetup (Pre compute) initializes all required decomposition and recomposition data structures inside a **DataCompression** container, which is housed inside the main **Honee** object. This function also builds the mass matrix from the PETSC DM for a given problem. Note that when building the mass matrix, this function adds back in nodes which may have been removed from the solvers’ computation due to a given boundary condition, i.e all nodes of the domain are included in the A-MGARD process.

DataCompPreCompute (Pre compute) performs all operations related to building the operator hierarchies, calling BoomerAMG/HYPRE functions to perform AMG coarsening. Note that for initial ease of implementation, the mass weighted Laplacian is calculated in HONEE, outside of the scope of BoomerAMG functions. This requires passing the mass Laplacian from HONEE to BoomerAMG, then outputting operators after a single coarsening level, and determining the coarsened mass matrix and subsequent mass weighted Laplacian in HONEE’s scope. The entire hierarchy is determined by repeating this process with the new coarse mass Laplacian.

Ideally, the calculation of coarsened mass and subsequent mass Laplacian matrices

would be performed within the scope of BoomerAMG. This alternate implementation will require substantial modifications to the BoomerAMG source code housed within PETSc. I leave this as optional future work as the method does not require this style of implementation in order to function effectively.

`DataCompDecompose` (Compute), takes in the stored operator hierarchies in the `DataCompression` container and the respective data to transform, and outputs a vector of the multilevel coefficients. It applies algorithm [1](#) in an MPI compatible fashion.

`DataCompRecompose` (Compute), takes in the stored operator hierarchies in the `DataCompression` container and the multilevel coefficient vector. It reverses the decomposition transform and recomposes the solution in original function space. It applies algorithm [2](#) in an MPI compatible fashion.

All other functions in `DataCompression.c` are supporting functions for the above.

2.8 Summary

This chapter presented the derivation and practical implementation of the AMG-based multilevel decomposition method, A-MGARD, for truly unstructured scientific data. The method was developed by identifying and bridging a critical gap between two well-established mathematical frameworks: the multilevel decomposition of Ainsworth et al. [\[1, 4, 3, 2\]](#), and algebraic multigrid methods [\[53, 10\]](#).

The multilevel decomposition framework was first presented as a modification of classical hierarchical decomposition, replacing nodal interpolation operators with orthogonal projections to produce multilevel coefficients that carry a richer multiscale representation of the original function. While powerful, the original framework by Ainsworth et al. required *a priori* knowledge of geometrically nested function space hierarchies and their associated operators, a requirement that fundamentally limits its applicability to meshes generated

through congruent element refinements. To remove this restriction, AMG was introduced as a means of algebraically determining the required hierarchy of function spaces, prolongation operators, and mass matrices directly from the mesh’s weighted graph Laplacian, without any assumptions on how the mesh was originally created.

The amalgamation of these two frameworks into A-MGARD was then completed by incorporating the AMG-derived operators into the multilevel decomposition’s recursive projection scheme. Practical implementation considerations were discussed, including the use of classical Ruge-Stuben coarsening and interpolation schemes for computational efficiency, a row-sum scaling prolongation smoothing procedure to enforce the partition of unity requirement, and the use of efficient solvers for the resulting mass matrix systems. The resulting decomposition and recomposition algorithms were presented, producing multilevel coefficients $\mathbf{u}_{mc}$ that occupy no more storage than the original solution vector $\mathbf{u}$. The algorithms recover the original solution within machine precision when no modifications are made to those coefficients.

Implementations of A-MGARD were made available through two software contributions. A Python implementation, built on the PyAMG and PyTorch ecosystems, is available as a standalone toolbox and serves as an accessible reference implementation of the full pre-compute and compute steps of A-MGARD. A parallel-capable implementation embedded within the PDE solver HONEE, leveraging the libCEED and PETSc ecosystems, demonstrates how A-MGARD can be deployed within a running computational simulation. Together, these implementations lower the barrier to entry for future adoption of A-MGARD in both research and engineering contexts.

Importantly, while A-MGARD was motivated by and presented in the context of unstructured data compression, the decomposed multilevel coefficients $\mathbf{u}_{mc}$ represent a broadly useful multiscale representation of the original data. As will be demonstrated in the following chapters, these coefficients can serve as inputs to a conventional lossy compression pipeline for unstructured scientific data, or as the basis for learning data-driven operators in a more

expressive multiscale function space for machine learning applications.

2.9 Future Work

Extension to higher order basis functions As it stands, the A-MGARD decomposition method starts from a solution living in a function space of linear Lagrange basis functions. We hypothesize that an even richer decomposition could be obtained by performing a two-step decomposition. First performing hierarchical decomposition in polynomial degree, starting with higher order basis function spaces and arriving at linear functions, and then proceeding with the current A-MGARD decomposition.

Matrix Free Regarding computational cost and overhead, future efforts should be put towards optimization of the decomposition and recomposition methods, specifically towards implementing a matrix free version of the operations. The current implementation, which relies on external AMG packages, requires that the mass matrix be fully assembled in order to determine the function space hierarchies and interpolation operators. A matrix free method could significantly reduce the storage requirements of the method while also providing computational speed up of the decomposition and recomposition operations.

Chapter 3

Application of AMG based multilevel decomposition to data compression

3.1 Background and Motivation

Advances in computational technology has fostered an environment where the rate of scientific data creation has far surpassed the ability to stream or store it sustainably. To meaningfully utilize and store the large volume of scientific data created by modern computing systems, it must be carefully reduced to efficiently pass through the comparatively small available bandwidth in the data I/O communication infrastructures, while maintaining accuracy and precision to the original data. Many data compression methods have been developed over the past few decades with variable levels of success, depending on the nature of the data. Unstructured scientific data poses a unique challenge regarding compression. The compressed data must be accurate to the original data to be of meaningful use, while the unstructured nature of the data renders majority of currently available spatial compressors ineffective in obtaining substantial levels of compression.

Current state-of-the-art lossless compression methods such as ZSTD, FPZIP, GZIP, BLOSC, FPC, SPDP [18, 42, 20, 8, 12, 16] ensure perfect adherence to original data; however, they are highly limited in the amount of compression they can perform, usually with compression factors less than 1.2x as demonstrated in [40]. This small amount of compression is unsustainable for the sheer volume of scientific data being produced. As such, greater compression savings are desired while maintaining an adequate amount of fidelity to the original data. Error-bounded lossy compression methods are a solution to this problem,

providing greater reductions in total data while maintaining an allowed upper bound on the reconstruction error between the compressed and original data.

Error-bounded lossy compression methods typically follow a decomposition, quantization, encoding, and lossless compression pipeline. While errors are only introduced by the quantization step, the amount of error and its relative amount of compression is highly dependent on the quality of the data passed from the decomposition step to the remainder of the pipeline. One of the principle distinguishing features between existing error-bounded lossy compression techniques is how they perform this decomposition step. Most recent cutting edge lossy scientific data compression method, SZ3 [39], adaptively chooses to perform this decomposition in a transform-based or prediction-based manner. SZ3 utilizes variants of ZFP [41] and SZ [55] for transform-based and prediction-based decomposition, respectively.

These decomposition methods permit the remainder of the pipeline to achieve great levels of compression; however, they are primarily developed to operate on data originating from regularly gridded meshes, also referred to as structured meshes. SZ3 has the capability of operating on unstructured meshes by treating data as a 1 dimensional array [39]; however, its compression performance diminishes due to the reduced correlation between the data's computational domain node locations and their storage locations in the 1 dimensional array, as will be shown in this research.

Transform-based compression method MGARD [22] was developed to address decomposition of data on unstructured grids. Compared to SZ3 and ZFP, MGARD works with unstructured data directly, although it is in actuality restricted to block-structured meshes as was presented in chapter 2. These meshes are obtained through successive levels of congruent element refinements, given by standardized geometric refinement pattern for a given simplex [22] [4] [2]. While these 'geometrically nested' meshes are globally unstructured in the classical sense, i.e. their nodes are not required to follow a gridded spacing, they are block-structured due to the congruent element refinement operations. This block-structure is the key in determining MGARD's enforceable error bounds during data , and is required

for the method to be applicable [22].

Many complicated geometries such as aircraft, vehicle suspension components, robotics components, etc. require body conforming meshes to perform finite element engineering analyses of their designs. The meshes used in these finite element analyses are often created with multiple isolated areas of element refinements, various regions of mixed element types, and wall-normal element scaling. These meshes thus require a breadth of element size, shape, and freedom in placement of their vertex nodes. Evidently, geometric structuredness of data is essentially lost for these types of problems.

To address the lack of lossy compression methods for truly unstructured data, we proposed to utilize the decomposition method presented in chapter 2 within existing data compression pipelines. As it was designed, A-MGARD’s AMG based multilevel decomposition is applicable to all types of meshes; structured, unstructured, mixed elements, regardless of how they were originally created, i.e. nested refinements are not required. This decomposition enables the use of typical data compression pipelines on unstructured data by extending MGARD’s geometric multigrid derived decomposition to be applicable to truly unstructured meshes.

Results from implementing our decomposition into data compression pipelines suggest that AMG-based multilevel decomposition for spatial compression of unstructured scientific data provides a viable alternative to the decomposition stage of contemporary compression methods. The remainder of this chapter will present the testing and results of AMG based multilevel decomposition applied to data compression, and compare it’s performance against geometric MGARD and SZ3 when applicable.

3.2 Investigation of AMG based multilevel decomposition compression performance

An investigative approach was taken to evaluate the performance of the proposed AMG-based multilevel decomposition against current state-of-the-art compression methods

applicable to unstructured meshes, namely MGARD and SZ3. The test cases in this chapter utilize a series of increasingly complex meshes, starting with a structured hex mesh, which is then incrementally modified to increase its complexity until it is unstructured. Each compression test consisted of performing decomposition of the original data $\mathbf{u}$, interpolated onto a given mesh, utilizing various decomposition methods, and passing the resultant coefficients to the default quantization, encoding, and lossless compression pipeline available in SZ3.

The following subsections will describe overall testing method and

3.2.1 General compression testing set-up and procedure

A sweep through varying amounts of maximal allowable error from the quantization, encoding, and lossless steps is performed for each possible decomposition method and mesh case pairing (where applicable), and the compression ratios achieved by the pipeline were recorded. The resulting compressed coefficients outputted by the pipeline were then used in the recomposition steps to obtain the compressed solution, $\tilde{\mathbf{u}}$, and compute realized compression error metrics. Errors introduced in the reconstructed data were measured as described in section [3.2.4](#)

As we seek to compare our multilevel decomposition method to current state-of-the-art, additional tests are performed where the original solution field is passed directly to SZ3. In these test, SZ3 determines its best available decomposition and passes those coefficients to the remainder of the compression pipeline. Both the 3D and 1D SZ3 schemes were used where applicable. These tests are representative of the currently best available method for truly unstructured domains.

3.2.2 Source data and original mesh configuration

A series of 2D and 3D data compression tests were performed using cutouts of the JHTDB DNS channel flow data set [\[33, 23\]](#) to test the capabilities of the proposed method. The selected data cutouts were obtained by taking $257 \times 257 \times 17$ and $129 \times 129 \times 65$ node

blocks (Cutout indices: $x = [0, 256]$, $y = [0, 256]$, $z = [0, 16]$ and $x = [0, 128]$, $y = [0, 128]$, $z = [0, 64]$ respectively) at time step 3100. The general region of these cutouts is highlighted by the green outline in fig. 3.1.

The data used in these tests consists of u_x , u_y , and u_z velocity data fields. Each velocity field for a given mesh is treated as a separate set of data $\mathbf{u}$ in the decomposition, i.e. each field is individually compressed.

It should be noted that the original JHTDB Channel Flow mesh is regularly spaced in x and z directions, and is variate in y , with highly anisotropic elements in the near wall boundary layer region, as can be observed in fig. 3.3a.

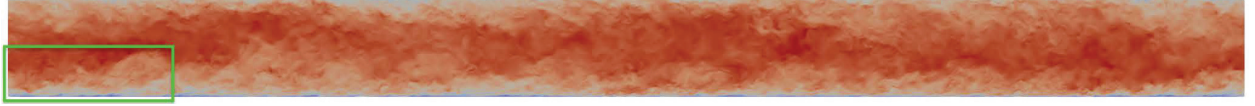


Figure 3.1: Full JHTDB Channel Flow domain, x velocity at time step 3100. Green box indicates the approximate location of the data cutouts for the data compression tests.

3.2.3 Modifying original meshes for induced unstructuredness

As mentioned above, various additional test meshes are created by a combination of nodal order permutations, element shape variations, and randomly perturbing the nodal locations of the original JHTDB mesh. Nodal order permutations were performed to remove potential physical position inductive bias from data location in memory, as the original JHTDB data follows x, y, z ordering and has highly correlated nodal physical domain position and storage index location in memory. The randomized permutations are performed to removing the correlation between a nodes' physical domain position and its location in memory, as unstructured meshes have low correlation between node numbering and physical location.

For nodal position perturbations, individual random nodal position perturbation, dx , dy , and dz , are limited to a maximum magnitude of 25% of the minimal original x, y , or z

dimension of their connected elements to ensure elements are not inverted. Original domain boundary nodes are only perturbed in directions which maintain them on their original boundary edges and faces to conserve the original domain volume; ie face nodes can't be perturbed in a direction normal to that face, and edge nodes can only be perturbed along the edge.

For element shape variations, the original hex meshes were converted to tetrahedral meshes by means of a Delaunay triangulation scheme in MATLAB. The final nodal numbering and positions after modifications (if applied) were passed to MATLAB's delaunay function, from which resulting connectivity information was used for A-MGARD decomposition.

Specific descriptions and nomenclature of the various resulting meshes are provided in the subsequent result sections. The mass matrices used in the A-MGARD decomposition were constructed from the new nodal numbering, connectivity, and nodal positions from each resulting mesh.

3.2.4 Measuring induced error from compression

The errors introduced during the compression process were measured and inspected *a posteriori*. Errors induced in the reconstructed solution $\tilde{\mathbf{u}}$ from compressed data were quantified by peak signal-to-noise ratio, *PSNR*, through the following formula:

$$PSNR = 20 * \log_{10} \left(\frac{\sqrt{n_L} (\max(\mathbf{u}) - \min(\mathbf{u}))}{\|\tilde{\mathbf{u}} - \mathbf{u}\|_{l^2}} \right)$$

PSNR values are inversely proportional to allowable error and achievable compression ratios. The larger the *PSNR* value is, the smaller the allowable error and resulting compression ratios, and vice versa.

The amount of compression achieved during each of the sweeps is reported as bit rate. Bit rate is the average number of bits used per data point in the compressed representation

of the data, and has units of [bits/value]. Working with 64 bit precision data (doubles), the bit rate for these compression tests is defined as:

$$\text{Bit Rate} = \frac{64 \text{ bit}}{CR}$$

where CR is the compression ratio defined as:

$$CR = \frac{\text{size of original data}}{\text{size of compressed data}}$$

As the proposed method only performs the decomposition step in the compression pipeline, the compression ratios obtained are those from quantization, encoding, and loss steps of the pipeline.

3.3 Testing effect of mesh on SZ3 errors

The following test aims to show the degradation of SZ3's performance when applied to increasing levels of data unstructuredness. The performance degradation, as will be shown, further motivates the practical use of A-MGARD in compression pipelines. This test make use of the original 129x129x65 and 257x257x17 JHTDB hexahedral meshes, and each mesh is modified to have a varying amount of unstructuredness as follows:

- **Ordered, dimension informed SZ3 (Ord)** make use the default dimension informed 3D decomposition scheme available in SZ3. This leverages the structured x,y,z ordering of the data for it's decomposition schemes. Makes use of original JHTDB mesh, as seen in fig. [3.3a](#).
- **Ordered, dimension informed SZ3 on perturbed nodal positions (Ord Pert)** makes use of the same dimension informed decomposition as Ordered, however the physical positions of mesh nodes have been perturbed and data fields to be compressed have been interpolated onto the new nodal positions using a natural-neighbor interpolation scheme. Example of this type of mesh can be seen in fig. [3.3b](#).

- **Randomized data order (Shfl)** makes use of the default 1D adaptive decomposition scheme available in SZ3. This scheme performs prediction or transform decomposition on the data, u , based on internal compression optimization heuristics. The data passed to SZ3 is first transformed from a 3D to 1D array, and the node numbering is then permuted such that x,y,z spatial ordering correlation is lost in the 1D array.
- **Randomized data order on perturbed nodal positions (Shfl Pert)** makes use of the same process as Shfl, however this is applied to the randomly perturbed nodal position solutions.

3.3.1 Testing approach

A compression percentage sweep test was performed on the turbulent flow channel data and mesh configurations described in section 3.3 on both the 129x129x65 and 257x257x17 cutouts. The x , y , and z velocity fields were individually compressed for each test case.

It is important to note that a key limitation of SZ3 is its focus on structured and block-structured data sets, where data storage locations in memory are highly correlated to their computational domain spatial locations. While SZ3 can operate on unstructured data by processing the data as a 1D sequence of values, its compression performance is highly reduced the more uncorrelated the data becomes in sequential memory storage, as is evident in the following results.

3.3.2 Mesh effect results

Figure 3.2 shows the effects each modification case had on SZ3's compression performance for both the 129x129x65 the 257x257x17 configurations. The trends are consistent between the two mesh size results, therefor the following discussion is implied for both.

Introduction of nodal perturbations causes a relatively small decrease in performance,

seen by the decrease in PSNR for the same bit rate in the Ord and Ord Pert cases. This small decrease is expected, as relative spatial to storage correlation is maintained. Once spatial correlation is lost, SZ3 experiences a large performance decrease, as seen in the Shfl and Shfl Pert cases. Further introduction of nodal perturbations has minimal effect, as Shfl and Shfl Pert cases have nearly identical performance.

The main take away from this test is that the largest contributor to SZ3's performance lies in the amount of correlation between its input datas' physical node locations and storage layout. These results emphasize the need for a method which is robust to node ordering, as unstructured meshes cannot guarantee that physically correlated data points are also correlated in memory.

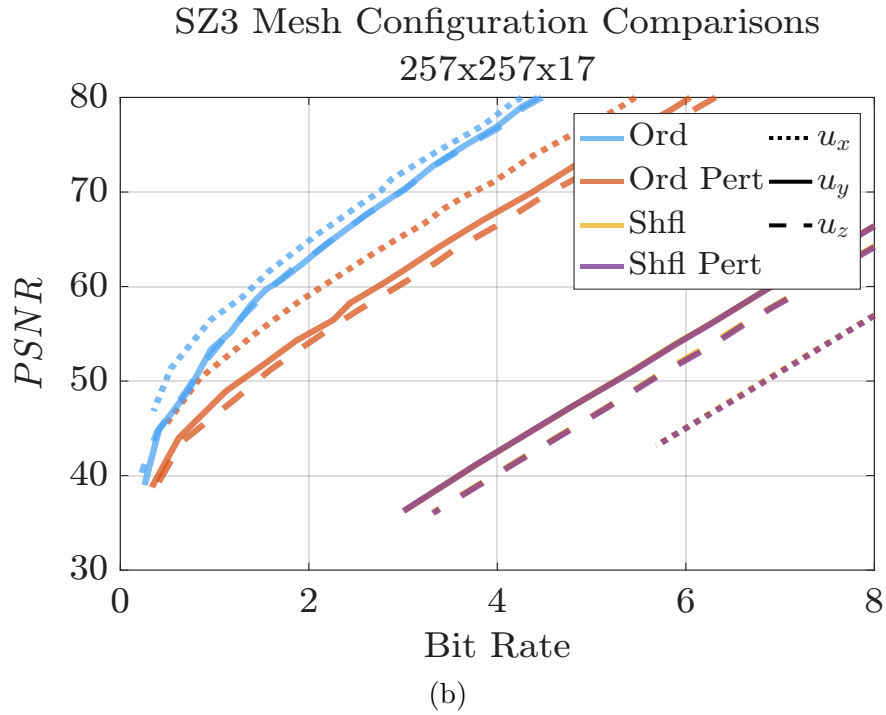
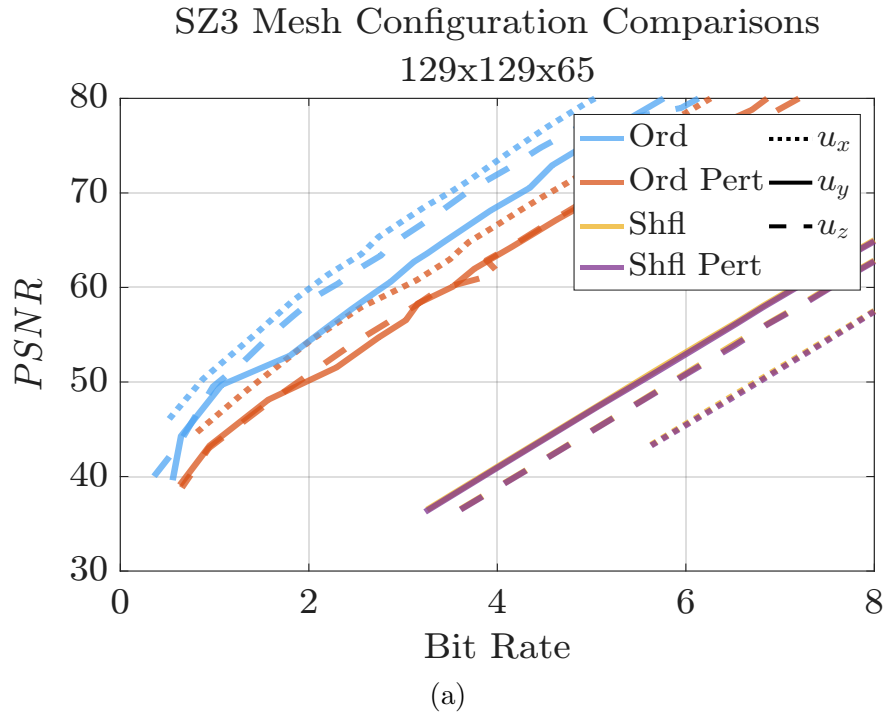


Figure 3.2: Comparison of mesh configuration effects on SZ3 compression performance. PSNR is plotted against bit rate. High PSNR and low bit rate is best. a) 129x129x65 node meshes b) 257x257x17 node meshes

3.4 Effect of decomposition methods and mesh type on compression errors

The tests in this section aims to determine the feasibility of the AMG-based decomposition method, when compared to it's geometric based counterpart, MGARD. The following compression tests utilized 2 decomposition methods for comparison:

- **AMG based multilevel decomposition (A-MGARD)** utilized AMG prolongation operators along with the multilevel projection solve (algorithm 1) to determine the multilevel coefficients u_{mc} to be compressed.
- **Geometric-multigrid based multilevel decomposition (G-MGARD)** utilized nested element bisection refinement interpolation as presented in 3 along with the multilevel projection solve (algorithm 1) to determine the multilevel coefficients u_{mc} to be compressed. This decomposition is akin to wavelet decomposition.

3.4.1 Decomposition method testing approach

Compression percentage sweep tests were performed on the turbulent flow channel data for the decomposition methods described above, where applicable. Each test made use of either the original JHTDB hexahedral mesh nodal positions as previously described, a perturbed hexahedral mesh, or various unstructured tetrahedral meshes.

Visual examples of these various meshes can be seen in ??, and further descriptions are available in sections 3.4.3 and 3.4.4.

The original JHTDB data interpolated onto these new meshes is stored in a 1D array, following a structured 3D x-y-z indexing system. Because unstructured meshes generally lack a consistent relationship between spatial location and memory ordering, we apply randomized node-number permutations to the mesh and correspondingly permute the data, thereby removing strong spatial correlation from the 1D storage sequence. Thus, we would like to emphasize that **all tests in section 3.4 are performed using permuted data**, which is akin to the SZ3 cases labeled Shfl in section 3.3. The mass matrices required for

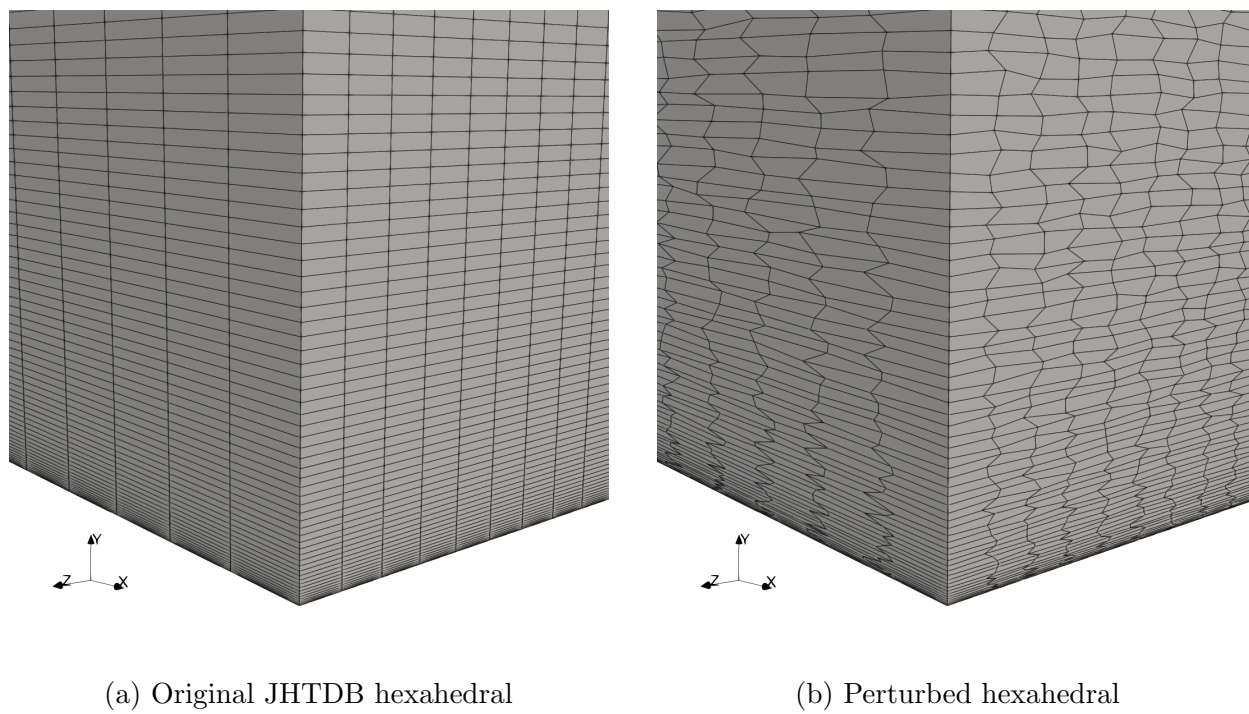


Figure 3.3: Comparison of hexahedral mesh configurations.

the decompositions are thus assembled for each mesh using the corresponding permuted connectivity information. The resulting x , y , and z velocity fields from interpolating original JHTDB data onto each mesh are individually compressed for each decomposition method. The following sections will each further describe the specific purpose of the tests and what combination of decomposition method and mesh were used.

3.4.2 A-MGARD vs G-MGARD on original JHTDB hex mesh

The following tests serve as a viability study between the proposed A-MGARD decomposition method and existing G-MGARD decomposition. We perform two separate tests on the original JHTDB mesh to establish a comparison on meshes where G-MGARD is applicable, i.e. the finest mesh can be treated as the result of a nested refinement pattern.

3.4.2.1 Averaged compression performance on 2D planes of 3D domain

As a simple starting point, the first tests were performed on 2D x-y plane slices of the Original JHTDB 3D domains. Each 2D slice was individually compressed using either A-MGARD or G-MGARD for its decomposition method in the compression pipeline, and the resulting *PSNR* and bit rates were averaged over all slices; 65 slices for the 129×129 domain and 17 slices for the 257×257 domain, as seen in figs. [3.4a](#) and [3.4b](#) respectively. A-MGARD decomposition has very similar performance as G-MGARD for all data fields in both the 129×129 and 257×257 cases.

3.4.2.2 Compression performance on 3D domain

To increase complexity, the subsequent decomposition method tests were performed on the original 3D mesh domains. The results for the $129 \times 129 \times 65$ and $257 \times 257 \times 17$ hex mesh cases can be seen in figs. [3.5a](#) and [3.5b](#), respectively.

For the $129 \times 129 \times 65$ domain in fig. [3.5a](#), A-MGARD and G-MGARD have near identical performance, with A-MGARD slightly outperforming G-MGARD in the low bit rate

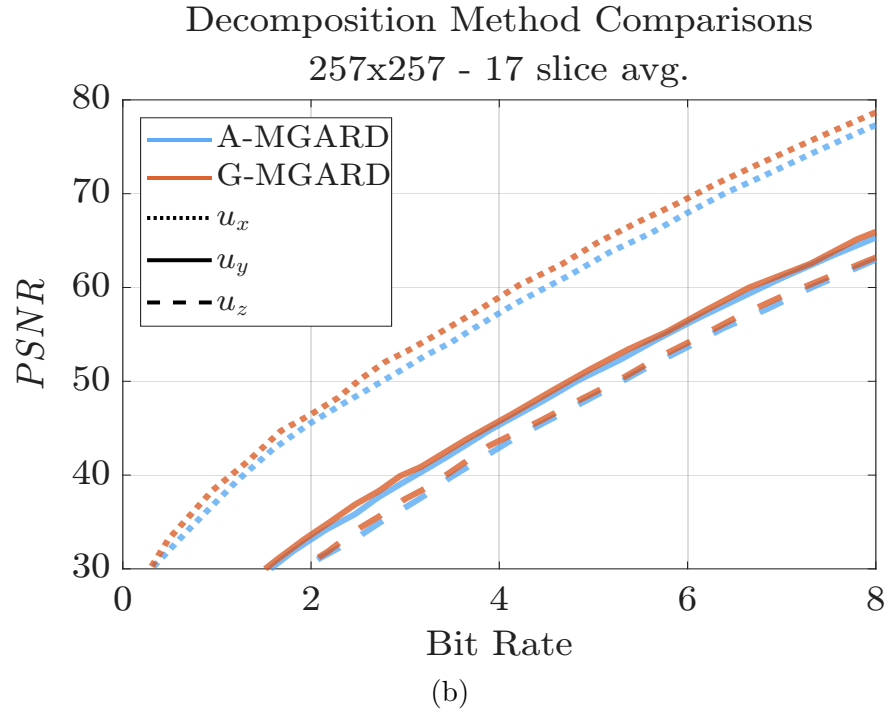
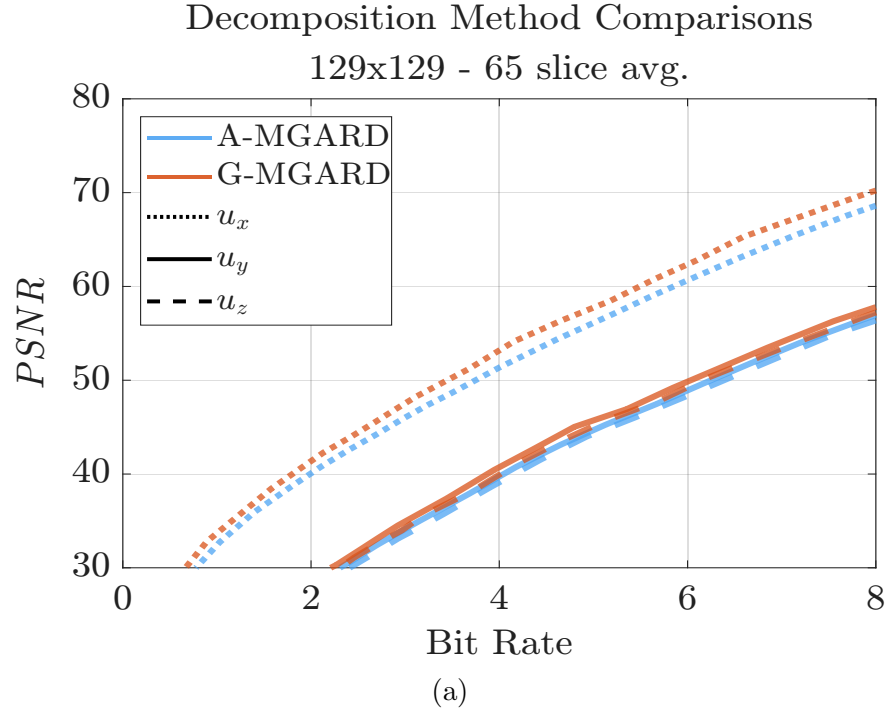


Figure 3.4: Comparison of compression methods on 2D slices of original hexahedral mesh. PSNR is plotted against bit rate. a) Average of 65 129×129 slices, b) Average of 17 257×257 slices

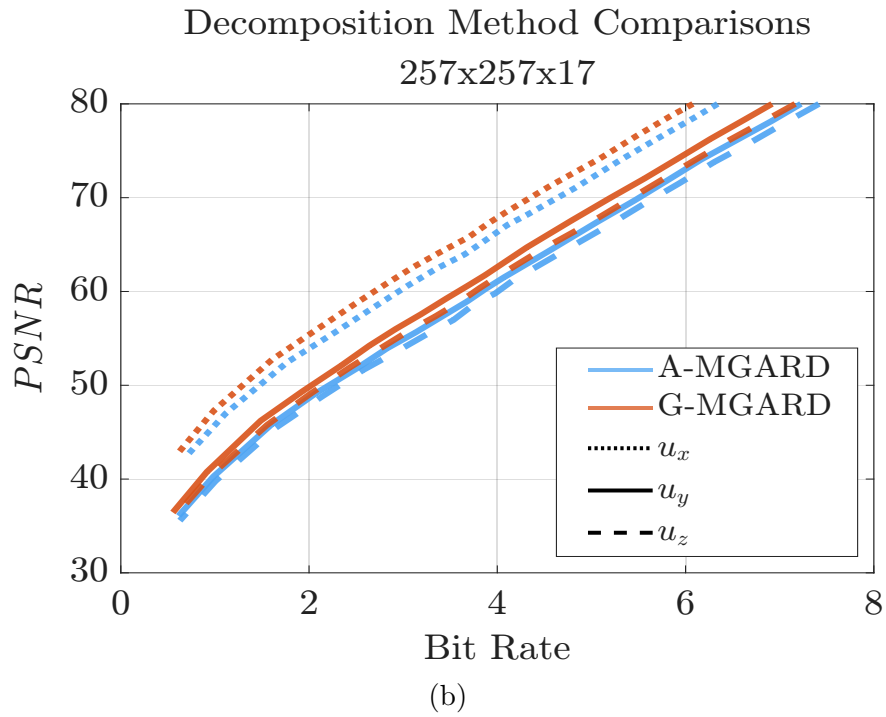
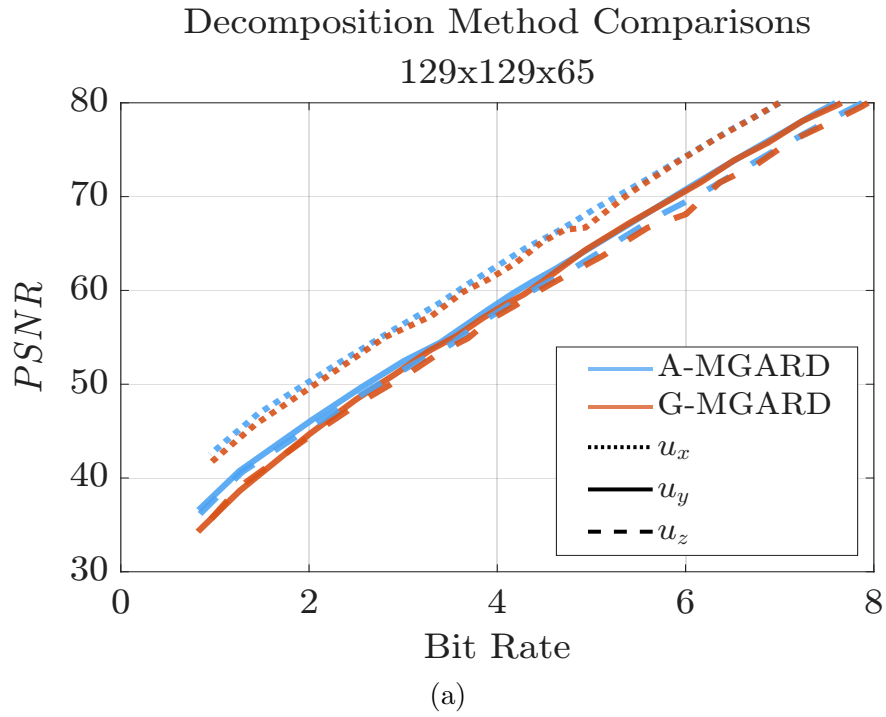


Figure 3.5: Comparison of compression methods on original hexahedral meshes. PSNR is plotted against bit rate. a) $129 \times 129 \times 65$, b) $257 \times 257 \times 17$

regime. Observing the $257 \times 257 \times 17$ case, G-MGARD slightly outperforms A-MGARD throughout the compression sweep. The narrower $257 \times 257 \times 17$ mesh appears to be a more favorable domain for both decomposition methods, as they obtain better errors throughout bit rate sweep range compared to their $129 \times 129 \times 65$ mesh counterparts. Interestingly, G-MGARD experiences a larger performance decrease in low bit rate range as the mesh goes from the narrow, and 2D plane dominated, domain of the $257 \times 257 \times 17$ mesh to a more balanced 3D domain with the $129 \times 129 \times 65$ mesh, i.e. when comparing individual field results across figs. [3.5a](#) and [3.5b](#).

Note that the $129 \times 129 \times 65$ mesh is composed of a much larger percentage of highly anisotropic elements than the $257 \times 257 \times 17$ mesh. This is indicative that A-MGARD is more adaptable to highly anisotropic meshes than G-MGARD, as the function spaces used in the decomposition are created by a mesh-informed interpolation determined AMG. With A-MGARD and G-MGARD exhibiting performance parity in both the averaged-2D and 3D domains, we consider A-MGARD to be a capable alternative decomposition method for data compression, while also being more generalizable than G-MGARD methods.

More impressively, as these results are obtained from permuted data, comparing A-MGARD's Perturbed performance to that of SZ3 Shfl Pert from fig. [3.2](#) shows an improvement in compression ratio by up to a factor of approximately 3.3, 1.8, and 1.8 for the u_x , u_y , and u_z fields respectively in the $129 \times 129 \times 65$ mesh case, and 4.6, 2.3, and 2.4 for the u_x , u_y , and u_z fields respectively of $257 \times 257 \times 17$ mesh case.

3.4.3 A-MGARD applied to perturbed hexagonal mesh

To further show the capability of the proposed decomposition method in handling unstructured data, a secondary hexahedral mesh is created by randomly perturbing the nodal locations of the original JHTDB mesh as previously described in section [3.2.1](#). The resulting perturbed mesh is composed of heavily skewed hexahedral elements, as seen in fig. [3.3b](#). As physical nodal locations of the mesh are modified, the original solution fields are interpolated

onto the new perturbed mesh using natural-neighbor interpolation. The interpolated fields are therefore separate solution $\mathbf{u}$ data vectors than those from the original JHTDB solution fields.

Due to the spatial perturbations, G-MGARD is no longer applicable as elements can no longer be properly nested, and any resulting “coarsened meshes” no longer maintain standard trilinear spaces required by G-MGARD. As such, we now look at the effect that perturbing the mesh nodes has on A-MGARD decomposition itself.

The results for the $129 \times 129 \times 65$ and $257 \times 257 \times 17$ perturbed meshes can be seen in figs. 3.6a and 3.6b respectively.

The introduction of node perturbation resulted in a slight reduction in PSNR for the same bit rate when compared to the original mesh cases. This reduction in compression performance is expected, as the elements have become skewed, especially in the highly anisotropic near wall boundary layer region of the flow. This further decreases the mesh quality, effecting mass matrices and the resulting interpolation operators and multilevel coefficients quality. Even though a reduction is expected when going from original node positions to perturbed ones, the A-MGARD method is shown to be more robust to mesh perturbations when compared to the SZ3 only results from fig. 3.2. A-MGARD and SZ3 had approximate decreases of 3 dB and 7 dB, respectively.

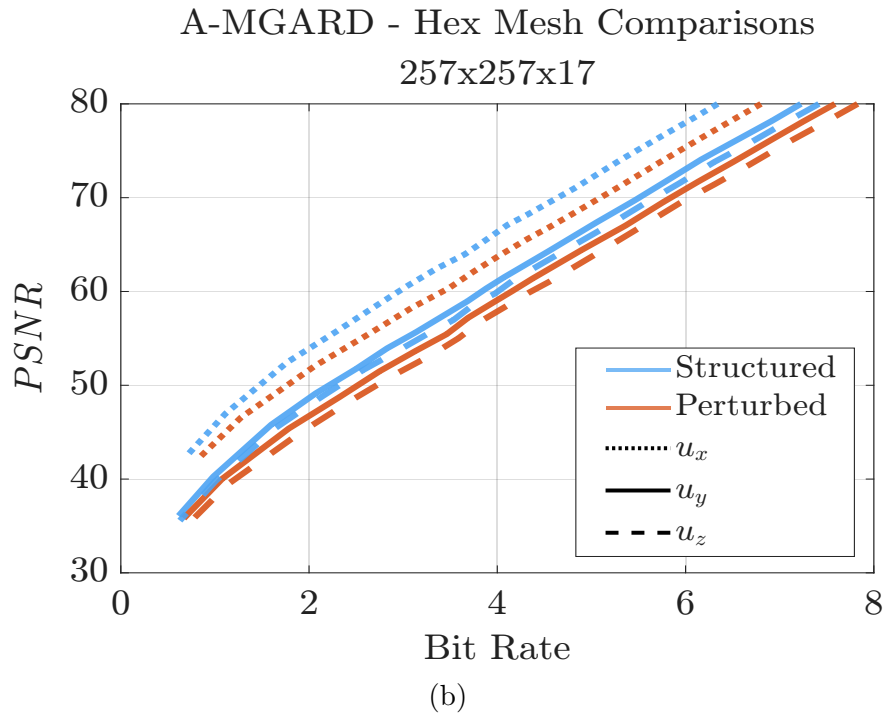
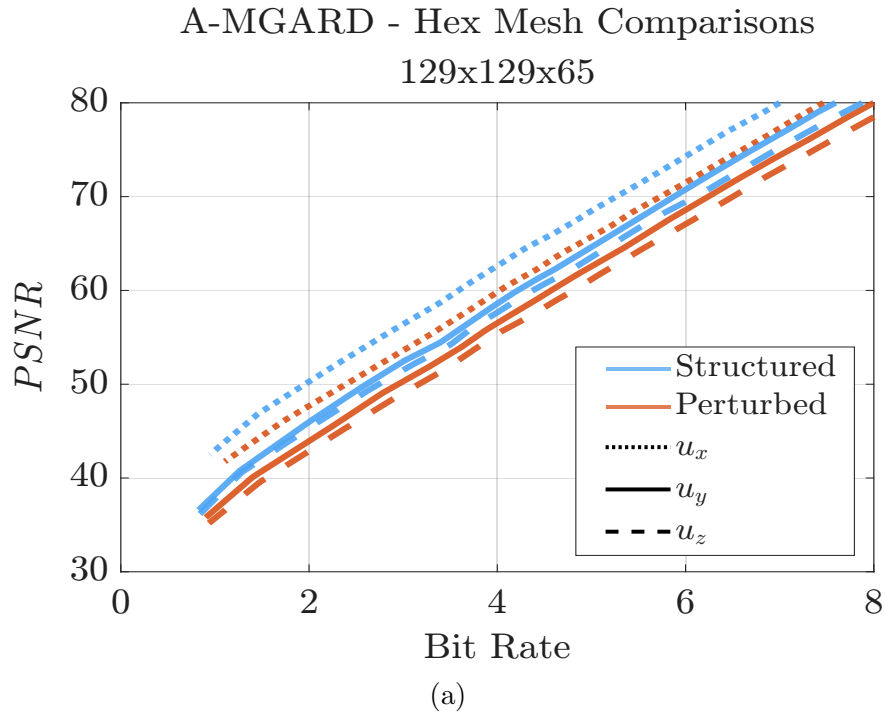


Figure 3.6: Node perturbation effects on A-MGARD applied to hexahedral meshes. PSNR is plotted against bit rate. a) $129 \times 129 \times 65$, b) $257 \times 257 \times 17$

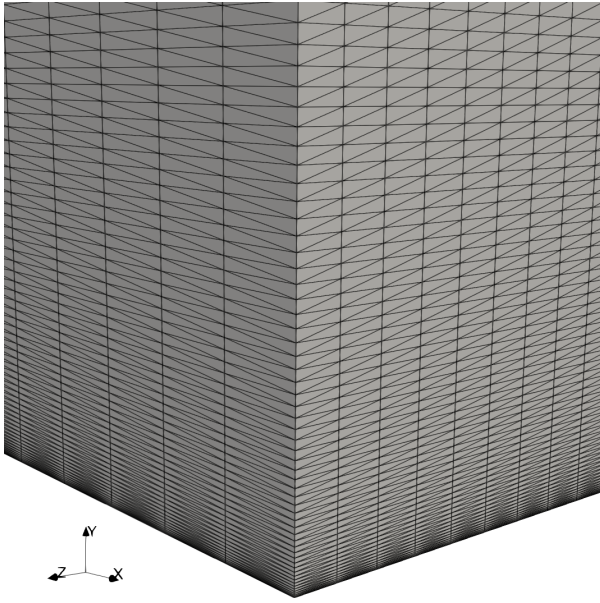
3.4.4 A-MGARD applied to unstructured tetrahedrals

To further test the unstructured capability of the A-MGARD method, several unstructured tetrahedral meshes are generated by combining various nodal perturbations, global node numbering permutations, and applications of Delaunay triangulations to the original JHTDB meshes. Meshes resulting from modifying the original 129x129x65 hex mesh will be identified as 'Block' meshes, and those from the original 257x257x17 hex mesh as 'Slab' meshes. The following list describes the various tetrahedral meshes as shown in fig. 3.7 and how they were obtained:

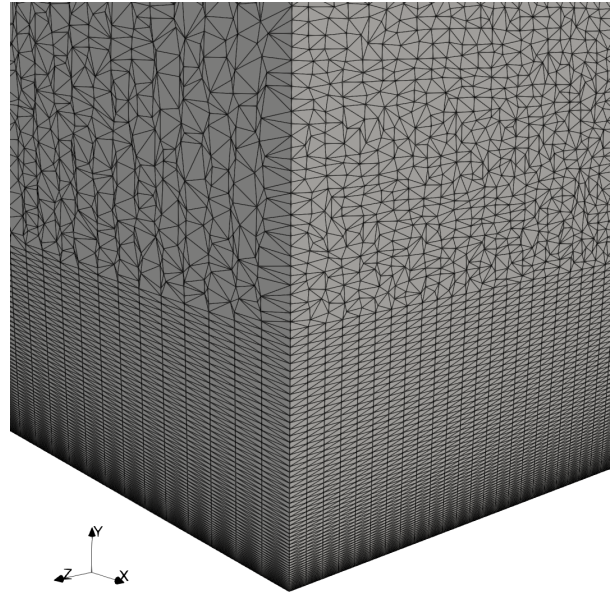
- **Original node location tetrahedral mesh (Tet)** is obtained by applying a randomized global node numbering permutation, and applying a Delaunay triangulation to the resulting numbering on the original JHTDB nodal positions. A visual example of this mesh can be seen in fig. 3.7a.
- **Structured boundary layer and unstructured freestream (Tet BL)** is obtained by applying a randomized global node numbering permutation, perturbing nodes which are above a specified boundary layer cut-off position, and applying a Delaunay triangulation. The boundary layer cut-off is taken as z index 65 for both the block and slab meshes. A visual example of this mesh can be seen in fig. 3.7b.
- **Delaunay triangulation followed by full domain nodal position perturbations (Tet-Pert)** is obtained by applying a randomized global node numbering permutation, applying a Delaunay triangulation to the resulting numbering, and perturbing the entire domain nodal locations. Nodes on domain faces and edges are only perturbed in a direction which keeps them on their original face planes and edge lines such that the domain volume is unchanged. A visual example of this mesh can be seen in fig. 3.7c.
- **Full domain nodal position perturbations followed by Delaunay triangu-**

lation (Pert-Tet) is obtained by applying a randomized global node numbering permutation, perturbing the entire domain nodal locations, then applying a Delaunay triangulation to the resulting nodal numbering and positions. The same volume preservation is applied to the face and edge nodes as with Tet-Pert. A visual example of this mesh can be seen in fig. [3.7d](#).

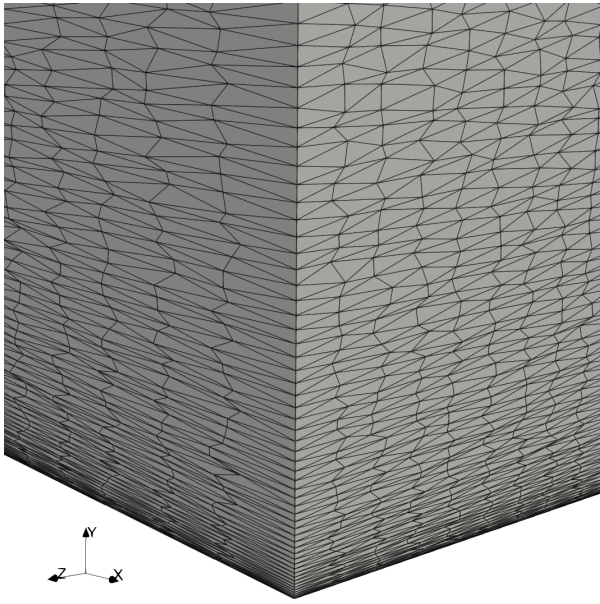
Again, when physical nodal locations of the mesh are modified, the original solution fields are interpolated onto the new tetrahedral meshes using natural-neighbor interpolation. The resulting interpolated fields are separate solution $\mathbf{u}$ data vectors than the previously interpolated fields.



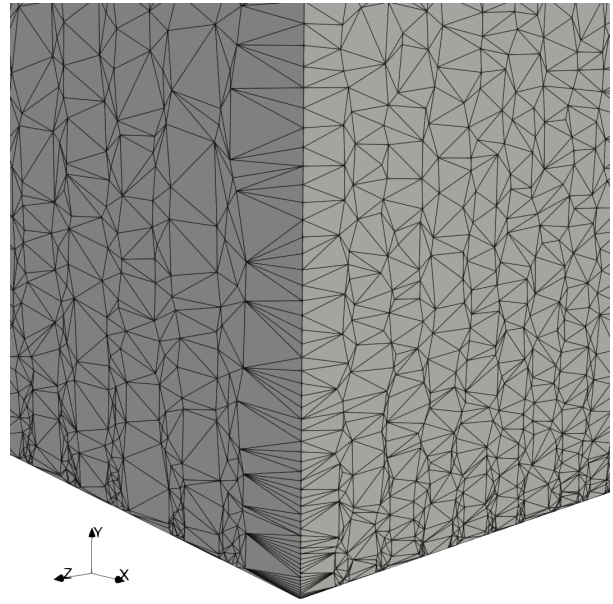
(a) Original node location tetrahedral mesh (Tet)



(b) Structured boundary layer and unstructured freestream (Tet BL)



(c) Delaunay triangulation followed by full domain nodal position perturbations (Tet-Pert)



(d) Full domain nodal position perturbations followed by Delaunay triangulation (Pert-Tet)

Figure 3.7: Comparison of tetrahedral mesh variations.

3.4.4.1 Tet vs Hex

We first compare how the Tet mesh effects the A-MGARD method when compared to the original JHTDB hexahedral mesh. These results can be observed in fig. [3.8](#). The Tet case experiences a small but expected decrease in performance when compared to the Hex case. While the Tet mesh has the same nodal positions as the Hex mesh, this decrease is attributed to an increase in number of mesh elements. The more abundant tet elements are smaller and far more anisotropic than the original hex elements. This contributes to an increase of integration error when building the mass matrices. The subsequent interpolation operators built upon the mass matrices will be slightly deteriorated by this error, thus effecting the multilevel coefficients obtained from decomposition and the subsequent compression and solution recompositions.

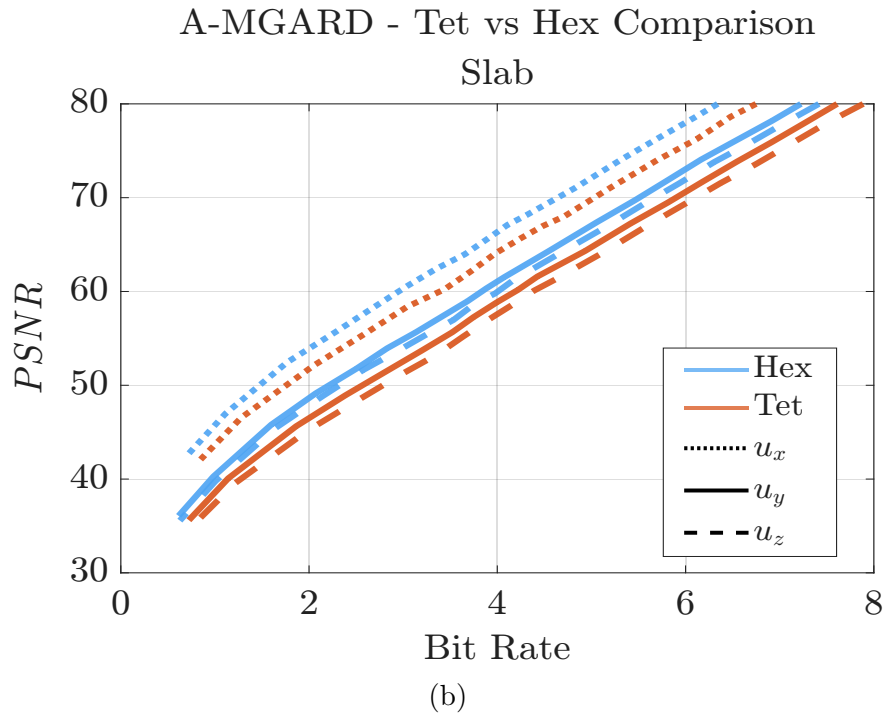
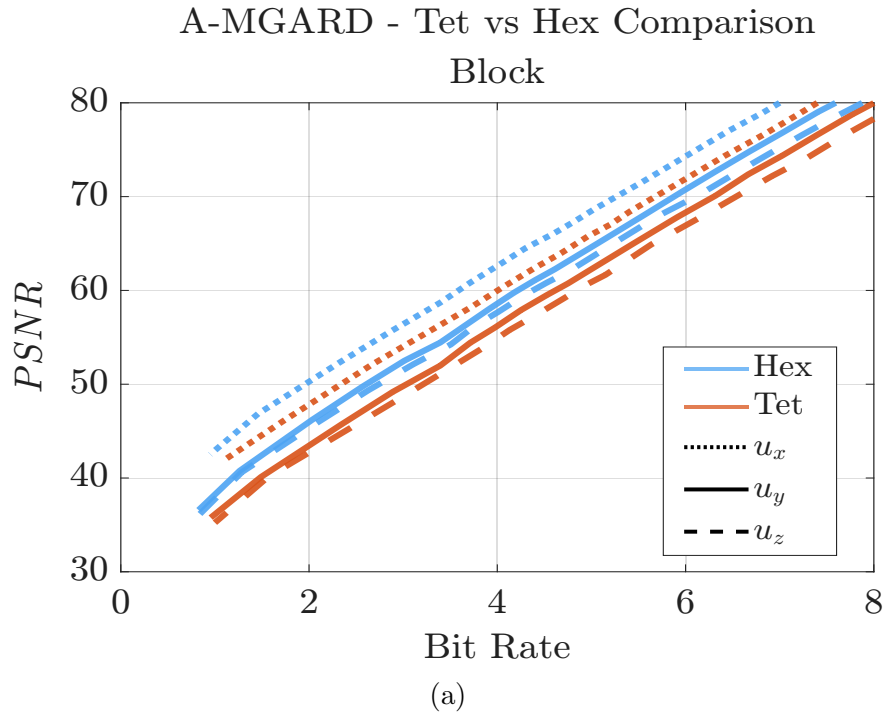


Figure 3.8: Effect of Tet mesh on A-MGARD. PSNR is plotted against bit rate. a) $129 \times 129 \times 65$, b) $257 \times 257 \times 17$

3.4.4.2 Increasing tet mesh unstructuredness

Next, we compare compression results from the Tet mesh to the Tet BL, Tet-Pert, and Pert-Tet meshes in order to observe A-MGARD's robustness to increased nodal perturbations and element anisotropy. The Tet case is considered to be the best case scenario, as all mesh elements are well structured and are equally anisotropic.

The Tet BL cases makes use of a mesh which is most representative of meshes used in practice, with a structured element region in the near wall boundary layer, and unstructured elements in the freestream region of a flow.

The Tet-Pert and Pert-Tet are worst case scenario meshes, with full domain unstructuredness, and heavily skewed and anisotropic tet elements, and are expected to have small performance degradations. Particularly, the Pert-Tet mesh has highly varying element sizes and connectivity in the x direction, shown in fig. 3.7d and is considered to be a torture test for the A-MGARD method since computational simulations generally avoid using such ill-constructed meshes.

The expected gradual decrease in compression performance can be observed as the meshes becomes more and more unstructured in each flow fields results in figs. 3.9 to 3.11. The u_y and u_z fields in figs. 3.10 and 3.11 are similarly effected by increased unstructuredness for both the Block and Slab meshes, demonstrating relatively small decreases in performance with increasing unstructuredness.

The Tet-Pert and Pert-Tet u_x results in fig. 3.9 show greater relative performance decreases, however this is expected as the u_x data itself is highly anisotropic, thus compounding the effects of the ill-constructed meshes. Most importantly, a similar performance decrease is observe for the u_x Tet BL cases as were observed in the u_y and u_z fields. As shown in fig. 3.2, comparing the worst-case A-MGARD Pert-Tet results against the SZ3 Shfl-Pert results, A-MGARD achieves greater PSNRs across all field. For the Block ($129 \times 129 \times 65$) mesh, the gains are approximately 16, 12, and 11 dB in the x, y, and z fields respectively. For

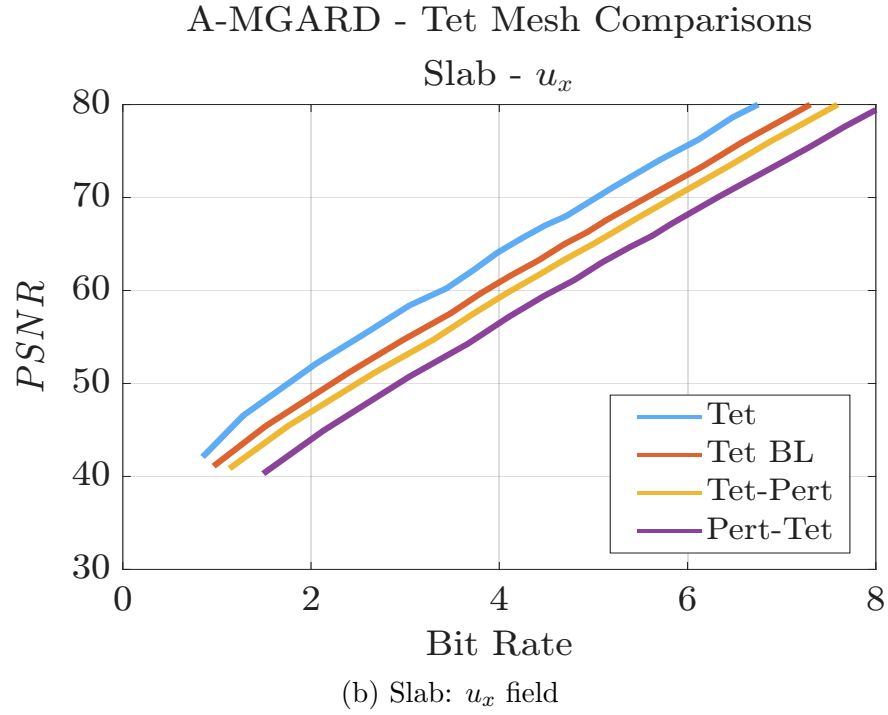
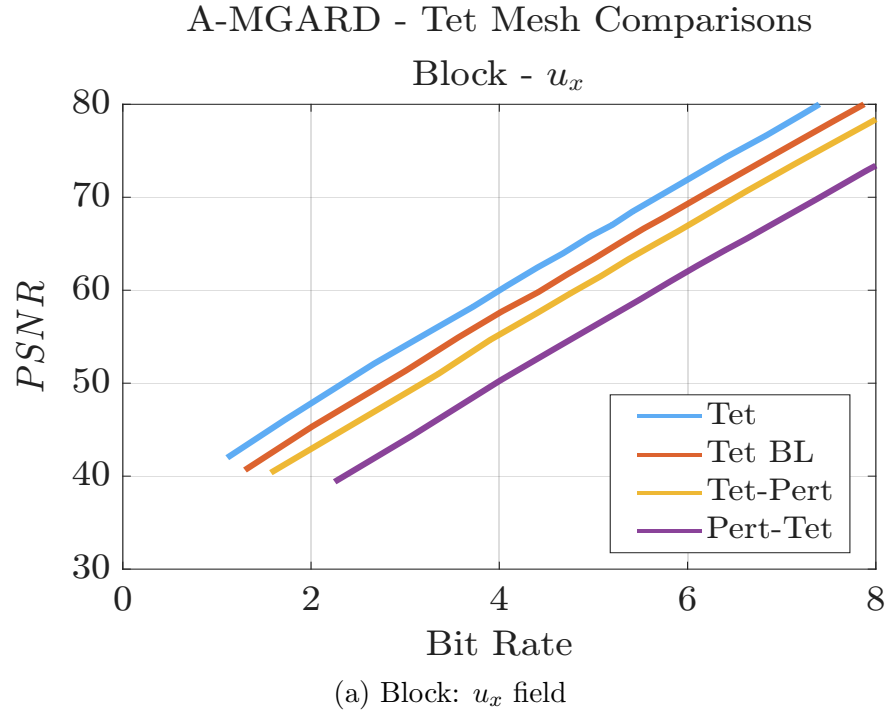


Figure 3.9: Comparison of u_x velocity field compression results between a) Block and b) Slab tetrahedral meshes.

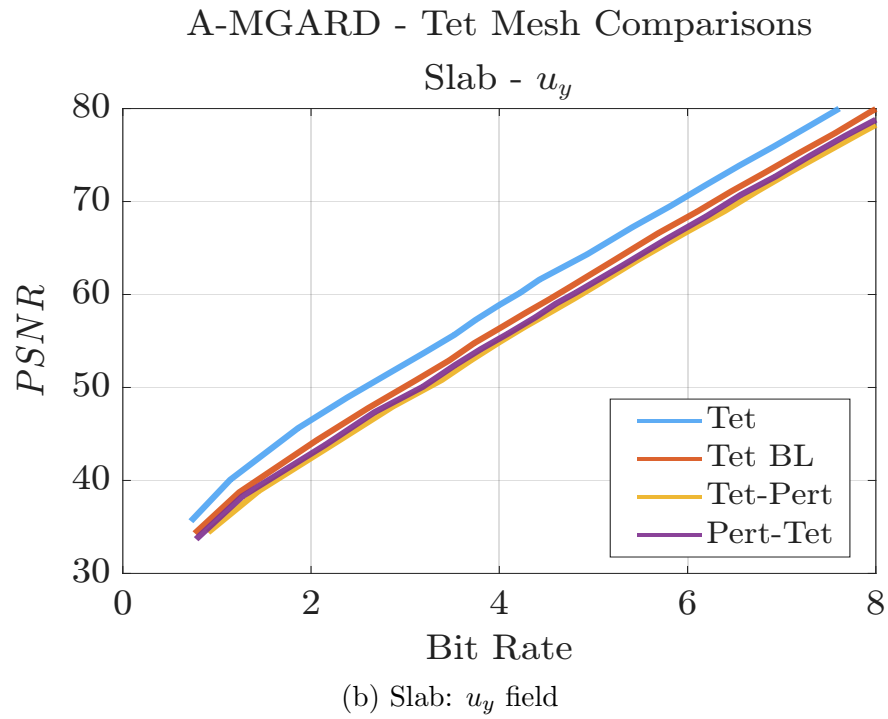
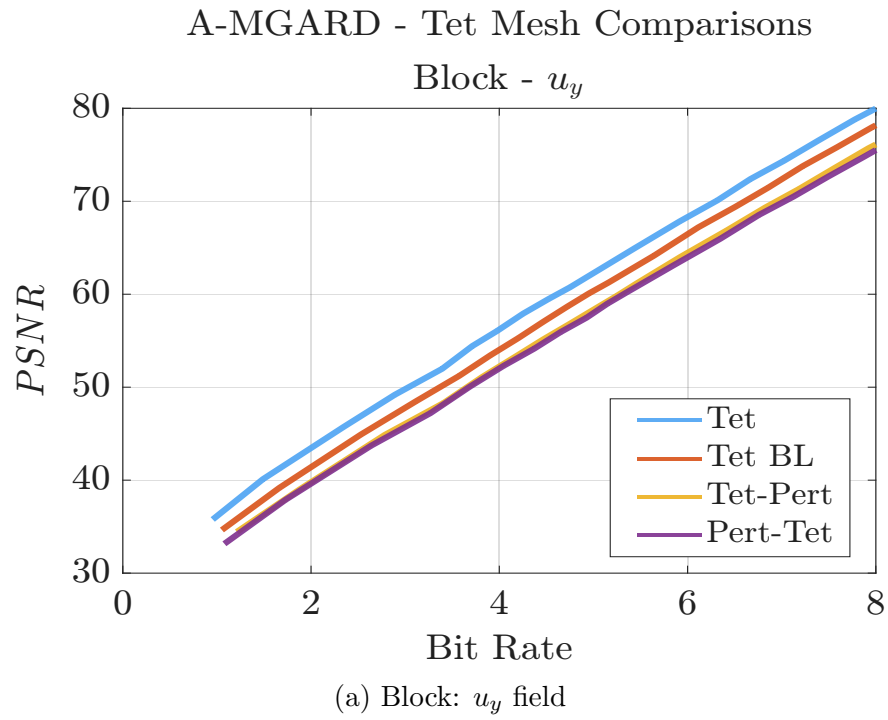


Figure 3.10: Comparison of u_y velocity field compression results between a) Block and b) Slab tetrahedral meshes.

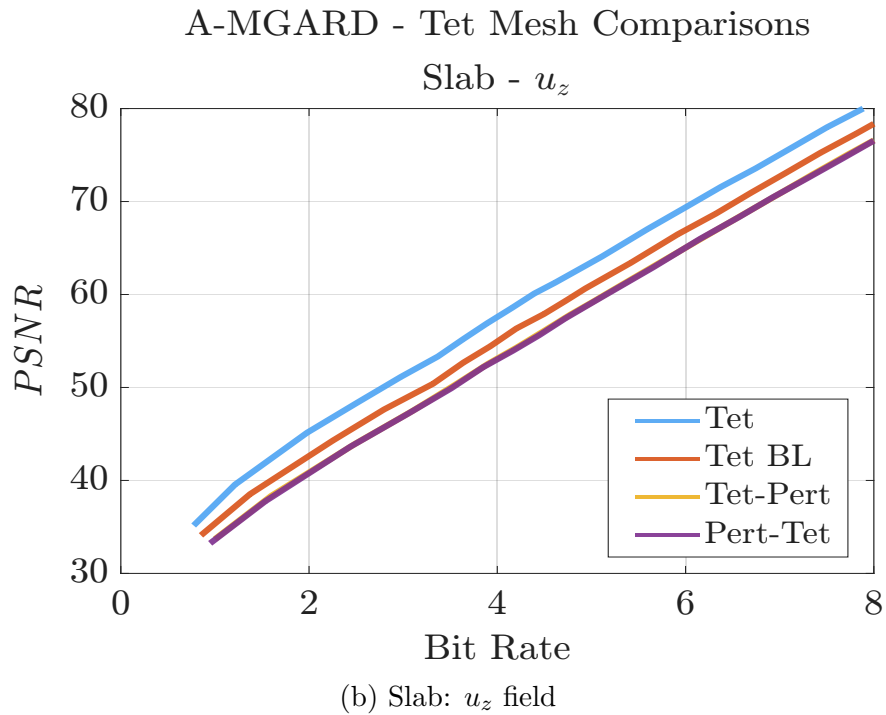
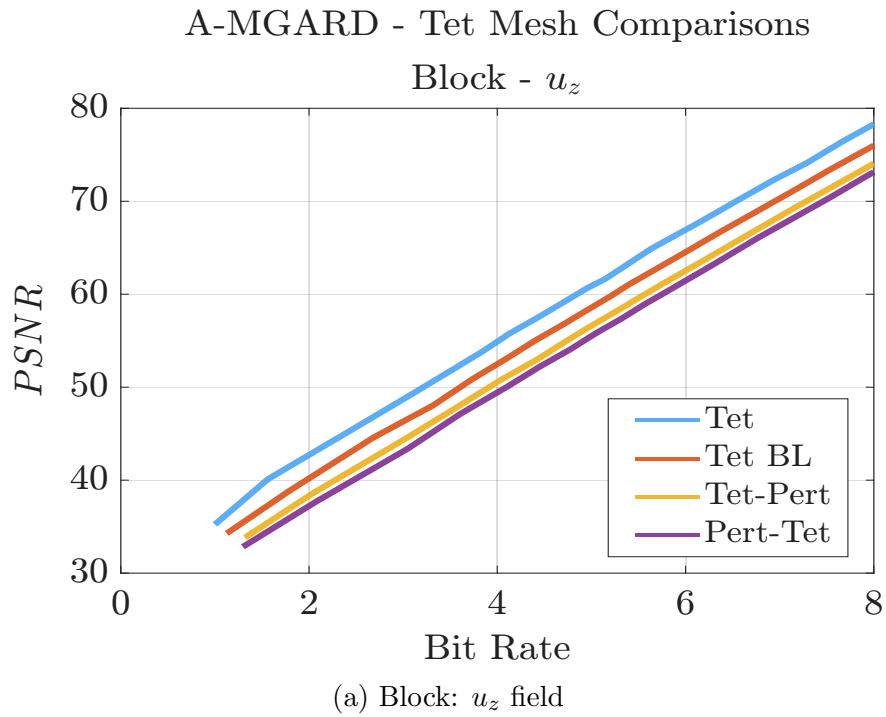
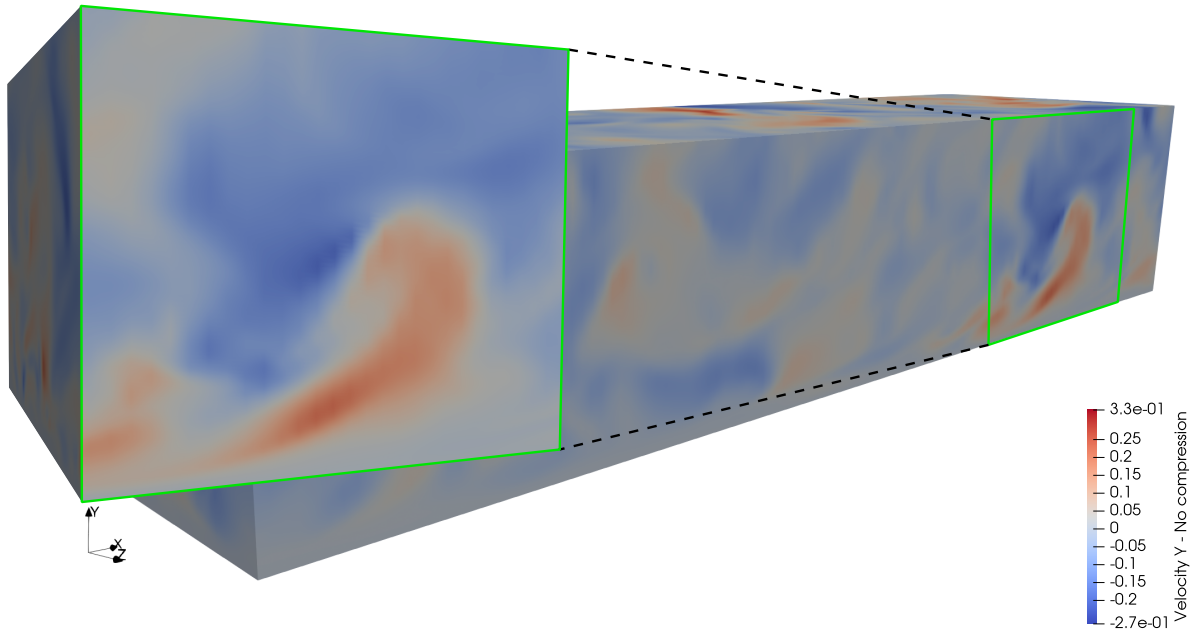


Figure 3.11: Comparison of u_z velocity field compression results between a) Block and b) Slab tetrahedral meshes.

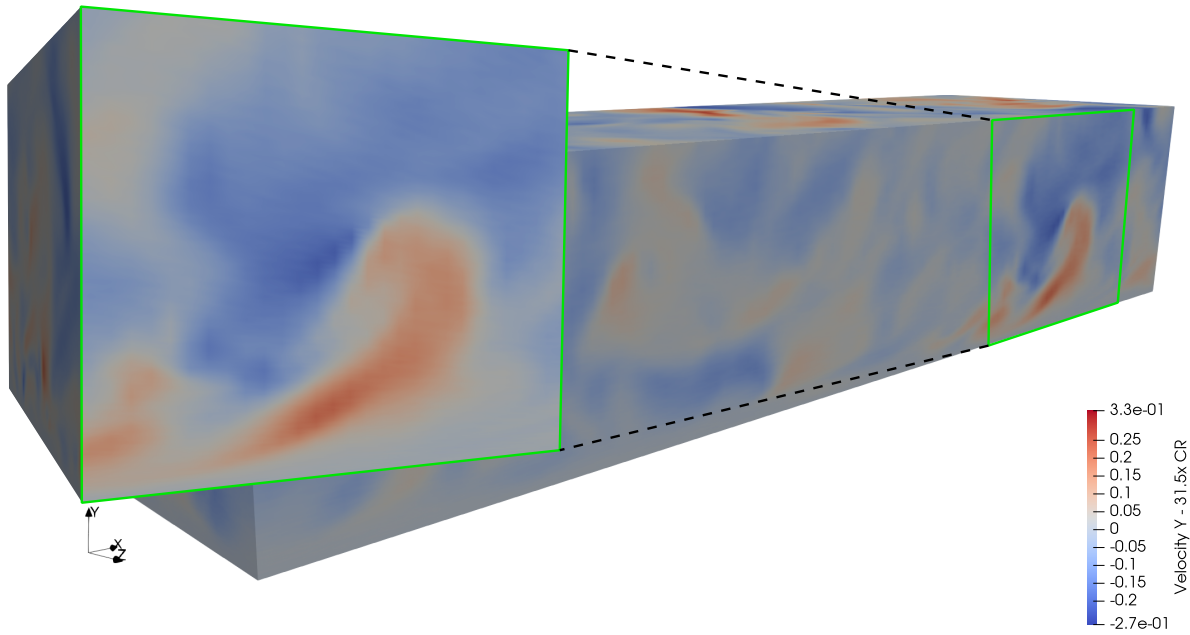
the Slab ($257 \times 257 \times 17$) mesh, the corresponding gains are approximately 23, 13, and 13 dB. These results indicate that the A-MGARD method is robust to unstructured meshes most likely to be used in scientific simulations.

3.5 Compressed solution visualizations

Additionally, A-MGARD decomposition for compression demonstrates great original solution adherence. Visualizations of the original and compressed fields for the $129 \times 129 \times 65$ u_y A-MGARD decomposition case and the $257 \times 257 \times 17$ perturbed u_z A-MGARD decomposition case can be seen in figs. 3.12 and 3.13 below. Figure 3.12 shows a visualization with a PSNR of 46.2 dB for a compression ratio of 31.5x, while fig. 3.13 shows a lower PSNR of 39.4 dB with higher compression ratio of 56.3x. As can be observed in the figures, A-MGARD decomposition performs exceptionally well in maintaining large and small scale features of the highly turbulent channel flow. Observing the green expanded region inlays, little to no compression artifacts can be observed in the higher PSNR (lower error, lower compression) visualization, and minimal artifacts are observable in the lower PSNR (higher error, higher compression) visualization.

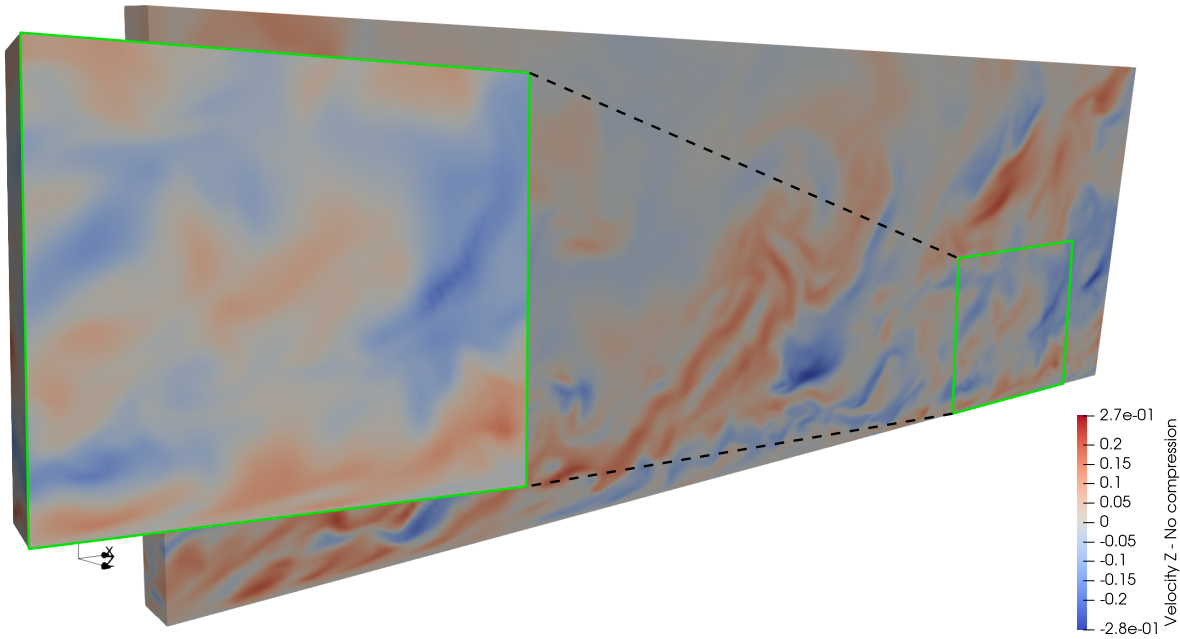


(a) Original u_y field, $129 \times 129 \times 65$ node mesh

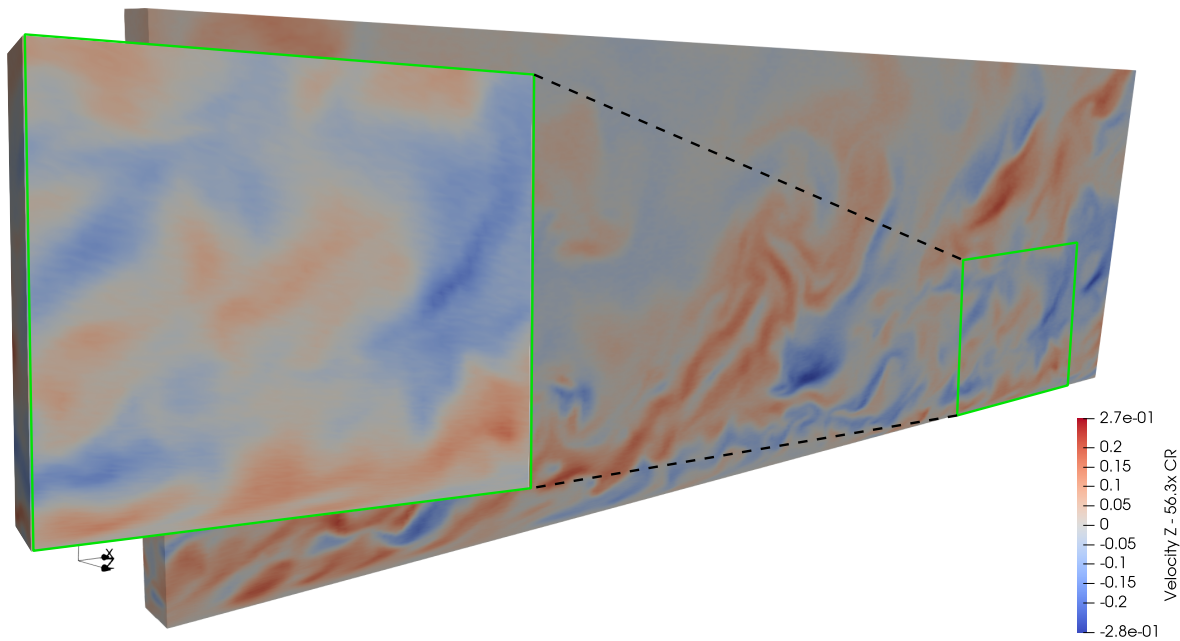


(b) Compressed u_y field, 46.2 dBPSNR @ 31.5x CR, $129 \times 129 \times 65$ node mesh

Figure 3.12: Visualization of A-MGARD decomposition compressed fields vs original, non-compressed field (u_y).



(a) Original u_z field, $257 \times 257 \times 17$ node perturbed mesh



(b) Compressed u_z field, 39.4 dBPSNR @ 56.3x CR, $257 \times 257 \times 17$ node perturbed mesh

Figure 3.13: Visualization of A-MGARD decomposition compressed fields vs original, non-compressed field (u_z).

3.6 Storage considerations in the context of data compression

As mentioned in section [2.4.3](#), the A-MGARD decomposition requires storing L sparse and progressively smaller prolongation operators as well as the original sparse mass matrix of the mesh in order to reconstruct the compressed solution in the original function space. When considering that the A-MGARD decomposition is to be applied over multiple time steps of a numerical simulation, these storage costs become negligible in comparison to the overall data savings that are achievable by transferring and storing compressed coefficients.

3.7 Summary and conclusions

This chapter presented the application of A-MGARD as a substitute decomposition method within a standard *decomposition, quantization, encoding, and lossless* pipeline for unstructured scientific data. Testing was performed on turbulent channel flow velocity data from the JHTDB dataset [\[33, 23\]](#), using a series of increasingly complex and unstructured mesh configurations derived from the original structured hexahedral JHTDB mesh. This complexity progression, from structured hexahedral, to perturbed hexahedral, to various tetrahedral configurations, was designed to systematically evaluate A-MGARD's robustness to increasing levels of mesh unstructuredness.

As a first step, the sensitivity of SZ3's compression performance to data ordering and mesh unstructuredness was established, confirming that SZ3's performance significantly degrades once spatial correlation between a node's physical location and its storage location in memory is lost. This motivated the need for a decomposition method that is agnostic to data storage ordering, which A-MGARD is by design.

Next, comparisons between A-MGARD and G-MGARD on the original structured hexahedral mesh demonstrated performance parity between the two methods across both 2D and 3D test domains, with A-MGARD showing greater robustness on the more anisotropic 129x129x65 mesh. This is a significant result, as it establishes A-MGARD as a capable

alternative to G-MGARD on meshes where both are applicable, while also being applicable to a broader class of truly unstructured meshes G-MGARD is not.

As mesh complexity was increased through nodal perturbations and Delaunay triangulations, A-MGARD demonstrated modest and expected performance degradation, maintaining substantial advantages over SZ3 even in the worst case Pert-Tet mesh configurations. In the worst case, A-MGARD achieved PSNR gains, for the same amount of compression, of approximately 20, 12, and 11 dB over SZ3 in the x , y , and z fields for the Block mesh, and 23, 13, and 13 dB for the Slab mesh. Compression factor improvements of 1.8x to 4.6x over SZ3 on randomly permuted data (A-MGARD from fig. 3.5 compared to SZ3 Shfl from fig. 3.2) further highlights the practical value of incorporating A-MGARD into compression pipelines for unstructured domain data. Solution visualizations confirmed that A-MGARD decomposition maintains excellent fidelity to original turbulent flow fields, with little to no compression artifacts observable at higher PSNR values, and only minimal artifacts at higher compression ratios.

Together, these results demonstrate that A-MGARD decomposition provides a viable and generalizable alternative to the decomposition stage of contemporary compression methods, with significant practical implication for data transfer and storage in unstructured domain computational simulation frameworks.

3.8 Future Work

The application of the A-MGARD decomposition in a data compression setting, as presented in this dissertation, is but a small subset of possible research which could bring significant improvements to the method.

Quantization Further research and development efforts could be applied to determining quantization methods specifically tailored to the multilevel coefficients of A-MGARD decomposition. A better quantization would lead to increased compression performance as this is the current step introducing errors in the compression pipeline.

Enforceable error bounds While the use of A-MGARD as a decomposition step in the SZ3 pipeline returns better compression results for the same amount of error as using SZ3 on unstructured data, there is not currently a provable upper bound on the expected error in the reconstruction of the compressed solution. This author believes that a provable error bounds could be determined for reconstruction in original solution space from the compressed multilevel coefficients, as the function spaces generated by the AMG based multilevel process hold orthogonality properties, and the compressed multilevel coefficients returned by SZ3 themselves already have a provable bound.

This work is beyond the scope of this dissertation, and could potentially be a dissertation in itself. I thus leave this as a proposed research avenue for an applied mathematician better equipped to determine this bound than myself.

Chapter 4

Application of AMG based multilevel decomposition to Neural Operators

4.1 Neural operator background and motivation

Neural operators, a sub class of operator learning architectures [7, 15], have become popular in learning surrogate models for non-linear PDE solvers in a data-driven approach. Neural operators are specifically classified as such by the requirement that they are discretization invariant while also being universal approximators [31]. As originally envisioned, the theoretical goal of neural operators is to learn a mapping operator between infinite-dimensional function spaces, from pairs of input and output data, such that the model is resolution invariant. In learning operators between infinite-dimensional function spaces, neural operators have the potential to be quite powerful, and are envisioned to serve as efficient alternatives to classical PDE solvers, where they can be used as time evolution predictors given an initial condition. For example, a neural operator would serve as a surrogate model of Navier stokes equations, where simulating flow over an airfoil could be achieved in a fraction of the time currently required by iterative PDE solvers.

Advancements in this field have yielded various neural operator architectures, such as the spatial methods of graph neural operators (GNO) [36] and multipole graph neural operators (MGNO) [37], the spectral methods of Fourier neural operators (FNO) [35] and wavelet neural operators (WNO) [56], and the branch-trunk architecture of deep operator networks, DeepONet [44]. The methods of the former two families all seek to satisfy universal approximation [26, 14] through a shared strategy [31] of parameterizing a kernel integral

operator via a layered neural network, where composition with pointwise nonlinear activation yields the expressive capacity to approximate continuous operators. Whether this property is fully realized depends on the architectural choices made in parameterizing the kernel, a point we return to in section [4.2.3.2](#).

GNO and MGNO take a graph based approach, and parametrize the kernel integral operator in physical space. This permits them to be truly mesh-independent and well suited for unstructured domain discretizations. In contrast, FNO and WNO opt to parameterize the kernel integral operator in transformation space, capturing desirable multi-scale features. This permits them to have an expressive and compact model architecture, resulting in highly efficient PDE operator models when compared to fully solving those same PDE's via classical methods such as FEM.

While, in theory, these neural operators can learn the approximation model on one discretization and apply it to a different discretization, in practice, these theoretical properties come with implementation constraints that limit their applicability in ways that are worth carefully considering. In the realm of spatial methods, graph neural operators, while truly being discretization invariant and being applicable to arbitrary domain discretizations, incur infeasible computational and storage costs with growing problem size due to scaling with $\mathcal{O}(N^2)$ data points [\[36\]](#).

In an effort to relieve computational overhead associated with GNO's, MGNO presented decomposing the physical domain into a hierarchy of coarsened graphs. The MGNOs resulted in a method which is guaranteed to perform at $\mathcal{O}(N)$ when used on regular grids, as it can leverage fast multipole method theory [\[24\]](#) to systematically construct the hierarchy. However, when on unstructured domains, the hierarchy is determined by a generalized random graph approach, making use of a radius-based coarsening. In this construction of MGNO, appropriate coarsening hyperparameters of radius r_ℓ and number of points N_ℓ are left to be determined and manually prescribed by the user. Additionally, the linear complexity derivation of MGNO implicitly requires a uniform point density as $r_\ell \sim N_\ell^{-1/d}$ and

$\mathcal{O}(2^{-\ell}J)$ [31]. When operating on non-uniform unstructured domains, such as in CFD simulations with highly refined boundary layers or far field coarsening, uniform point density is lost and the $\mathcal{O}(N)$ guarantee no longer holds. The authors of [31] themselves suggest combining the generalized random graph with active learning to adaptively choose regions of finer discretization, yet offer no mechanism, theoretical framework, or implementation concept for how this could be feasibly achieved. To the best of the present authors' knowledge, this suggestion remains an unexplored open problem.

In a parallel vein, the spectral neural operator methods of wavelet and Fourier neural operators, WNO and FNO, present the very useful concept of learning an operator in a transformed function space, bringing compactness and expressivity to these models. While having $\mathcal{O}(N)$ [56] and $\mathcal{O}(N \log N)$ [35] costs respectively, WNO and FNO methods are also limited to regular grids in their practical implementations. WNO implementations assume that data is regularly gridded, and the wavelet decompositions require that grid sizes are dimensioned by factors of 2. FNO's practical implementation, in order to make use of fast Fourier transforms, requires that the grid be uniformly discretized, and the domain itself must be periodic.

Interpolation onto a uniform structured domain has been suggested as a work around to use these neural operators for unstructured discretizations [31], however this is not without its own issues. In order to capture the most densely discretized regions of an unstructured mesh, the entire regularly gridded interpolation domain must match that of the smallest unstructured discretization. This leads to unnecessarily large data sets compared to the original unstructured layouts.

Alternatively, you can interpolate onto a regular grid sized to on the average unstructured discretization size, however this entirely removes any mesh-embedded physical properties resolved in the flow such as boundary layers, shocks, expansions, or other refinement regions. Additionally, this interpolation is required for every single sample in your training and testing data sets. It is easy to see that computational cost quickly escalates when using

external measures to adapt these neural operators to unstructured domains.

Most recently, and also parameterizing in transformed coefficient space, Multiwavelet-based Multigrid Neural Operator (M2NO) [34] attempts to relax the uniform grid constraint of spectral methods by replacing Fourier or Wavelet bases with a multigrid structure built on predefined multiwavelet restriction and prolongation operators. The authors further demonstrate that this multigrid in transform-space approach can also serve as an efficient preconditioner for classical iterative solvers. However, M2NO authors explicitly note that the method is restricted to structured grids, and that its accuracy degrades on highly non-uniform or adaptive meshes [34]. They thus identify the future work direction of coupling graph-based kernels with geometric-multigrid as an alternative to remedy this restriction, corroborating the current gap this paper addresses.

The practical implementations required to achieve discretization invariance of these approaches, including recent multigrid-transform hybrids, introduce constraints that inherently specialize each method to a particular class of problem geometry and discretization. We therefore propose that a compact and expressive neural operator architecture, specifically designed for unstructured domain discretizations, can help fill a crucial gap for neural operators, lending them to being applied to a broader spectrum of engineering problems of interest.

The following work develops a computationally feasible mesh-informed multilevel neural operator (MNO) specifically developed for unstructured domain discretizations. The MNO is designed with the future goal of serving as a timestep initial solution generator within an iterative PDE solver, reducing iterations and accelerating solver convergence as has demonstrated in [66, 32, 54, 49]. As it is envisioned to operate on a per-discretization basis for a given problem, the MNO does not need to be discretization-invariant, and it is intentionally designed for a complementary regime to existing discretization-invariant neural operator approaches, while taking inspiration from these methods.

The MNO trades generality across discretizations for practical feasibility on the com-

plex unstructured geometries that arise in real engineering problems. Rather than viewing a problem's domain discretization as an obstacle to be circumvented, the MNO purposely leverages it as a core capability, using the discretization to better capture multiscale phenomena on complex engineering geometries. The MNO draws inspiration from neural operator concepts presented in previous methods, specifically the dual branch approach and principle of learning weights and biases in a transformed multiscale function space, as is conceptually done in WNO and FNO. We replace typical Fourier or wavelet decompositions with algebraic multigrid multilevel decomposition (A-MGARD), as presented in chapter 2. A-MGARD relieves the restriction of requiring a normally gridded domain, opening the door to a greater set of problems of interest and real world engineering applications that current approaches are not designed to address.

4.2 Development of Multilevel Neural Operator

The typical goal of a neural operator network is to learn a mapping between 2 infinite dimensional spaces from a collection of observed input-output pairs of data. Given a domain $\Omega \in \mathbb{R}^d$, define the separable Banach spaces $\mathcal{A} := \mathcal{C}(\Omega; \mathbb{R}^{n_a})$ and $\mathcal{U} := \mathcal{C}(\Omega; \mathbb{R}^{n_u})$ with functions taking value in $\mathbb{R}^{n_a}$ and $\mathbb{R}^{n_u}$. These Banach spaces form normed vector spaces $(\mathcal{A}, \|\cdot\|_{\mathcal{A}})$ and $(\mathcal{U}, \|\cdot\|_{\mathcal{U}})$. Thus, the neural operator aims to learn the nonlinear operator $\mathcal{D} : \mathcal{A} \mapsto \mathcal{U}$ where the spaces' input and output functions are denoted by $\mathbf{a} : \Omega \mapsto \mathbf{a}(x \in \Omega) \in \mathbb{R}^{n_a}$ and $\mathbf{u} : \Omega \mapsto \mathbf{u}(x \in \Omega) \in \mathbb{R}^{n_u}$, respectively. Assuming N data pairs $\{(a_j, u_j)\}_{j=1}^N$ are available, and that $u_j = \mathcal{D}(a_j)$, operator $\mathcal{D}$ can be approximated by neural networks:

$$\mathcal{D} : \mathcal{A} \times \theta_{\mathcal{NN}} \mapsto \mathcal{U}; \theta_{\mathcal{NN}} = \{\mathcal{W}_{\mathcal{NN}}, b_{\mathcal{NN}}\} \quad (4.1)$$

where $\mathcal{M}_{\mathcal{NN}}$ and $b_{\mathcal{NN}}$ are the weights and biases of the discrete parameterization of the neural network, $\theta_{\mathcal{NN}}$. While $\mathbf{a}$ and $\mathbf{u}$ are continuous, practical implementation requires to operate on n_{Ω} discrete points $\{x_i\}_{i=1}^{n_{\Omega}}$ of the domain Ω . Therefore, it is assumed that data $\{a_j \in \mathbb{R}^{n_{\Omega} \times n_a}, u_j \in \mathbb{R}^{n_{\Omega} \times n_u}\}_{j=1}^N$ is available.

Note that $\mathbf{u}$ now represents the output functions to remain consistent with neural operator nomenclature.

4.2.1 MNO Architecture description

To determine the nonlinear operator $\mathcal{D}$, the proposed multilevel neural operator will follow a similar network architecture as both WNO and FNO: Lifting inputs to a higher channel space representation with operator $P_{\mathcal{N}}$, passing those augmented inputs through multiple layers of a dual-branch non-linear operator $G_{\mathcal{N}}$, followed by projecting the resulting non-linear layer coefficients to the target output space with operator $Q_{\mathcal{N}}$. The neural network determines the non linear operator $\mathcal{D}$ by finding the network parameters associated to $P_{\mathcal{N}}$, G , and $Q_{\mathcal{N}}$ which minimize loss function $\mathcal{L} = \mathcal{U} \times \mathcal{U}$ as follows:

$$\theta_{\mathcal{N}} \in \arg \min_{\theta \in \theta_{\mathcal{N}}} \mathcal{L}(\mathcal{D}(a), \mathcal{D}(a, \theta)) \quad (4.2)$$

The layout of this MNO architecture can be seen in fig. 4.1, and each component will now be described in further detail.

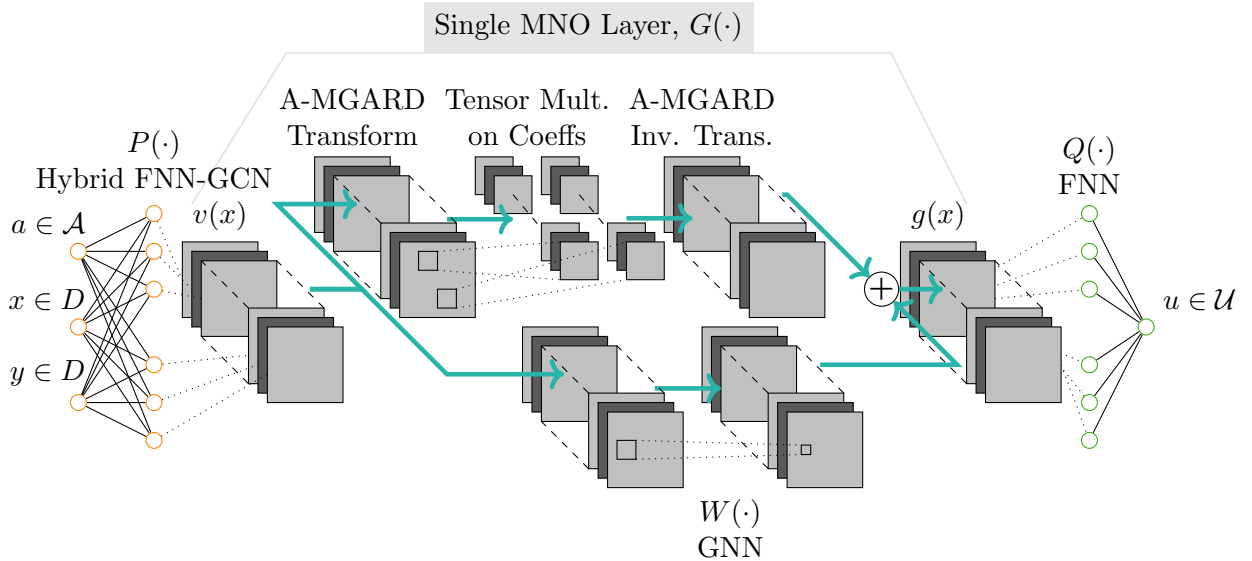


Figure 4.1: Visual example of MNO Architecture for a single MNO layer. Practical implementations have > 1 MNO layers.

4.2.1.1 Lifting Layer

Inputs $a(x, t) \in \mathcal{A}$ are lifted to a higher dimension channel space by means of an operator $P_{\mathcal{N}}(a) : \mathbb{R}^{n_a} \mapsto \mathbb{R}^{n_v}$, such that $v(x) = P_{\mathcal{N}}(a(x))$, $v(x) \in \mathbb{R}^{n_v}$. $P_{\mathcal{N}}$ is usually parameterized by a shallow fully connected neural network (FNN) for WNO and FNO methods. However, due to the assumed unstructured setting of the proposed method, an alternate approach is taken by utilizing a hybridized graph convolution network (GCN) in place of an FNN as a means to encode mesh information into the lifted inputs. This substitution will be covered in further details in section [4.2.4](#)

4.2.1.2 Non Linear Update Layers

Next, the lifted inputs $v(x)$ are passed into an update operator $G : \mathbb{R}^{n_v} \mapsto \mathbb{R}^{n_v}$ such that $v_{j+1}(x) = G(v_j(x))$, $\forall j = 1, \dots, l$, where l are the amount of step-wise updates, referred to as layers, required to achieve satisfactory convergence of the model. As with typical neural operators, update operator G is split into an upper and lower neural network computation branch.

Upper Branch The upper branch is a kernel integral style operator branch which performs decomposition on $v(x)$ to obtain coefficients in algebraic multilevel space, upon which a kernel function is parameterized by a neural network. This parameterization in transformed space helps boost expressivity in the network; the transformed space enables superior multi-scale and localized feature detection, similar to FNO and WNO's approach. The transformed space in MNO is obtained from our wavelet-like transformation, A-MGARD [2](#), whose operator is denoted as $\mathcal{M}$. A-MGARD leverages aspects of the underlying PDE discretization to generate an algebraic multilevel decomposition, effectively capturing both global and localized, multiscale phenomena. With the multilevel decomposition operator, we define the kernel integral operator of the upper branch of the MNO architecture as:

$$(K(a; \phi)v_j)(x) = \mathcal{M}^{-1}(R_\phi \cdot \mathcal{M}(v_j))(x), \quad x \in D \quad (4.3)$$

Where R_ϕ is a tensor learned by the network which captures the effect of the integral kernel. The decomposition and recomposition operators of A-MGARD, $\mathcal{M}$ and $\mathcal{M}^{-1}$ are further defined in section [4.2.2](#)

Lower Branch The lower branch of update operator $G(\cdot)$ is linear transformation operator, typically performing a linear convolution on $v(x)$ utilizing a convolution neural network (CNN). The proposed MNO architecture substitutes CNN in the lower branch for a geometric neural network (GNN), to be further detailed in section [4.2.4](#). The strength of this architecture lies in the dual branched approach. The upper integral kernel branch specializes in global features extraction through the multilevel decomposition, and the lower branch further enhances the modeling of localized features.

Complete update layer Upon completion of each respective branch, their outputs are summed together and passed to a non-linear activation function, $g(x)$. Thus, the updates from layer operator $v_{j+1} = G(v_j)$ takes the typical neural operator structure for approximation of nonlinear operators:

$$v_{j+1}(x) := g((K(a; \phi)v_j)(x) + Wv_j(x)), \quad x \in D, j \in [1, l] \quad (4.4)$$

where $K : \mathcal{A} \times \theta_K \rightarrow \mathcal{L}(\mathcal{U}, \mathcal{U})$ is the kernel integral operator of the decomposition branch, $W : \mathbb{R}^{n_v} \rightarrow \mathbb{R}^{n_v}$ is the linear transformation branch, and $g(\cdot) : \mathbb{R}^{n_v} \rightarrow \mathbb{R}^{n_v}$ is the non-linear activation function which operates component wise.

4.2.1.3 Transform to output space

Upon completion of the l layers of G , outputs $v_l(x)$ are passed to a local linear transformation operator, $Q_{\mathcal{N}}$, to determine the final desired output in solution space $u(x) = Q_{\mathcal{N}}(v_l(x))$.

4.2.2 Defining decomposition transform $\mathcal{M}$ with A-MGARD

The proposed MNO method utilizes an alternate decomposition space in the integral kernel branch to alleviate domain discretization restrictions. Instead of using a wavelet or Fourier decomposition, the method aims to utilize an algebraic based multilevel decomposition, A-MGARD presented in [2] which transforms input functions onto hierarchical orthogonal algebraic multilevel spaces. With A-MGARD, functions from the spatial domain are projected onto progressively coarsened basis function spaces. These function spaces are adaptively learned from the given domain discretization and associated topology of a given problem.

Similar to wavelet decomposition, this enables a multi-resolution representation of a neural network parameter space through the decomposition of physical domain data onto the projected spaces. Most notably, A-MGARD decomposition is not restricted to cartesian, structured, or periodic domains. Alleviating these classical restrictions opens the door to develop neural operators for problems requiring non-regular domain discretizations, as is often the case with modern day engineering problems of interest.

Recalling eq. (4.5), we modify its variables to have a form compatible with the neural operator nomenclature. Thus, a given function, f , is written as sum of it's decomposition coefficients, $c(x_i)$ by:

$$f = \sum_{\ell=0}^L \sum_{i \in \mathcal{N}_\ell^*} c(x_i) (I - Q_{\ell-1}) \psi_i^\ell \quad (4.5)$$

Recall from [2] that the operation of $(I - Q_{\ell-1}) \psi_i^\ell$ forms ‘out-of-level’ orthogonal basis functions, denoted $\equiv \tilde{\psi}_i^\ell$. These basis functions of level ℓ are orthogonal to all other basis functions which are not on that same level:

$$\int_{\Omega} \tilde{\psi}_i^\ell \tilde{\psi}_j^g = 0 \quad \forall i \in \mathcal{N}_\ell^*, j \in \mathcal{N}_g^*, \text{ and } \ell \neq g$$

‘Out-of-level’ basis functions $\tilde{\psi}_i^\ell$ are in essence pseudo wavelet bases created in an ad hoc manner at each level of the decomposition.

The algorithmic process of the multilevel decomposition results in the following definition for a given level of the multilevel decomposition operator:

$$(\mathcal{M}f)_\ell(i) = \langle f, \tilde{\psi}_i^\ell \rangle = \int_{\Omega} f(x) \cdot \tilde{\psi}_i^\ell(x) dx \quad (4.6)$$

where $\tilde{\psi}_i^\ell(x)$ are the orthogonal basis functions which replace the Wavelet and Fourier basis from WNO and FNO respectively. Naturally, it follows that the full multilevel decomposition operator is defined as:

$$\mathcal{M} : f \mapsto \{c_\ell(i)\}_{\ell=0, \dots, L}^{i \in \mathcal{N}_\ell^*} \quad (4.7)$$

where $c_\ell(i) = \langle f, \tilde{\psi}_i^\ell \rangle$ are the coefficients obtained when projecting f into multilevel space.

The inverse operator $\mathcal{M}^{-1}$ is then defined as

$$\mathcal{M}^{-1} : \{c_\ell(i)\} \mapsto f(x) = \sum_{\ell=0}^L \sum_{i \in \mathcal{N}_\ell^*} c_\ell(i) (I - Q_{\ell-1}) \psi_i^\ell \quad (4.8)$$

providing completeness of $f = \mathcal{M}^{-1}(\mathcal{M}(f))$.

Noting that the discretized finest level projection $Q_L \mathbf{u}(x) = \mathbf{u}(x)$, the entire hierarchy of projections and resulting multilevel coefficients $c_\ell(i)$ can be determined from original data and an appropriate hierarchy of function spaces and interpolation operators from AMG as presented in chapter [2](#)

4.2.3 Parameterization in Multilevel Space

A-MGARD's multilevel decomposition creates $L + 1$ sets of multilevel coefficients c_ℓ , providing a hierarchical multi-resolution representation of a given function $\mathbf{u}$. In an effort to remain as computationally efficient as possible, MNO follows FNO and WNO in truncating coefficients associated with 'finer' (analogous to higher frequency modes in FNO) projections of $\mathbf{u}$, as the coarsest levels of decomposition contain the most important features of the function. Similar to wavelet spaces, the hierarchical nature of the multilevel decomposition spaces ensures that the learned parameters at coarse levels are automatically propagated to the finer levels through the inverse multilevel transform. This permits MNO to also learn on

a reduced subset of dominant coefficients, notably those associated with the coarser levels of the decomposition, without loss of information

As the multilevel coefficients are only non-zeros for nodes associated with $\mathcal{N}_\ell^*$, the size of the neural network parameter space required is equivalent to n_{r-1} for each channel, where r are the number of retained levels of coefficients. Therefore, $\mathcal{M}(v_j, r) \in \mathbb{R}^{n_{r-1} \times n_v}$ when retaining coefficients c_ℓ for the r coarsest levels. Similar to WNO, the parameterization of R_ϕ in multilevel space is accomplished by integrating out ϕ as a product of learned neural network tensor $R \in \mathbb{R}^{n_{r-1} \times n_v \times n_v}$ with $\mathcal{M}(v_j, r)$. The action of the integral kernel operator parameterization in transform space, $R \cdot \mathcal{M}(v_j)$, from eq. (4.3) can then be expressed as:

$$(R \cdot \mathcal{M}(v_j, r))_{i_1, i_2}(x) = \sum_{i_3=1}^{n_v} R_{i_1, i_2, i_3} \mathcal{M}(v_j, r)_{i_1, i_3}, \quad i_1 \in [1, n_{r-1}], \quad i_2, i_3 \in \{1, \dots, n_v\} \quad (4.9)$$

where r is a specifier for each retained level.

4.2.3.1 FNO and WNO influence on design of the MNO architecture.

MNO takes influence and guidance in its architecture design from FNO and WNO, specifically regarding parameterizing the kernel integral operator, $K(a; \phi)$, in transform space. The formulation in both FNO and WNO is conceptually analogous to a data-driven approximation of Green's function

$$u(x) = \int_{\Omega} H(x, y) f(y) dy \quad (4.10)$$

with the following neural network approximation:

$$(K(a; \phi) v_j)(x) := \int_D k(x, y, a(x), a(y); \phi) v_j(y) dy \quad (4.11)$$

where $k_\phi : \mathbb{R}^{2(d+d_a)} \mapsto \mathbb{R}^{n_v \times n_v}$ is a neural network parameterized by $\phi \in \theta_K$. To remain discretization invariant, they proceed by removing solution dependence on a terms from eq. (4.11), and define the shift-invariant neural network kernel function $k_\phi = k(x - y; \phi)$.

From this assumption, the kernel integral operator becomes a convolution operator:

$$(K(\phi) v_j)(x) := \int_D k(x - y; \phi) v_j(y) dy \quad (4.12)$$

Algorithm 4 MNO Algorithm Example - 1D Problem

Require: N -samples of the pair $\{a(x) \in \mathbb{R}^{n_D \times n_a}, u(x) \in \mathbb{R}^{n_D \times n_u}\}$, coordinates $x \in \mathcal{D}$, discretized domain mass matrix $\mathbf{M}_L$, and network hyperparameters.

- 1: Stack the inputs: $\{a(x), x\} \in \mathbb{R}^{n_D \times 2n_a}$.
- 2: Determine and store function space hierarchy $\{V_\ell\}_{\ell=0}^L$ and operators $\mathbf{M}_\ell, \mathbf{P}_\ell$
- 3: **for** epoch = 1, ..., epochs **do**
- 4: Uplift the input using transformation $P_{\mathbf{NN}}(\cdot)$: $v_0(x) \in \mathbb{R}^{n_D \times n_v} = P(\{a(x), x\} \in \mathbb{R}^{n_D \times 2n_a})$.
- 5: **for** $j = 1, \dots, l$ **perform the iterations:** $v_{j+1} = G(v_j)$ **do**
- 6: Decompose the input using A-MGARD decomposition: $\mathcal{M}(v_j(x)) \in \mathbb{R}^{n_m \times n_v}$.
- 7: Parameterize the NN kernel k_ϕ in the multilevel space: $R_\phi * \mathcal{W}(v_j(x))$. $\triangleright$ eq. (4.9)
- 8: Reconstruct the convoluted input: $v_{j+1}^K(x) = \mathcal{M}^{-1}(R_\phi * \mathcal{M}(v_j(x)))$. $\triangleright$ eq. (4.3)
- 9: Perform the linear transform: $v_{j+1}^W(x) = W * v_j(x)$ using GNN.
- 10: Normalize and add the outputs of step 8 and 9: $\tilde{v}_{j+1}(x) \in \mathbb{R}^{n_D \times d_v} = \left(\frac{v_{j+1}^K}{\|v_{j+1}^K\|} + \frac{v_{j+1}^W}{\|v_{j+1}^W\|} \right) (x)$.
- 11: **if** $j \neq l$ **then**
- 12: Apply the activation to complete the layer update: $v_{j+1} \in \mathbb{R}^{n_D \times n_v} = g(\tilde{v}_{j+1}(x))$. $\triangleright$ eq. (4.4)
- 13: **end if**
- 14: **end for**
- 15: Apply an activated linear transformation: $r(x) \in \mathbb{R}^{n_D \times n_v/2} = g(Q_1(v_l(x)))$ ($Q_1(\cdot) : \mathbb{R}^{n_v} \mapsto \mathbb{R}^{n_v/2}$ is a FNN).
- 16: Compute the final output: $\hat{u}(x) \in \mathbb{R}^{n_D \times n_u} = Q_2(r(x))$ ($Q_2(\cdot) : \mathbb{R}^{d_r} \mapsto \mathbb{R}^{n_u}$ is a FNN).
- 17: Compute the loss: $\mathcal{L}(u, \hat{u})$.
- 18: Compute the gradient of the loss: $\frac{\partial \mathcal{L}(u, \hat{u})}{\partial \theta_{\mathbf{NN}}}$.
- 19: Update the parameters of the network using the gradient.
- 20: **end for**

Ensure: Solution space $u \in \mathcal{U}$, parameters of NN $\theta_{\mathbf{NN}}$.

Applying the convolution theorem, the convolution in physical space becomes a pointwise multiplication in transform space, $\mathcal{T}(k * v_j) = \mathcal{T}(k) \cdot \mathcal{T}(v_j)$. Rather than computing $\mathcal{T}(k)$ analytically, FNO and WNO directly parameterize this quantity as a learned tensor R_ϕ yielding:

$$(K(\phi)v_j)(x) = \mathcal{T}^{-1}(R_\phi \cdot \mathcal{T}(v_j))(x), \quad x \in D \quad (4.13)$$

where $\mathcal{T}$ and $\mathcal{T}^{-\infty}$ are their respective forward and inverse transforms.

We highlight that in practical implementations, the action of the kernel integral operator becomes an approximation rather than an exact operation. For example, FNO makes the further assumption that the kernel function k_ϕ is periodic, enabling it to use a discrete Fourier series expansion which diagonalizes the transformation and turns the convolution into a point-wise multiplication. This periodicity assumption restricts FNO's applicability to problems where periodic boundary conditions are appropriate or can be reasonably approximated. FNO then truncates this expansion to a maximum number of Fourier modes, which further approximates the action of the kernel integral on transformed coefficients.

In a similar vein, WNO, which was inspired by FNO, makes use of a truncated set of discrete wavelet transformation, which permitted not only a hierarchical decomposition in frequency domain but also spatial domain. The discrete wavelet transform in a generalized sense is a projection of a function Γ onto a discretized basis, $\psi_{\tau,m}$:

$$\mathcal{W}(\Gamma) = \int_D \Gamma(x) \psi_{\tau,m}(x) dx \quad (4.14)$$

where $\tau \in \mathbb{Z}^+$ and $m \in \mathbb{Z}^{+*}$. Since the use of truncated discretized wavelets turns the action of $K(a; \phi)$ onto v_j into an approximation, and since $K(a; \phi)$ is a bounded linear operator for fixed a and ϕ , we hypothesized that any complete basis satisfying $u = \mathcal{M}^{-1}(\mathcal{M}(u))$ can similarly approximate the action of the integral kernel.

This motivates the replacement of the wavelet basis with the problem adapted AMGARD basis. Recognizing that R_ϕ in eq. (4.13) is a learned neural network tensor capturing the action of the integral kernel in transform space, and not the transform of the kernel k_ϕ ,

the definition of eq. (4.13) is independent of the specific transform used. Replacing $\mathcal{T}$ with the problem adapted multilevel basis $\mathcal{M}$ to obtain eq. (4.3) naturally followed.

4.2.3.2 Classification of the MNO within the Neural Operator Framework

While the Fourier and wavelet bases enable FNO and WNO to satisfy discretization invariance in the sense that learned parameters are defined independently of any particular discretization, both architectures operate on a fixed, finite truncation of their respective bases. FNO operates on k_{max} modes, and WNO operates on a fixed number of wavelet decomposition levels. As briefly discussed in section 4.1, universal approximation in the neural operator sense requires the capacity to approximate any continuous operator, which in the basis-coefficient setting is achieved asymptotically through basis refinement. However, at any fixed truncation, FNO and WNO cannot represent operators whose action depends on content beyond their retained basis coefficients, and therefore do not strictly satisfy universal approximation by their own construction. Universal approximation is recovered in the limit through basis refinement, and it is under this argument that FNO and WNO retain their classification as neural operators.

Crucially however, this basis refinement argument is only practically realizable for FNO and WNO on the uniform structured grids their implementation requires. FNO requires a uniform discretization in order to utilize the Fast Fourier Transform capability [31, 35], and WNO requires that the discretization be uniform discretizations of $n = (2^a)$ points to perform recursive decomposition using discrete wavelet transforms [56]. MNO replaces the Fourier and wavelet bases with a problem adapted A-MGARD multilevel basis, for which the analogous basis refinement, adding finer levels of multilevel decomposition, is realizable directly on unstructured discretizations.

Discretization invariance in the MNO setting follows from Kovachki et al.’s own acknowledgment that any finite-dimensional neural network combined with interpolation is resolution invariant [31]. We therefore argue that any neural network method whose basis

can be sufficiently refined to achieve universal approximation in the limit, and which achieves discretization invariance through interpolation, satisfies the conditions for classification as a neural operator. The A-MGARD basis used in MNO is thus one instantiation of this general principle.

Ultimately, the neural network approximating the action of the integral kernel in these neural operators is agnostic to the specific decomposition transformation method chosen. For FNO, WNO, and MNO, the network is learning the weights and biases for the action of the integral kernel in a function space differing from the physical domain space, in order to have an enriched representation of the original function. The true purpose in choosing a specific transformation function space for a neural operator is that of an inductive bias for the problem the neural operator is attempting to approximate. Periodic and smooth problems benefit from globally structured function spaces as in FNO, and multiscale problems like Navier-Stokes benefit from wavelets’ ability to represent local features of varying scale. MNO is thus a neural operator tailored to unstructured domain discretizations, one that purposefully trades generality across discretizations for practical feasibility on the complex geometries that existing approaches are not designed to address, as will be demonstrated in sections [4.3](#) and [4.4](#)

4.2.4 Unstructured Domain Considerations for $P(\cdot)$ Lifting

Due to the irregularity of unstructured domains, special considerations must be taken to ensure topological information can be embedded into the augmented functions $v_j(x)$. While nodal coordinates are provided as inputs to the network, the connectivity (neighbors) of those nodes cannot be inferred as it typically would on uniform or tensor product domains. Without embedding connectivity into the lifted inputs, the neural network would only have point-local information, and no true context on how each node is related to its neighbors. Embedding topological information also enables the network to more efficiently distinguish between boundary nodes and interior nodes, and can more readily capture gradient-driven

processes.

A natural way of embedding topological information is by leveraging a graph convolution layer within the lifting operator $P(\cdot)$. The lifting operator for the proposed architecture employs a hybrid shallow FNN-GNN structure where the inputs are processed along 2 paths and combined. The first path applies a non linear transformation (activated shallow FNN layer) to capture local features, and the second path performs neighborhood aggregation via a graph convolution network (GCN) layer to incorporate mesh connectivity. The GCN conveniently makes use of the weighted Laplacian matrices from the AMG pre-computation steps, providing the edge weights and connections required by the GCN. Both paths are then combined using a linear activation (shallow FNN) layer. Each component provides learnable parameters within the overall MNO network. The resulting lifted inputs provide rich connectivity aware inputs for the MNO layers.

4.2.5 Embedding periodicity in the MNO

Many problems of interest often have periodicity constraints placed on their boundaries. Should a given problem have periodic boundary conditions, it is hypothesized that embedding the periodicity of the boundary within the neural network can help improve learning and its resulting models. Take an arbitrary 1D, N node, periodic domain as shown in fig. [4.2](#)

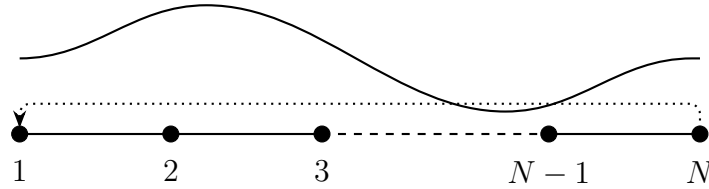


Figure 4.2: Arbitrary periodic 1D domain. Node N is connected back to node 1 to embed periodicity.

The end nodes of the domain, 1 and N , should ideally be able to share information within the network. However, without an explicit connection in the MNO setting, the periodicity of this domain must be inferred by the network. A simple way of embedding

pseudo periodicity is by modifying the mass matrix $\mathbf{M}_L$, and subsequently the Laplacian operator $\mathbf{A}_L$, through addition of mass entry corrections for pairs of nodes which share a periodicity. The mass matrix for this simple 1D problem has periodic correction terms, $\mathbf{c}$, added to the diagonal terms $(1, 1)$ and (N, N) , as well as the off-diagonal entries of $(1, N)$ and $(N, 1)$.

$$\begin{pmatrix} m_{11} & m_{12} & 0 & 0 & 0 \\ m_{21} & m_{22} & m_{23} & 0 & 0 \\ 0 & m_{32} & \cdots & \cdots & 0 \\ 0 & 0 & \cdots & \cdots & m_{nN} \\ 0 & 0 & 0 & m_{Nn} & m_{NN} \end{pmatrix} \xrightarrow{+\mathbf{c}} \begin{pmatrix} m_{11} + c_{11} & m_{12} & 0 & 0 & c_{1N} \\ m_{21} & m_{22} & m_{23} & 0 & 0 \\ 0 & m_{32} & \cdots & \cdots & 0 \\ 0 & 0 & \cdots & \cdots & m_{nN} \\ c_{N1} & 0 & 0 & m_{Nn} & m_{NN} + c_{NN} \end{pmatrix} \quad (4.15)$$

The inclusion of the periodic correction terms has 2 direct effects on the neural network. The first is that multilevel decomposition can now perform interpolations between nodes 1 and N, providing a better global sense of features at different scales. The second is that the Laplacian informs the graph based networks of the periodic message pathways, where improved local relativity can be obtained at boundaries. Effects of embedding periodicity are presented in section 4.3.

? neighborhood information or relationships or just improved locality?

4.2.6 Consideration for Convolution Branch, $W(\cdot)$

Further considerations should be taken in the MNO layers, specifically the linear convolution branch, $W(\cdot)$, which is classically performed with a CNN. In practical implementations, as in WNO, the CNN branch is simply performing channel mixing for each spatial coordinate of the domain. While simple, this technique is likely insufficient for unstructured meshes as physical location cannot implicitly be inferred from index position in the data arrays. Instead of utilizing CNN in the lower convolution branch, we propose to perform a ‘Graph Laplacian Convolution’ which we denote as GNN.

The GNN is essentially a graph augmented $k = 1$ CNN convolution. It first does a linear transform on $v(x)$ to perform channel mixing as is performed with $k = 1$ CNN,

followed by a spatial mixing of those outputs through a graph convolution using the physics informed weighted Laplacian. The operation of the Laplacian Convolution can be expressed as $\mathbf{A}_L \otimes (Wv(x))$, where W is the learnable weight matrix for the linear transform, and A_L is the mass Laplacian. The outer product with $\mathbf{A}_L$ further embeds connectivity and helps perform local feature smoothing through a graph diffusion. The effects of utilizing GNN in the lower branch are also presented in section [4.3](#).

4.3 Periodicity and Convolution Branch Study

In this section, we seek to confirm the hypothesis that embedding periodicity improves the approximation models obtained from an MNO network. We also seek to confirm that utilizing GNN instead of CNN for the convolution branch operator, $W(\cdot)$, is beneficial while operating with an MNO architecture. The study was performed on a 1D time independent advection wave case and a 2D time dependent Navier-Stokes case utilizing various combinations of periodicity embedded branches of the MNO network.

1D Advection Wave The 1D test was performed on Advection Wave data¹, the same used in [\[45, 56\]](#), to learn the best approximation model for the following problem:

$$\begin{aligned} \partial_t u(x, t) + \nu \partial_x u(x, t) &= 0, \\ x &\in (0, 1), \quad t \in (0, 1), \\ u(x - \pi, t) &= u(x + \pi, t), \\ x &\in (0, 1), \quad t \in (0, 1) \end{aligned} \tag{4.16}$$

where ν is the advection velocity. The initial condition was reported as

$$u(x, 0) = h \mathbf{1}_{(c-\omega/2, c+\omega/2)}(x) + \sqrt{\max\{h^2 - (a(x - c))^2, 0\}} \tag{4.17}$$

where $\mathbf{1}$ is an indicator function, and c , ω , and h are the center, width, and height of a square wave. The values of c , ω , and h reported for the creation of the data were

¹ 1D Advection wave data can be obtained from <https://github.com/lu-group/deeponet-fno/tree/main/data>

randomly chosen from $[0.3, 0.7] \times [0.3, 0.6] \times [1, 2]$, while $\nu = 1$. The waves were discretized on a uniform 40 point 1D domain and are advected with time step of 0.025, for a total of 40 time steps. The goal for this time independent model was to learn the operator which maps to 5 time-steps ahead, $\mathcal{D} : u_0(x) \mapsto u(x, t = 0.125)$. At this timestep, the initial wave have been advected such that a fraction of the leading parabolic wave has exited the boundary at $x = 1$ and re-entered the periodic domain at $x = 0$.

2D vorticity Navier-Stokes The 2D time dependent test was performed on vorticity data^[2], the same used in [56][35], to learn an approximation model for the 2D Navier-Stokes equations for vorticity:

$$\begin{aligned}\partial_t \omega(x, y, t) + u(x, y, t) \cdot \nabla \omega(x, y, t) &= \nu \Delta \omega(x, y, t) + f(x, y), \quad x, y \in (0, 1), \quad t \in (0, T], \\ \nabla \cdot u(x, y, t) &= 0, \quad x, y \in (0, 1), \quad t \in [0, T], \\ \omega(x, y, 0) &= \omega_0(x, y), \quad x, y \in (0, 1).\end{aligned}\tag{4.18}$$

where $\nu \in \mathbb{R}$ and $f(x, y)$ are the fluid viscosity and the source function. u and ω are the velocity and vorticity fields of the fluid. Viscosity was set as $\nu = 10^{-3}$, and initial vorticity is taken as $\omega(x, y, 0) = \omega_0(x, y); x, y \in (0, 1)$. The goal of this model is to determine the operator which maps vorticity fields at time steps $t \in [0, 10]$ to vorticity fields up to $t \in (10, 20]$. Therefor the learned operator is $\mathcal{D} : \omega_{(0,1)^2 \times [0,10]} \mapsto \omega_{(0,1)^2 \times (10,20]}$. The data used is from [cite source], where it was created with a Crank-Nicolson scheme with a time-step of 10^{-4} , a simulated initial vorticity from a Gaussian random field as $\omega_0(x, y) = \mathcal{N}(0, 7^{3/2}(-\Delta + 49I)^{-2.5})$, and fixed source function $f(x, y) = 0.1(\sin(2\pi(x+y)) + \cos(2\pi(x+y)))$. The data, $\omega_{1:20}(x, y)$, are sample stacks of 64×64 resolution 2D solutions for every $\Delta t = 1s$, which were flattened to 1D arrays of 4096 data points. Predictions are performed using a recurrent neural network (RNN) style learning architecture, where the first prediction is used as an input for the 2nd prediction, creating a ‘rolling window’ of predictions and inputs.

² 2D Navier-Stokes data can be obtained from <https://drive.google.com/drive/folders/1UnbQh2Wwc6knEHbLn-ZaXrKUZh7pjt->

4.3.1 Approximation model architecture set-ups

Approximation models for these problems were learned using multiple network architectures. Tables 4.1 and 4.2 summarize the model architectures for each problem. Model labels for the tests are structured as Upper^(*)-Lower^(*) branch types, where inclusion of a superscript * in a result label denotes the branch utilizes the non-periodic version of it's respective mass or Laplacian matrix input. For example, M*-GNN is the label for a non-periodicity multilevel decomposition upper branch with a periodicity embedded lower GNN branch.

All the MNO style architectures made use of the PyTorch AdamW optimizer [43], with cosine annealing learning rate scheduling, as early testing found this combination to obtain the best and most stable learning results among available options. The starting and ending learning rates, batch sizes, and optimizer weight decay used by the models in tables 4.1 and 4.2 are provided in table 4.3 for each respective problem.

Table 4.1: 1D Advection Wave problem - network architecture test configurations

Model	Number of Samples		Network Configuration					
	Training	Testing	Lifting	$P(\cdot)$ Channels	$K(\cdot)$ Layers & Type	$W(\cdot)$ Layers & Type	$Q(\cdot)$ Channels	$g(\cdot)$
M*-GNN(*)	900	100	FNN-GCN	2 → 128	4, Multilevel(*)	4, GNN(*)	128 → 64 → 1	GeLU
M*-CNN*	900	100	FNN-GCN	2 → 128	4, Multilevel(*)	4, CNN*, k=1	128 → 64 → 1	GeLU
GNN	900	100	FNN-GCN	2 → 128	N/A	4, GNN	128 → 64 → 1	GeLU
WNO	900	100	FNN	2 → 96	4, Wavelet - db6	4, CNN*, k=1	96 → 128 → 1	GeLU

Table 4.2: 2D time-dependent N-S problem - network architecture test configurations

Model	Number of Samples		Network Configuration					
	Training	Testing	Lifting	$P(\cdot)$ Channels	$K(\cdot)$ Layers & Type	$W(\cdot)$ Layers & Type	$Q(\cdot)$ Channels	$g(\cdot)$
M-GNN(*)	1000	100	FNN-GCN	12 → 64	4, Multilevel	4, GNN(*)	64 → 32 → 1	GeLU
M-CNN*	1000	100	FNN-GCN	12 → 64	4, Multilevel	4, CNN*, k=1	64 → 32 → 1	GeLU
GNN	1000	100	FNN-GCN	12 → 64	N/A	4, GNN	64 → 32 → 1	GeLU
WNO	1000	100	FNN	12 → 30	4, Wavelet - db6	4, CNN*, k=1	30 → 128 → 1	GeLU

The GNN labeled model serves as a comparison for neural network models currently applicable to unstructured domains with similar computational complexity as MNO architectures. The authors recognize that there exist superior, mesh invariant graph-based neural

Table 4.3: Problem specific optimizer settings used in combination with models from tables 4.1 and 4.2

Problem	Training Optimizer Settings					
	Epochs	Batch Size	LR Scheduler	Initial LR	LR min	Weight Decay
1D Wave Advection	500	20	Cosine Annealing	1e-3	1e-5	1e-4
2D Navier Stokes	500	20	Cosine Annealing	1e-2	1e-5	1e-3

network than the GNN model architecture presented here, such as graph kernel networks (GKN) [36]. However, GKNs are extremely cost prohibitive on larger problems, due to their computational complexity scaling as $\mathcal{O}(N^2)$, and rapidly become unfeasible in the MNO application context.

Note that the WNO model is purpose-built for structured, gridded data and serves here as a benchmark ceiling rather than a direct competitor. WNO is expected to outperform the MNO-based architectures (M-GNN, M-CNN, and GNN) on these structured test cases. The key result, as will be shown, is that certain MNO architectures approach WNO performance despite being designed for the more general setting of unstructured meshes.

Finally, all CNN convolution branches are non-periodic by construction, as the resulting output of CNN is inherently not dependent on $\mathbf{M}_L$ or $\mathbf{A}_L$. For labeling consistency, CNN branches will be labeled as CNN*.

As shown in tables 4.1 and 4.2, the lifting $P(\cdot)$ operator for all MNO architecture test cases was performed with a periodically embedded Laplacian matrix $\mathbf{A}_L$ in the GCN, while the MNO layers had combinations of periodic or non-periodic upper and lower branches.

4.3.2 Effects of embedding periodicity in $K(\cdot)$ and $W(\cdot)$

The first test conducted was to determine if, and how, embedding periodicity within the MNO layers improved model approximations. Figure 4.3a shows the results for the 6 possible combinations of periodicity and convolution branch architectures tested on the 1D Advection wave problem. Observing the results in fig. 4.3a, it is clear that entirely omitting periodicity, M*-GNN* and M*-CNN* cases, severely under-performs architectures where periodicity

was embedded in at least one branch. From cases with embedded periodicity, those where periodicity is only embedded in the $K(\cdot)$ integral branch, M-GNN* and M-CNN*, performed the best. Interestingly, embedding periodicity in both branches, the M-GNN case, performs worse than only embedding periodicity in the integral branch, M-GNN*. The additional inclusion of periodicity in the convolution branch is likely over constraining the network and duplicating periodicity information already learned from the $P(\cdot)$ lifting operator.

Shifting focus to now compare M-GNN* and M-CNN* to the GNN and WNO models, fig. 4.3b shows that M-GNN* and M-CNN* are nearing WNO performance, while GNN struggles to learn the desired mapping operator. Further investigating the model predictions, the Root Mean Square error of each model's position dependent prediction outputs was taken across all samples, such that a RMSE domain curve could be plotted. The position dependent standard deviation across all samples was also taken to highlight prediction variance across the samples for each model. The results from these test are plotted in fig. 4.4

A key observation from these results is that the square wave region is a pain-point for all models, as this region has the highest RMS error and Standard deviation across samples. Most notably, GNN's RMSE is approximately an order of magnitude larger than the others across the entire domain, with relatively immense standard deviations at the domain boundaries and square wave region. The MNO and WNO models follow similar error trends along the domain, all showing peak errors and largest standard deviations in the region of the leading edge of the square wave (wave is advected in $+x$ direction). As expected, the MNO style models only slightly under-perform the WNO model, showing performance parity to the current state-of-the-art for structured domains.

The main conclusion from this test is that including the periodicity in the multilevel integral branch, $K(\cdot)$, significantly improves performance when MNO is applied to a domain with known periodic boundaries.

parity might also imply time/cost

could soften
"may be!"
or hypothesis
+1
over-robust

correct but not = so $\neq$ parity

new

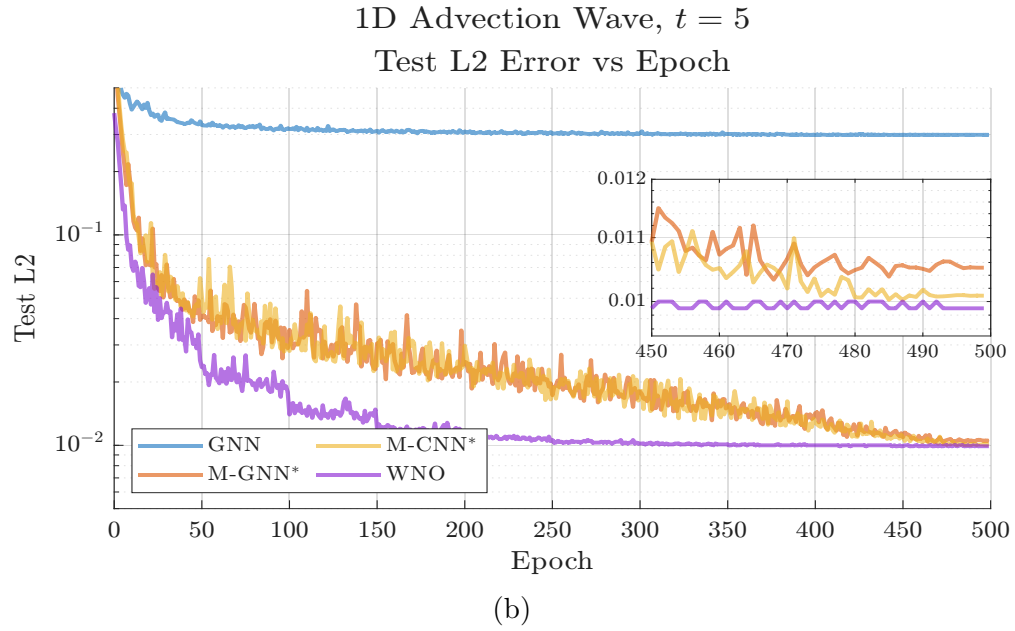
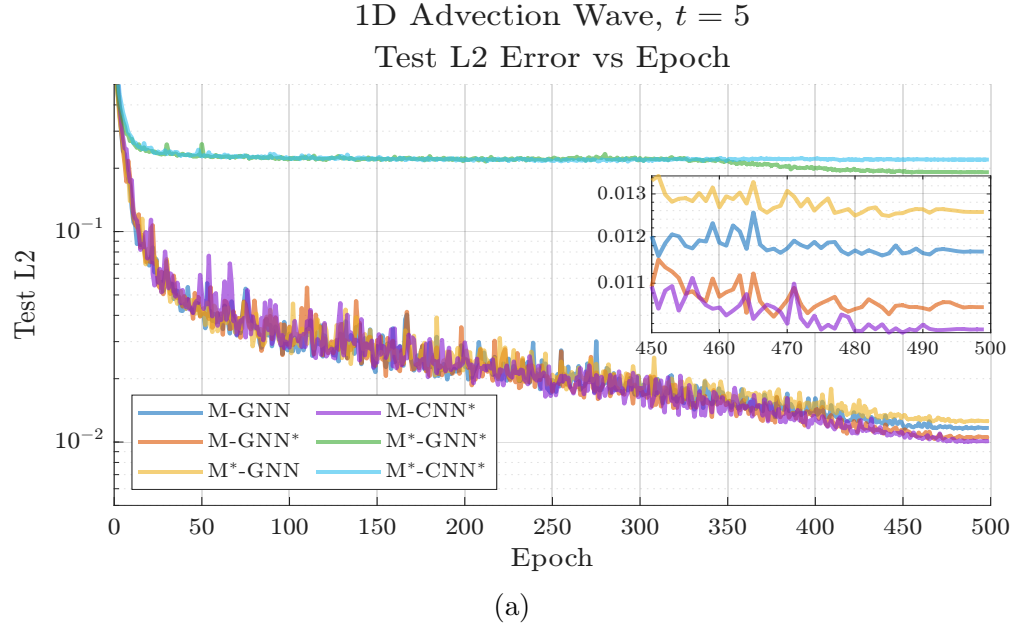


Figure 4.3: l2 testing errors vs epoch for 1D advection. a) Results for various combinations of embedding periodicity in $K(\cdot)$ and $W(\cdot)$ branches. b) Results for MNO style models compared to GNN and WNO. Superscript * denotes non-periodic for that branch. Inlays are close-ups of epochs 450 to 500

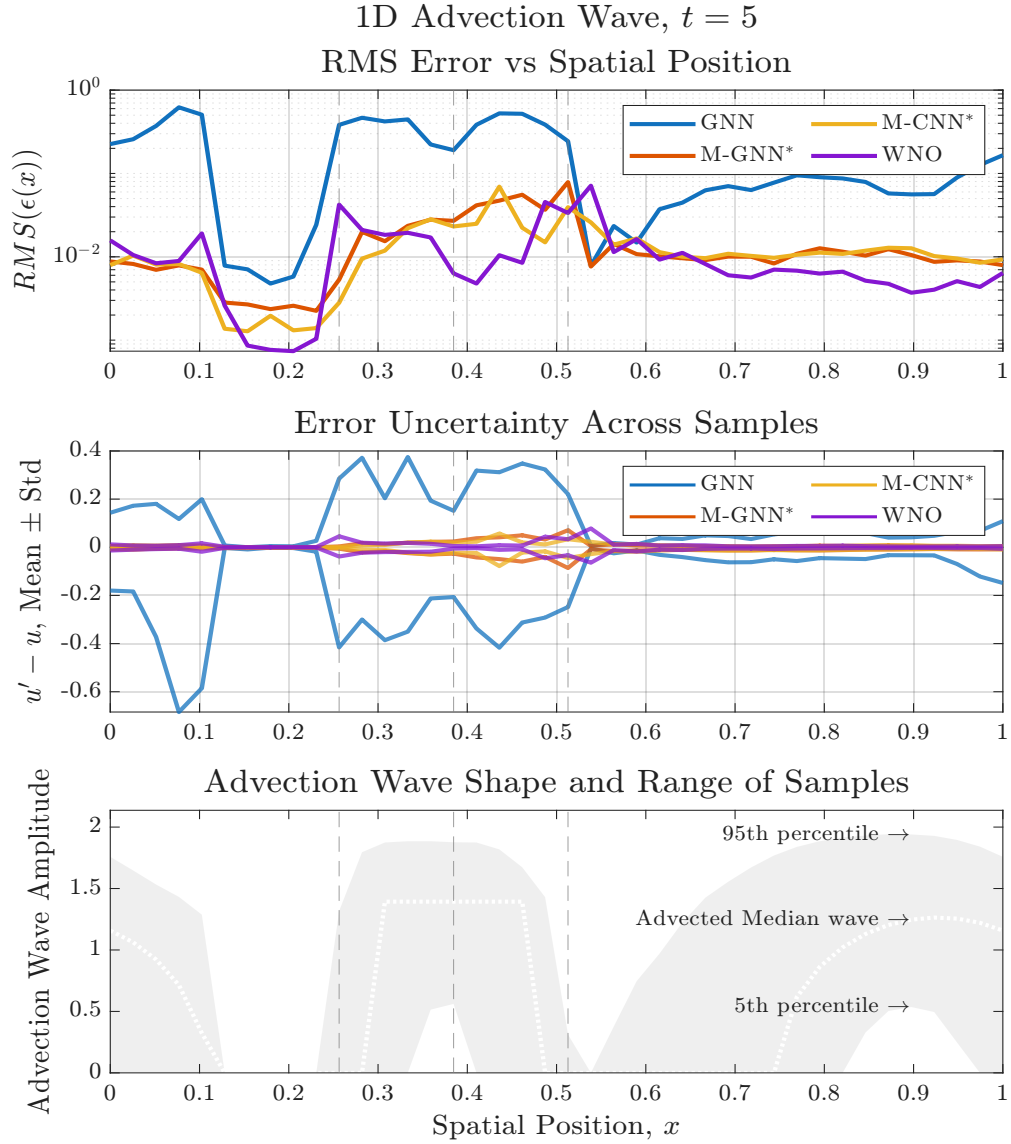


Figure 4.4: top: positional RMSE across samples. middle: positional mean error $\pm$ Std across samples. bottom: median wave shape and amplitude of the data sample set at $t=5$, as well as the 95th and 5th percentile bounds of the wave amplitudes (area under curve). Dashed vertical lines are to identify trouble regions pre, mid, and post square wave.

4.3.3 Convolution Branch, GNN vs CNN

Recalling the discussion in section 4.2.6 of utilizing GNN instead of CNN for the convolution branch, the l2 test error results in fig. 4.3a suggest that CNN might perform better. However, the test was performed on a structured 1D domain, where relative positioning could easily be inferred by the network. A more difficult domain, such as the one from the 2D Navier-Stokes problem previously described, is likely to benefit from including GNN in the convolution branch for MNO architecture. Based on the periodicity study results from section 4.3.2, the following 2D Navier-Stokes problem tests will only consider the periodically embedded multilevel branches, as detailed in table 4.2

Observing results in fig. 4.5a, GNN once again performs poorly in comparison to the MNO style architectures. GNN achieved a final l2 testing error of $\sim 12.5\%$, while M-GNN* and M-CNN* have final errors of $\sim 2.5\%$ and $\sim 3.1\%$, respectively. As expected, these results under perform WNO, which is reported in [56] to have a final l2 test error of $\sim 0.31\%$. Observing the per timestep RMSE across samples and spatial locations in fig. 4.5b, GNN error begins larger and grows at a much greater rate than for M-GNN* and M-CNN*. In this more difficult problem, the GNN convolution branch performs better than the CNN version. This can be attributed to beneficial neighbor connection information provided by the GNN, whereas the same information is now more difficult for the network to infer when operating with the CNN branch. As such, we propose that the MNO architecture should employ a GNN convolution for it's lower branch over the simple channel mixing $k = 1$ CNN which is typically used in other neural operators.

Recalling that time steps [1...10] were used to determine time steps [11...20], fig. 4.6 presents the truth, prediction, and absolute error across 4 time steps for a randomly selected sample of the M-GNN* approximation model of the 2D Navier-Stokes problem. As can be observed in fig. 4.6, the predicted solutions have good agreement with the truth solutions across the time steps. Large scale features are captured, with errors originating from finer

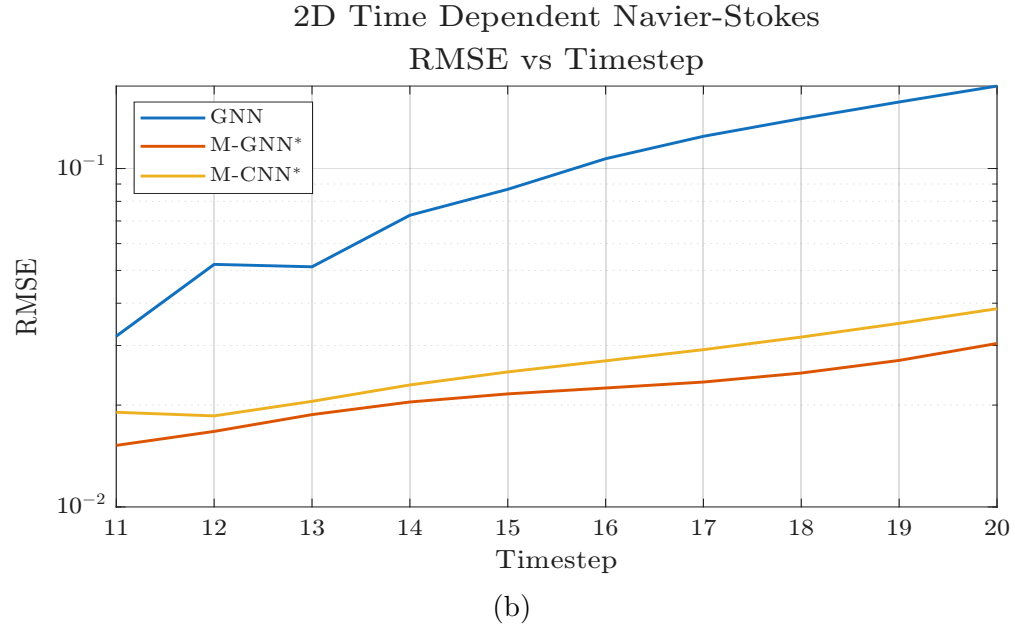
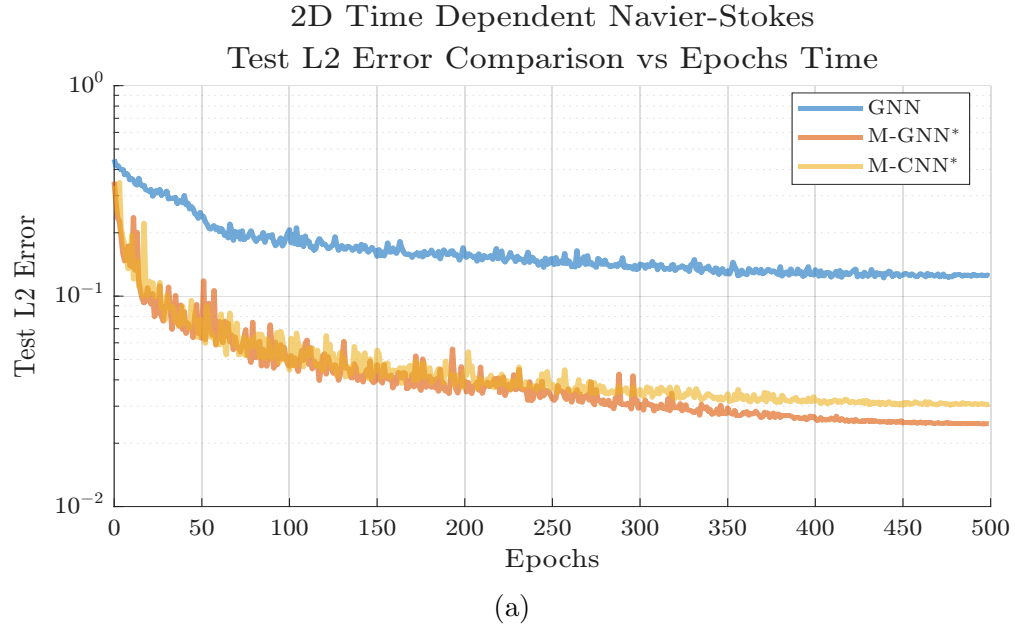


Figure 4.5: Approximation model errors for 2D Navier-Stokes problem. a) l2 test error and RMSE per predicted timestep. b) RMSE across samples and position vs predicted timestep. Superscript * denotes non-periodic for that branch.

scale features. A slight degradation in model accuracy occurs as time-steps get further away from the training data set, consistent with the trend in fig. 4.5b. The first predicted time step has an RMSE of $\sim 1.5\%$ and the 10th and final has an RMSE of $\sim 3.0\%$. This is attributed to the RNN style network used for this problem; predicted time steps are used as truth inputs in a rolling window of training time steps, therefore errors from early predictions propagate into the further time steps.

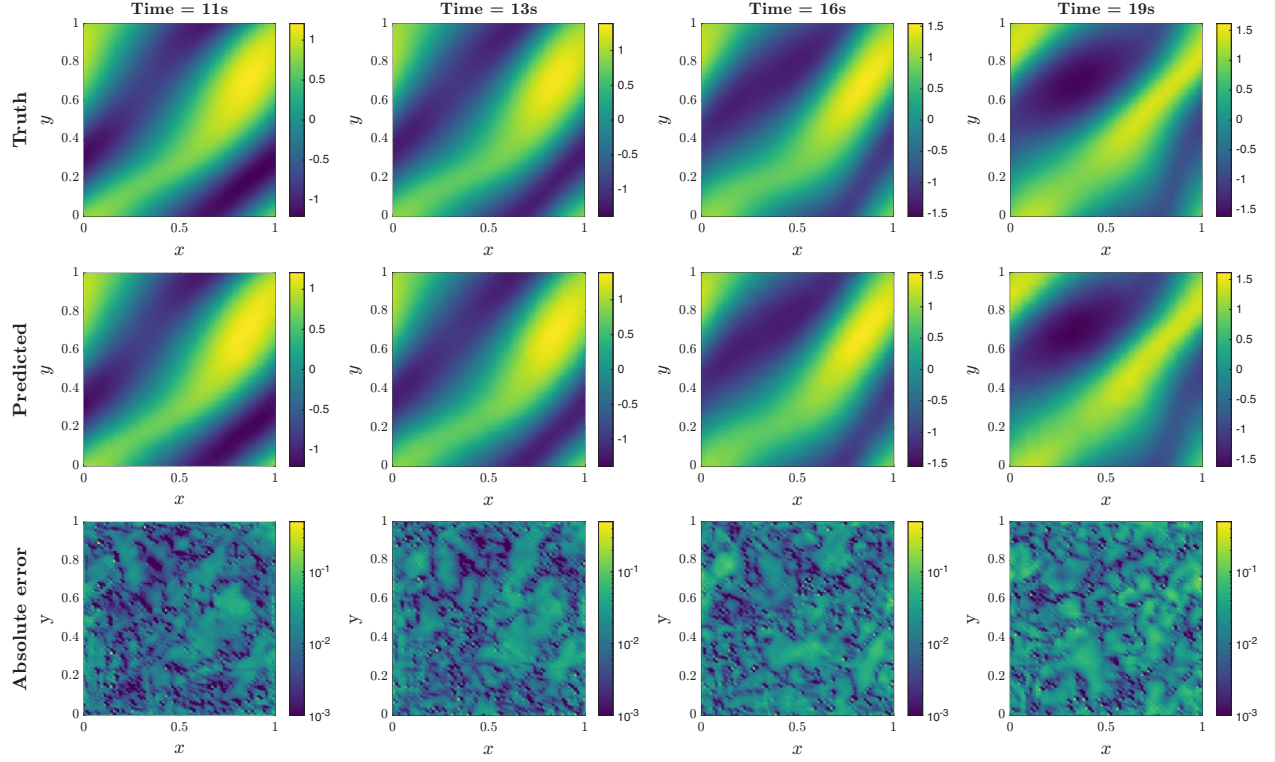


Figure 4.6: Evolution of model predictions over time for the 2D Navier-Stokes problem. Results are presented in a grid, each column shows results for a given time step of the model. Each row, from top to bottom, represents the true solution, predicted solution from the trained MNO model, and absolute error over the time steps. Note that color scales vary by time, yet are consistent between truth and prediction for a given time

4.4 MNO for approximation models on unstructured domains

With the MNO architecture proving to be a comparable option to current state-of-the-art neural operators in structured settings as shown in section 4.3, we now demonstrate that it excels in the unstructured setting. The unstructured problem of interest is cylinder in cross flow, where an unstructured hexagonal body fitted mesh is used to simulate the flow. The cylinder in cross flow is a classic Navier-Stokes PDE simulation which exhibits laminar vortex shedding for low Reynolds numbers, with expected Strouhal numbers nearing $St = 0.2$.

The mesh, as seen in fig. 4.7, has non uniform node spacing, with skewed elements in the region surrounding the cylinder. The mesh is 3 dimensional, 1 element thick in the spanwise direction, for total of 4200 nodes. The flow field data for training was simulated using HONEE [29], to solve the weak form, SUPG-stabilized, primitive variable set of the compressible Navier-Stokes equations, as presented in [63]. The flow solutions are determined using an implicit SUPG time stepping method, with a time step of 0.05s. The Initial

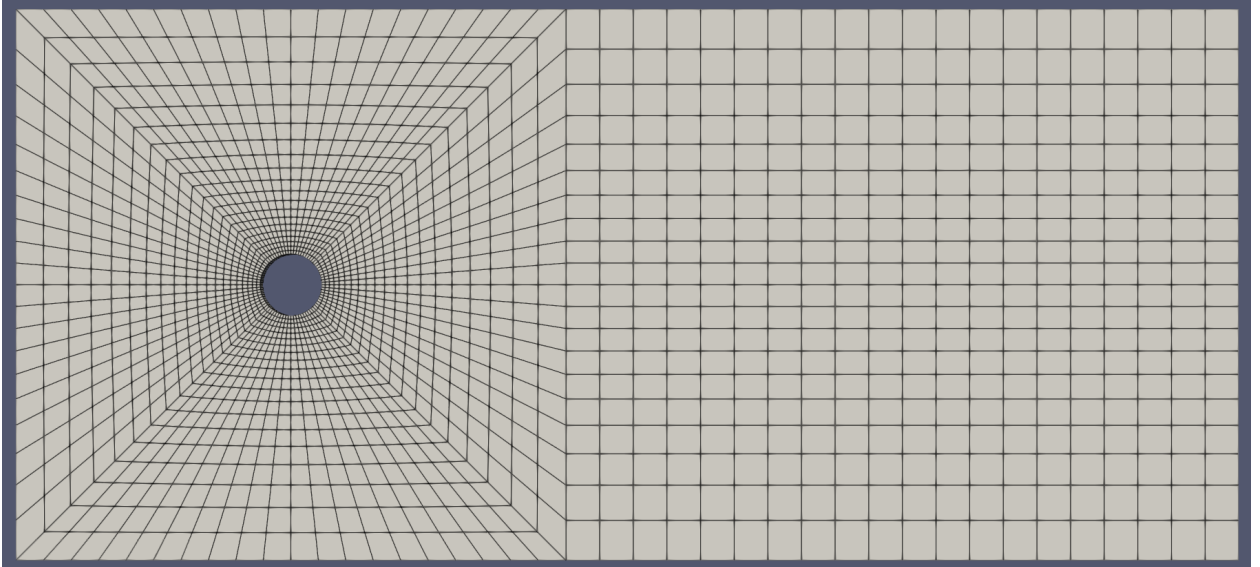


Figure 4.7: 3D unstructured mesh used in vortex shedding case.

freestream velocity is set to $U = [1, 0, 0]m/s$, viscosity to $\mu = 0.01kg/m \cdot s$, and the cylinder

diameter, the characteristic length for the problem, is set to $D = 1m$. The flow then has Reynold number of $Re = 100$. Pressure and temperature are also set to $P = 7143Pa$ and $T = 24.92K$, such that freestream mach number is $Ma = 0.01$ and thermal conductivity is set to $k = 14.34W/m \cdot K$, to ensure typical Prandtl number for air of $Pr = 0.71$.

The simulation was run for a total of 5000 time steps, such that the flow has developed steady vortex shedding physics. Each shedding cycle has a duration of approximately 5.55s, or every 111 time steps, for a shedding frequency of 0.18 Hz. The flow's Strouhal number which is a function of flow field fluctuations over time is given the following equations:

$$St = \frac{fD}{U} \quad (4.19)$$

where f is the frequency of vortex shedding, $D = 1$ m and $U = 1$ m/s are the cylinder diameter and freestream velocity as previously described. The flow's Strouhal number is $St = 0.18$, which is nearing the expected $St = 0.2$ for these flows.

4.4.1 MNO model and training set-up

For the purpose of training the MNO model, the 5000 time steps were split into 250 non-overlapping samples of 20 time steps each. Each sample neither spans the full shedding cycle nor is a natural fraction of one, encouraging the network to learn the underlying Navier-Stokes dynamics rather than a simpler periodic function.

The first 4000 time steps, which include the transition from the uniform flow initial state to fully developed vortex shedding, were used as training data, and the remaining 1000 time steps, where shedding has reached a steady condition, is used for testing. This equals to a sample split of 200 and 50 for training and testing, respectively.

We applied the MNO architecture to learn approximation models for the velocity magnitude field and the pressure field. The goal of these model was to determine the operator which maps the respective fields at sample time steps $t \in [1, 10]$ to fields up to $t \in (10, 20]$. Therefore, the learned operators were $\mathcal{D}_{|\vec{u}|} : |\vec{u}|_{(0,1)^2 \times [1,10]} \mapsto |\vec{u}|_{(0,1)^2 \times (10,20]}$ and

$\mathcal{D}_p : p_{(0,1)^2 \times [1,10]} \mapsto p_{(0,1)^2 \times (10,20]}$ for velocity magnitude and pressure, respectively. Training and testing were performed using an RNN style input data architecture as was done in section 4.3

The M-GNN model had the following architecture: FNN-GCN lifting, from 13 to 64 channels, 4 layers of M-GNN, followed by 64 to 32 to 1 channel linear transformations, and GeLU activation. The AdamW optimizer was set-up with Cosine Annealing learning rate scheduling, with initial $LR = 1e - 2$, final $LR = 1e - 5$, weight decay of $1e-3$, batch size of 20. The GNN model had the same set-up, with exception of utilizing 4 layers of GNN instead of M-GNN. The models were trained over 500 epochs.

Note that the maximum pressure fluctuation magnitude is on the order of 0.0134% of the mean pressure field. Due to such a large difference in order of magnitude, when passing in the total pressure field, the neural network settles into determining the mean pressure field instead of identifying the mechanisms driving the fluctuations. Thus, when determining the mapping operator for pressure, $\mathcal{D}_p$, the time dependent mean pressure across samples and spatial position, $\bar{p}(t)$, was subtracted from each respective time step such that the model could learn the pressure fluctuations.

To avoid data leakage between training and testing, $\bar{p}(t)$ was calculated from the training samples exclusively, and subtracted from both the training and testing data. The total predicted pressure fields were obtained in post-processing by re-adding $\bar{p}(t)$.

4.4.2 MNO model training results - Velocity magnitude

The first test was performed on the velocity magnitude flow field. The testing l2 error is shown in fig. 4.8a, where the GNN architecture was also tested as a comparison. Once again, M-GNN far outperforms the GNN architecture, settling in to $\sim 0.7\%$ compared to $\sim 3.3\%$ l2 error after 500 epochs. Observing the time dependent RMSE error across test samples and spatial position in fig. 4.8b, the M-GNN performs well with an RMSE of $\sim 0.7\%$ across all time steps, indicating that the error introduced in a predicted timestep, and used

as a new input to the model, minimally effects the following timestep prediction.

While low l2 test errors are promising, a successful approximation model should be able to obtain solutions which maintain the characteristics of the vortex shedding flow, notably the Strouhal number, and the pressure lift and drag forces experienced by the cylinder. We first examine the adherence to Strouhal number obtained from velocity magnitude.

When determining Strouhal number from the prediction data, only the last 10 time steps in each outputted 20 time step sample are predicted values. Thus, a Lomb-Scargle Periodogram method is required in order to handle the gaps in prediction data when samples are re-ordered chronologically. The resulting power spectrum analysis can be observed in fig. 4.9, where it is clear the models velocity magnitude predictions maintain the shedding frequency of 0.18 Hz.

M-GNN has better agreement to the truth power, whereas GNN over-predicts the power, and thus the amplitude of the oscillations. This is further apparent in fig. 4.10, where modeled velocity magnitude of the downstream sample point were compared against the full truth data for both the M-GNN and GNN models.

The prediction quality between the two models is now very apparent. Observing the top plot in fig. 4.10 M-GNN adheres to truth data with minimal discrepancy, consistent with the reported RMSE values previously discussed. GNN's predictions show good agreement in windows of steady acceleration, but fail in windows of large acceleration fluctuations.

This is further corroborated in the middle plot, where $error = Prediction - Truth$ (left y-axis) is plotted for both models. The velocity magnitude truth (dashed line, right y-axis) is included in this middle plot as a visual reference to identify the portions of the fluctuation cycle where these errors occur. M-GNN has consistently low errors from sample to sample, whereas GNN's error magnitudes fluctuate between samples. These fluctuations correspond to over-shooting the magnitude in each sample, as error is positive when velocity magnitude is positive, and vice-versa.

The bottom plot in fig. 4.10 shows relative error of the predictions for all 50 sample

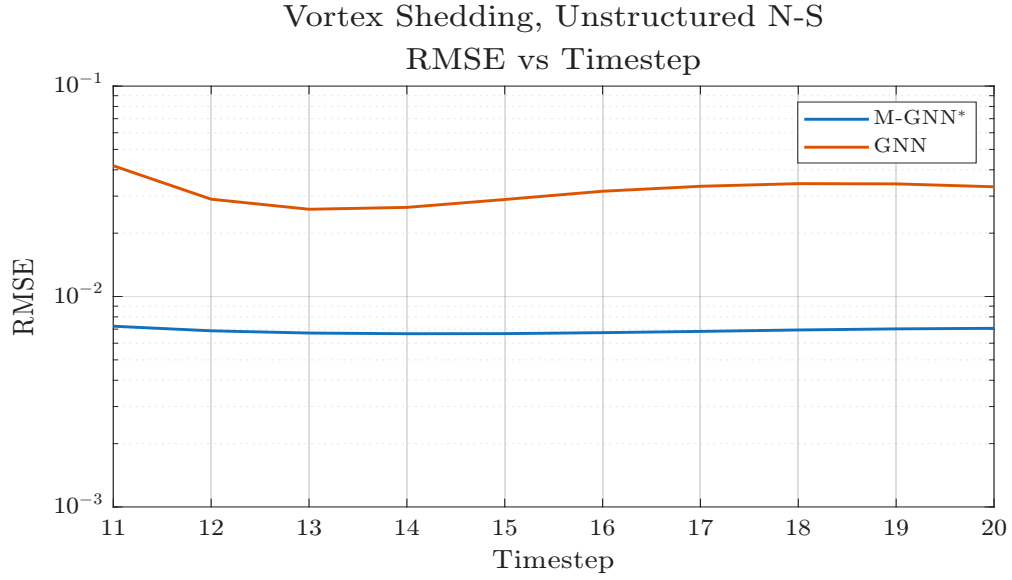
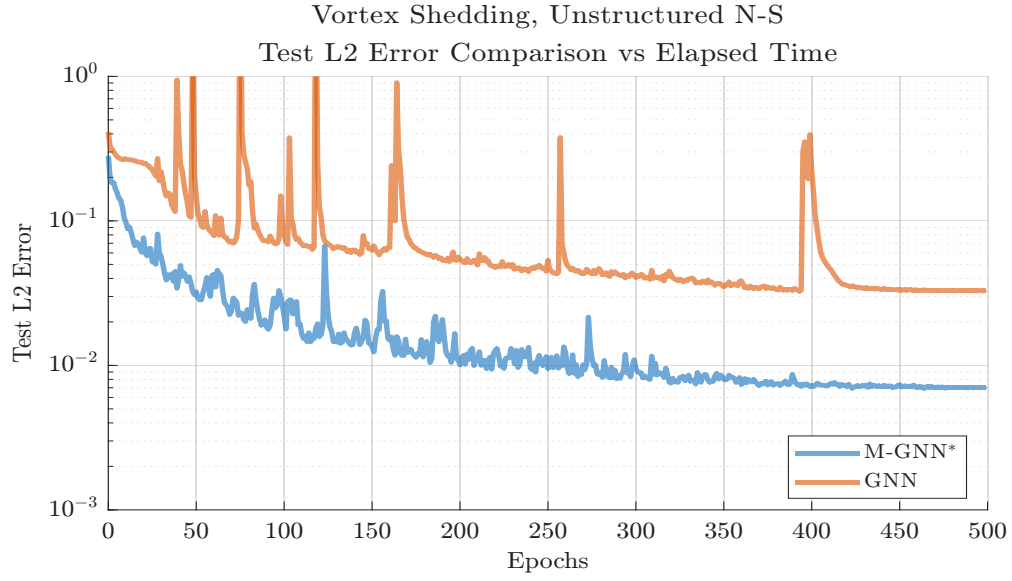


Figure 4.8: Approximation model errors for velocity magnitude, $|\vec{u}|$, of the 2D Navier-Stokes problem. a) l2 test over epochs. b) RMSE across samples and position vs predicted timestep. Superscript * denotes non-periodic for that branch.

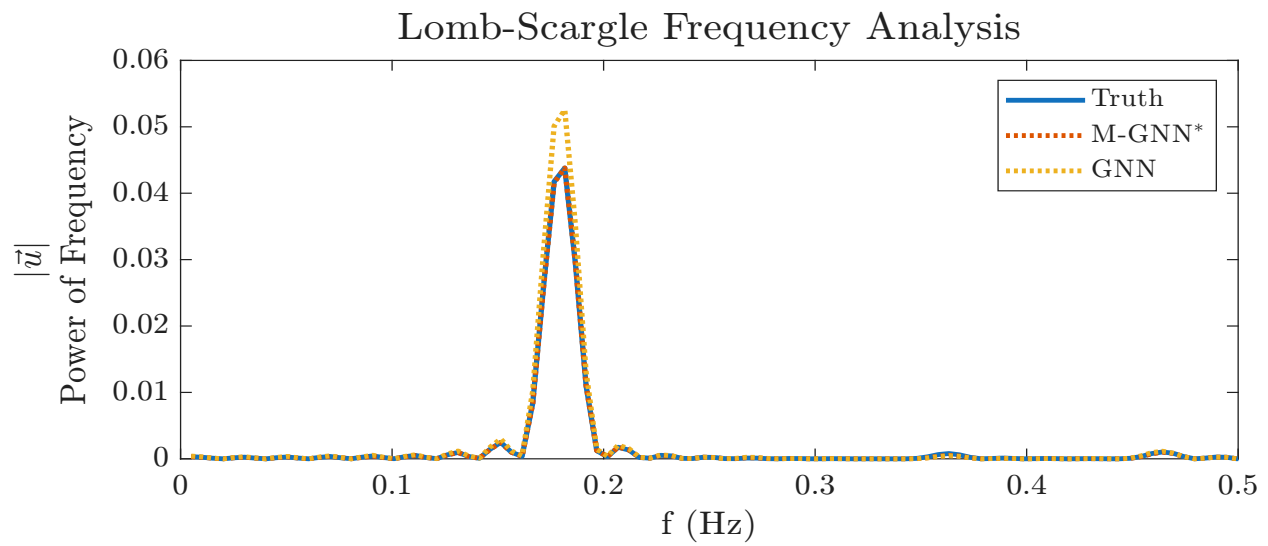
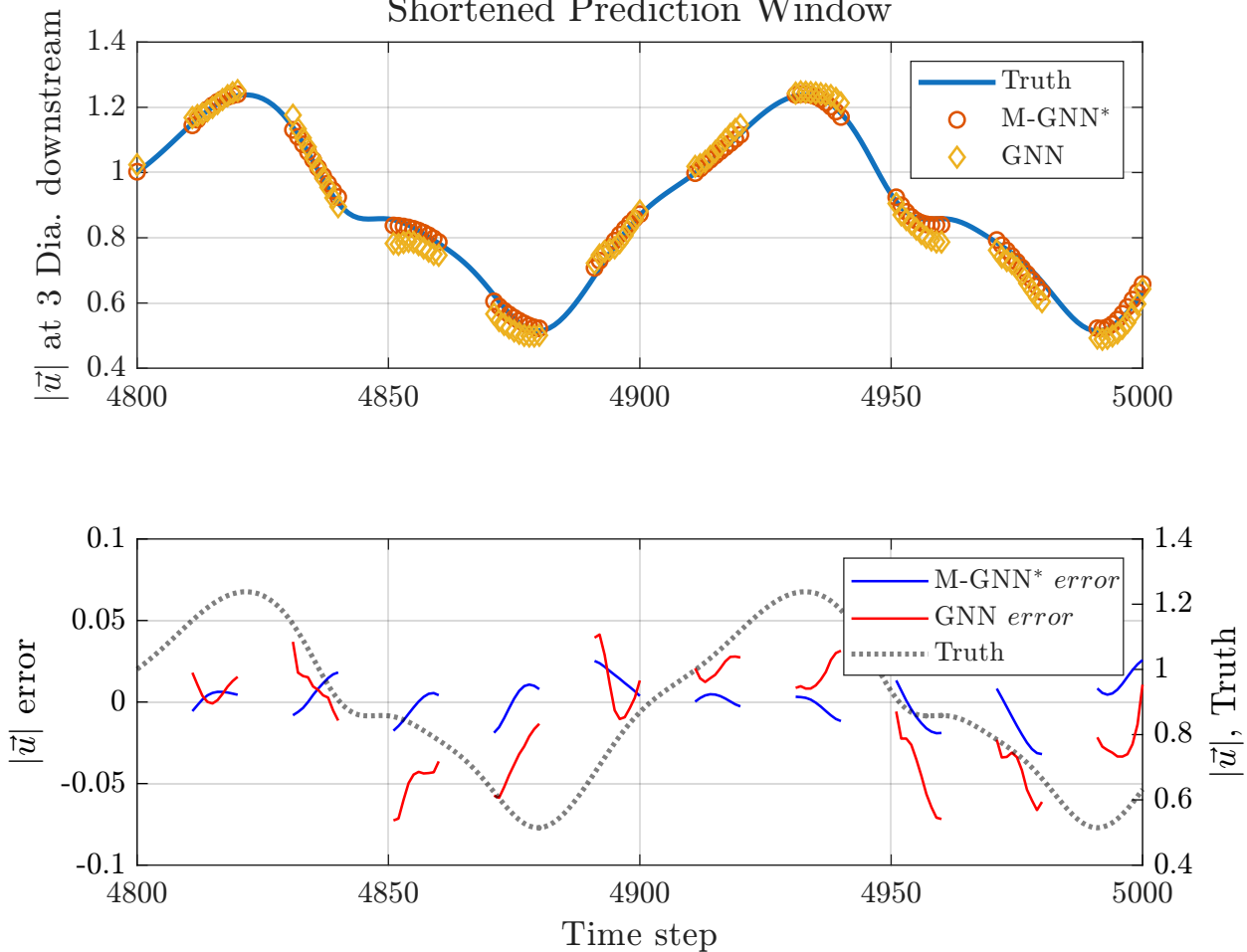


Figure 4.9: Lomb-Scargle power density analysis of velocity magnitude. Comparison of M-GNN and GNN to truth

Vortex Shedding, Unstructured N-S

 $|\vec{u}|$ - Predicted vs Truth

Shortened Prediction Window



Full Prediction Window

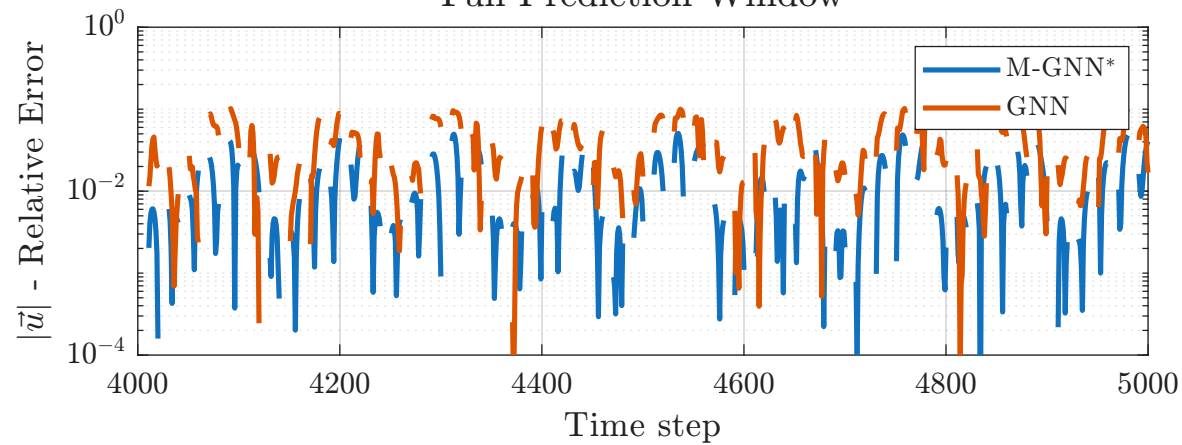


Figure 4.10: Top to bottom: Model predictions compared to truth; $error = prediction - truth$, with truth data as visual reference; relative error for all predicted time steps.

time windows. Again, M-GNN has consistently lower error across all time steps. We will now examine how the M-GNN model performs when the predicted fields are used to determine derived quantities of interest.

4.4.3 MNO model training results - Pressure

As mentioned, other important derived quantities from this vortex shedding flow are the model's predicted pressure lift and drag forces experienced by the cylinder. As M-GNN was shown to be far superior to GNN in previous tests, the pressure field was modeled using only the M-GNN architecture to determine mapping operator $\mathcal{D}_p$.

The pressure lift and pressure drag experienced by the cylinder can be calculated by numerically integrating the pressure field over the surface of the cylinder. This is accomplished by post-processing the fields in ParaView, where nodal pressure field values (ParaView point data) for the cylinder surface are first converted to element face values (ParaView cell data). The face normals and areas of the elements of the cylinder surface are extracted and multiplied against the element face pressure values. This ParaView cell data field is then integrated over the entire cylinder face surface to determine the resulting pressure lift and drag forces experienced by the cylinder. This calculation is performed for both the truth data and prediction data for each individual time step. Modeling error and physical statistics were determined for all modeling time steps.

To better understand the mathematically derived lift and drag results, let's first observe the average modeled pressure error on the surface of the cylinder. Figure 4.11 shows the signed (+ or -) time-averaged relative error of pressure on the surface of the cylinder.

The signed relative errors on the surface of the cylinder shows an under-prediction of pressure (negative error, blue region) for the upstream portion (-x dir) of the cylinder, with better agreement in the downstream region (+x dir). Due to this asymmetry about the x-normal plane, further results are expected to show a consistent under-prediction in mean pressure drag force. While under predicting the upstream pressure, the errors have much



Figure 4.11: Signed time-averaged relative error of pressure field predictions on surface of cylinder.

Table 4.4: Pressure lift and drag prediction statistics

Statistic	Pressure Lift			Pressure Drag		
	Truth	Prediction	%Diff	Truth	Prediction	%Diff
Mean	1.5809e-3	1.4309e-3	-9.5	1.1376e-1	1.1251e-1	-1.1
Amplitude	3.0556e-2	2.9699e-2	-2.8	8.5352e-4	9.4397e-4	10.6

better symmetry about the y-normal plane, and thus we do not expect any consistent over- or under-predictions in mean lift force.

Next, we analyze the periodicity of the predicted lift and drag, and compare it against it's truth values. This was once again performed utilizing a Lomb-Scargle analysis on the predicted time steps, as was done for velocity magnitude. The lift frequency is expected to match the shedding frequency of $\sim 0.18\text{Hz}$, and drag is expected to be twice that of lift, $\sim 0.36\text{Hz}$, a known behavior of vortex-induced drag. Figure 4.12 shows the dominant frequencies of the power spectra analysis.

As expected, the predicted frequency values agree with the expected values. The power of frequency for lift is slightly less than the truth, thus slightly under-predicted peak oscillation amplitude is expected for lift. Conversely, the power frequency of drag is greater than the truth, thus we expect an over-prediction of drag oscillation amplitude in our continued analysis.

The lift and drag prediction results for all time steps are observable in figs. 4.13 and 4.14 where each figure shows, from top to bottom, the predicted versus truth values, error, and relative errors. Mean and amplitude statistics for lift and drag are presented in table 4.4, and these statistics will now be discussed in context of results in figs. 4.13 and 4.14. We pretext this discussion with the note that resulting lift profile of a vortex shedding cylinder is most dominated by amplitude of oscillations, while the drag is mean dominated.

Lift Results Observing lift results in fig. 4.13 predictions have good agreement with the truth, with only a slight under-shoot of the oscillation amplitude, as expected

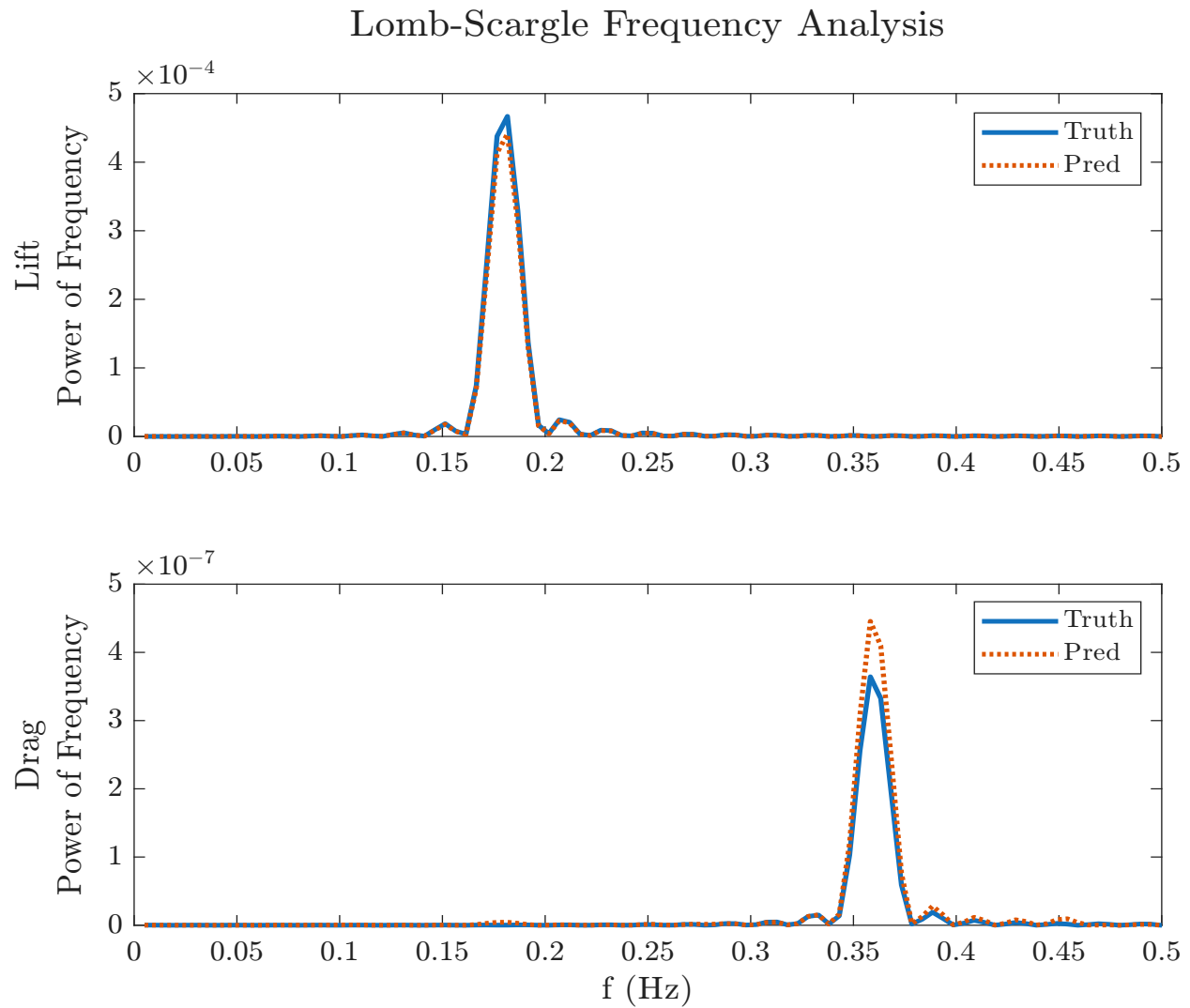


Figure 4.12: Lomb-Scargle power density analysis of lift (top) and drag (bottom). Predicted and truth.

from the power spectra analysis. This under-shoot is also corroborated in the middle plot, where error oscillations are 180° out of phase with the lift. This results in a -2.8% percent difference in amplitude as presented in table 4.4. The bottom plot shows the relative error for all prediction windows, which remains consistent throughout all time steps. The large relative error peaks correspond exactly with prediction windows passing through 0 lift, and are numerical artifacts due to dividing by values approaching 0.

As mean lift is near-zero, the relative errors are numerically inflated, yet the relative error norm is kept low at 3.5% . The mean lift percent difference of -9.5% in table 4.4 is similarly a numerical artifact of the near-zero mean, and carries minimal physical significance. The RMSE of lift is 0.07% across all 500 predicted time steps. The model thus performs well in predicting lift due to low and consistent errors in oscillation magnitudes.

Drag Results Observing drag results in fig. 4.14 predictions follow the same oscillations as the truth. However, the predictions consistently under-predict the mean drag. This under-prediction is most easily seen in the middle plot, where errors for all samples are negative, which is consistent with the signed mean relative error for pressure in fig. 4.11 where pressure at the front of the cylinder is under-predicted, and the rear of the cylinder is closer to truth. As with lift, relative error in the bottom plot is consistent across all samples.

While being under predicted, the modeled drag forces have a relative error norm of 1.15% , with a RMSE of 0.13% across all 500 predicted time steps. The drag force oscillation amplitude, while showing a $+10.6\%$ percent difference, represents an absolute error of less than 1% of the mean drag force, which is considered negligible. The learned model thus performs well at capturing drag, with low errors in the mean, it's dominant component.

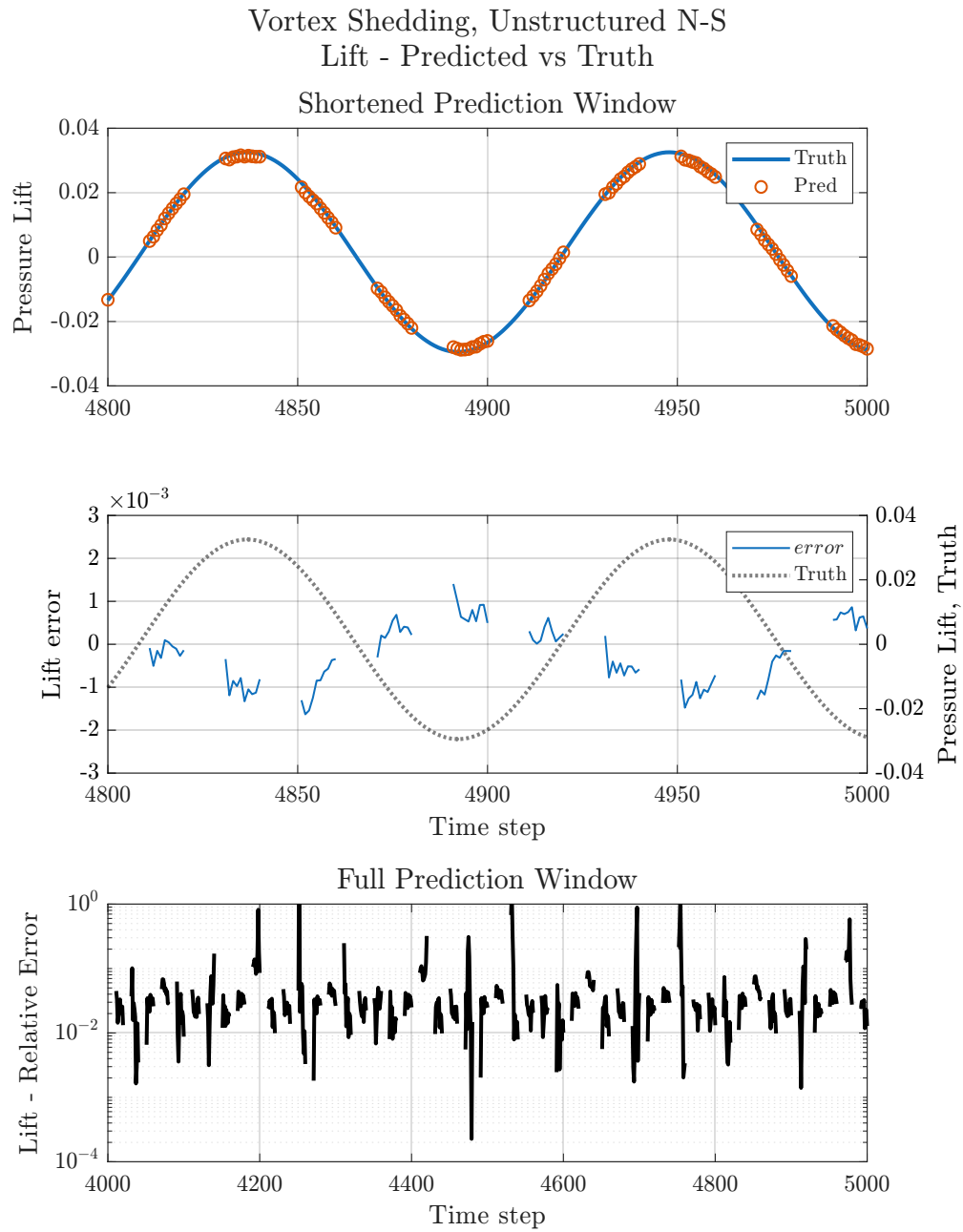


Figure 4.13: Integrated pressure lift forces from modeled pressure field. Top to bottom: Model predictions compared to truth; $error = prediction - truth$, with truth data as visual reference; relative error for all predicted time steps.

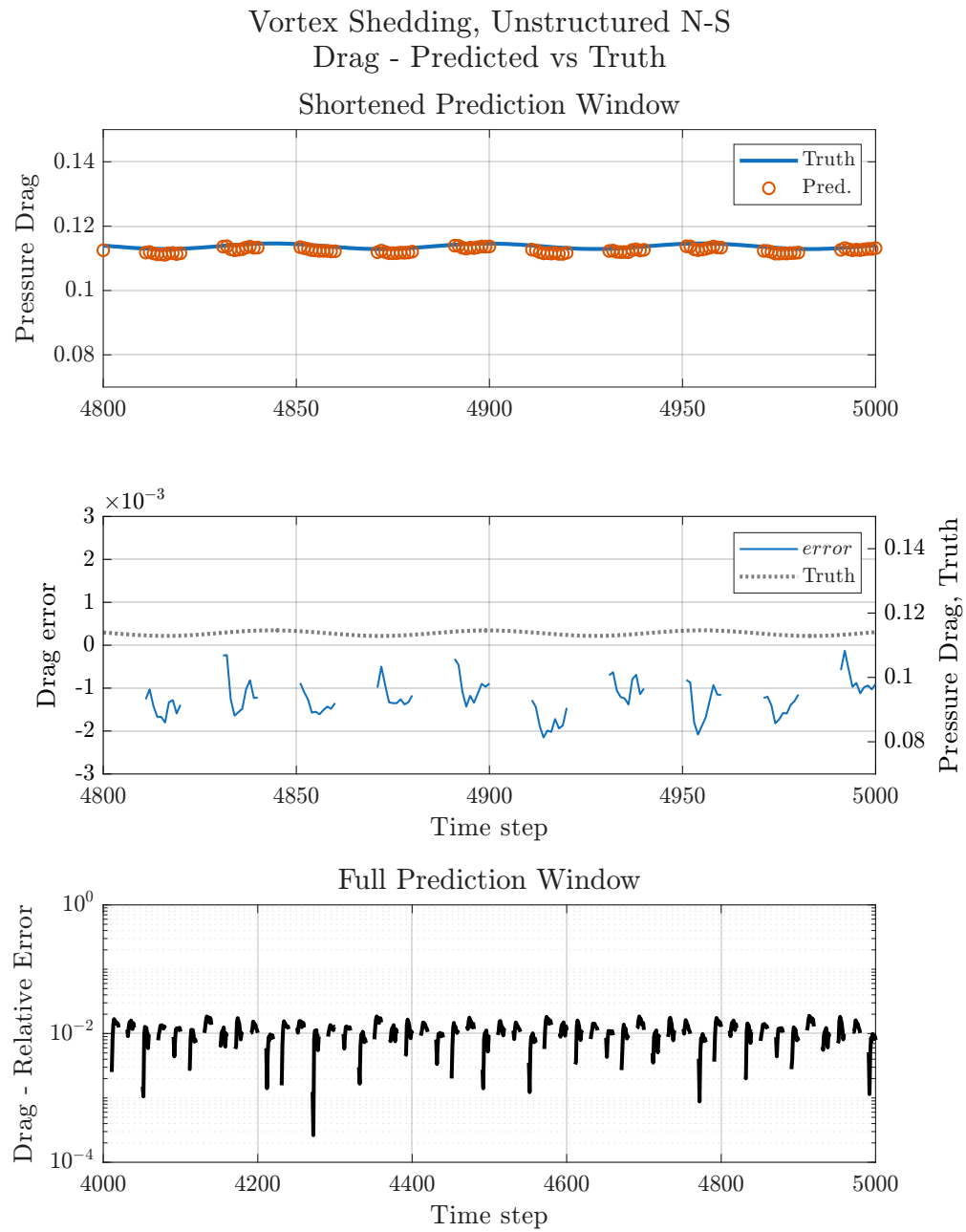


Figure 4.14: Integrated pressure drag forces from modeled pressure field. Top to bottom: Model predictions compared to truth; $error = prediction - truth$, with truth data as visual reference; relative error for all predicted time steps.

4.5 Software Implementation

As previously mentioned in chapter 2, the implementation of the MNO architecture is available in https://github.com/Jeff-Hadley/MNO_Toolbox.git. Example model training implementations can be found in the files beginning with `MNO_3D_VortexShedding`. The following line numbers make reference specifically to the `MNO_3D_VortexShedding_0_01LR_20BS_3Levels_Pressure.py` example (This file will be referred to as the executable). Lines 254 to 386 are where the entire MNO architecture model class is defined. Within that MNO class, lines 284 to 309 are where the lifting $P(\cdot)$, upper $G(\cdot)$ and lower $W(\cdot)$ branch layers, and the final projection $Q(\cdot)$ are defined and initialized. The `forward` function starting at line 341 is how input data is passed through the network, this block of code is equivalent to the architecture presented in fig. 4.1, where each MNO layer is a loop at line 352.

The remainder of the file, starting at line 389 is the main execution of the training and testing of the model. The example is highly commented and fairly self explanatory. The following are the main ‘blocks’ of code to build and train the model:

lines 389–512: Sets tunable parameters for the optimizer, scheduler and epoch loop based on the user inputs to the executable. Loads in and manipulates the training and testing data such that it is put into PyTorch tensor format compatible with the MNO architecture.

lines 514–532: Loads the training and testing data into the PyTorch training device (CPU or GPU).

lines 536–566: Creates the MNO model based on the architecture previously defined and the user inputs to the executable. Also initializes the learning optimizer and the learning rate scheduler from user inputs.

lines 582–800: This is the training block, it sets the requested epochs to start from (if restarting from a checkpoint), then loops over all training epochs. Performs the

forwards and backwards passes of the optimizer, and records and reports current training and testing errors.

lines 802–839: Uses the final learned model as a prediction tool and reports performance.

lines 841–860: Saves the model and model dictionary state such that you can use the model in another application if desired.

4.6 Summary and Conclusions

This chapter presented the development and evaluation of the Multilevel Neural Operator, a novel neural operator architecture designed for learning surrogate models of complex PDEs on unstructured domain discretizations. The MNO is motivated by the practical limitations of existing neural operators, namely GNO’s prohibitive $\mathcal{O}(N^2)$ scaling, and FNO and WNO’s restriction to uniformly discretized structured grids. The MNO was developed to fill a gap left by these methods, targeting the complex unstructured geometries that define real engineering problems.

In its design, the MNO follows the dual-branch architecture established by FNO and WNO, replacing their Fourier and wavelet decompositions with A-MGARD in the upper integral kernel branch, and substituting the classical CNN convolution in the lower branch with a graph Laplacian convolution (GNN) to embed mesh connectivity information. The theoretical basis for replacing the wavelet or Fourier transform with A-MGARD rest on two observations.

First, the learned tensor R_ϕ approximates the action of the integral kernel in transformed space, and is therefore agnostic to the specific transformation used, provided completeness of the decomposition and recomposition is satisfied, i.e. $u = \mathcal{M}^{-1}(\mathcal{M}(u))$.

Second, and more broadly, we argued that MNO satisfies the condition for neural operator classification on the same theoretical footing as FNO and WNO; universal approx-

imation is achieved asymptotically through basis refinement of the A-MGARD multilevel hierarchy, and discretization invariance is recoverable through interpolation, consistent with Kovachki et al.’s own framework [31].

Crucially, unlike the Fourier and wavelet bases whose basis refinement argument is realizable on uniform structured grids, A-MGARD’s basis refinement is realizable directly on unstructured discretizations, as its function space hierarchy is derived from the problem’s own domain discretization. Additionally, a method for embedding periodic boundary conditions directly into the mass and Laplacian matrices used by MNO was introduced, enabling the network to leverage known problem periodicity without neural network architectural modifications.

The MNOs architectural design choices were validated through a series of structured and unstructured test cases. A periodicity and branch architecture study on 1D advection wave and 2D Navier-Stokes problems on structured grids demonstrated that embedding periodicity in the upper kernel integral branch significantly improved model performance for periodic problems. Results showed that MNO architecture approached WNO performance on structured domains despite being designed for more general unstructured settings. The study also revealed that GNN convolution outperforms the classical CNN lower branch on more complex problems, confirming that embedding mesh connectivity in the lower branch is beneficial when spatial relationships cannot be inferred from data storage ordering alone.

The MNO was then applied to a physically meaningful engineering problem, a vortex shedding cylinder in cross flow discretized on an unstructured body-fitted mesh, where current state-of-the-art neural operators are not applicable. The M-GNN version of the MNO architecture successfully learned surrogate models for both velocity magnitude and pressure fields, capturing dominant physics of the vortex shedding flow with low and consistent errors across all predicted time steps. Frequency analyses confirmed near-perfect adherence to expected Strouhal number, and derived quantities of lift and drag forces experienced by the cylinder were predicted with relative error norms of 3.5% and 1.15% respectively. This

demonstrated that the MNO is capable of learning the underlying flow physics directly from data, while being agnostic to the governing equations.

Together, these results demonstrate that the MNO is a viable and practically feasible neural operator architecture for unstructured domains, trading discretization independence for applicability to the complex geometries encountered in real engineering problems. The A-MGARD decomposition, originally developed for data compression purposes, proves to be broadly applicable mathematical tool, serving equally well as the foundation of a neural operator architecture as it does as a decomposition method for data compression pipelines.

4.7 Future Work

The work presented in this chapter is hopefully only the beginning of research efforts that can be put forth towards expanding and improving the use of A-MGARD decomposition in neural network architectures. As with all first attempts at developing neural network architectures, further computational efficiency is likely to be found by performing optimization studies. Each operator of the architecture is likely to have potential improvements; whether it be the choice of activation functions for $G(\cdot)$ and $g(\cdot)$, the channel reduction factors of projection $Q(\cdot)$, number of layers for $K(\cdot)$, etc. The "final" architecture presented in this work was attained by a trial and error approach, with general guidance from other neural operators like MNO and FNO, and yielded impressive results with minimal adjustments to typical neural network parameters.

In expanding it's use, the MNO could be used as an unstructured domain surrogate model in Learned Gradient Flow (LGF) optimizer research [48]. LGF frameworks are learning surrogate ODE models of optimizers, in an effort to replace traditional optimizers, like ADAM, where expensive gradient and objective function evaluations must occur. Having a more efficient optimizer applicable to unstructured domains would accelerate the surrogate model discovery of complex PDE's that was presented in this work.

In a parallel line of thinking, the MNO framework should be inserted into a PDE

solver as an in-situ initial guess generation tool for residual based time-stepping methods of extensively long simulations. Direct numerical simulations (DNS) could benefit greatly from accurate time-advanced initial guesses in the residual iteration cycle, reducing the total amount of iterations and reducing solver time. Practically, the model would be trained on a parallel process from the beginning of the DNS, expanding the training and testing sample sets as greater amounts of time steps become available. Once a sufficient amount of samples are available, and training and testing errors of the model reach a desired tolerance, the model could then be activated inside of the DNS iteration loop to generate improved initial guesses for the residual solve.

4.7.1 Proof of concept results of using MNO as a PDE solver initial-guess generator

As a proof of concept of using the MNO as a initial guess generator, a separate model training was performed on a refined domain vortex shedding case. This new training case used a discretization which is twice as fine as the original case in section 4.4. Instead of learning each field individually, the MNO model was configured to simultaneously learn 4 of the 5 fields in the solution state $\mathbf{Y}$ generated by the HONEE simulation: x-dir velocity u , y-dir velocity v , pressure p , and temperature T . The spanwise (z-dir) velocity, w , was omitted as it is a constant 0, and is not required to be learned for this problem.

New data was generated for this problem, using a new time step of 0.40s and changing the internal HONEE PDE solver to use a gen-alpha [30] iterative scheme.

The data used for training was also shifted to a later period in the simulation, such that training and testing data only includes time steps of fully developed vortex shedding. Additionally, as time step for the solver was increased from the original 0.05s to 0.40s, a single shedding period is now fully covered by 14 time steps. As the training window remained at using 20 time steps, the model training now sees more than a full period of the vortex shedding oscillations.

To determine whether the use of MNO predictions could accelerate the convergence of the gen-alpha solver, the initial guess solution state, which is usually a persistence ‘same $\mathbf{Y}$ ’ (use the last solved timestep solution), is replaced by the advanced state predicted by the MNO model from previous timestep inputs. Table 4.5 shows a series of arbitrarily chosen time steps for each method, where the amount of non-linear iterations, linear iterations, and the residuals for each iteration are shown.

Table 4.5 demonstrates that MNO can provide linear solver iteration count savings, as well as non-linear iteration savings. While the first non-linear residual is consistently higher for MNO, the subsequent residuals decrease faster than for ‘Same $\mathbf{Y}$ ’. This results in MNO only requiring 4 non-linear iterations where ‘Same $\mathbf{Y}$ ’ requires 5. Additionally, the total amount of linear iterations for MNO is on average less than for ‘Same $\mathbf{Y}$ ’. These trends were observed throughout the entire simulation run, and MNO averaged 20% less linear iterations than ‘Same $\mathbf{Y}$ ’ for the same convergence criteria.

These proof of concept results are a promising first attempt at using the MNO model as an iterative solver accelerator, and further work is actively being pursued to streamline and solidify its use in this application.

Table 4.5: Comparison of solver iteration counts between ‘Same $\mathbf{Y}$ ’ and MNO prediction. listed are linear and non-linear iteration counts, residual norms, and total linear iteration counts for each respective method.

	Same Y			MNO		
Time Step	SNES Iteration	Residual Norm	Linear Iterations	SNES Iteration	Residual Norm	Linear Iterations
48	0	2.20E+01	61	0	2.71E+02	90
	1	6.69E+00	175	1	2.42E+00	133
	2	3.30E-01	181	2	3.85E-02	166
	3	1.56E-03	207	3	1.86E-04	132
	4	1.56E-05	167	Converged	1.97E-06	
	Converged	1.22E-07	Total 791			Total 521
49	0	1.91E+01	115	0	2.64E+02	109
	1	6.71E+00	206	1	2.36E+00	135
	2	3.30E-01	206	2	1.97E-02	170
	3	1.53E-03	200	3	1.62E-04	143
	4	1.49E-05	181	Converged	1.47E-06	
	Converged	1.13E-07	Total 908			Total 557
50	0	1.41E+01	140	0	2.59E+02	133
	1	6.75E+00	265	1	2.29E+00	193
	2	3.31E-01	349	2	3.73E-02	268
	3	1.56E-03	157	3	2.05E-04	235
	4	1.35E-05	117	Converged	1.87E-06	
	Converged	9.83E-08	Total 1028			Total 829
51	0	1.13E+01	91	0	2.62E+02	157
	1	6.78E+00	117	1	2.38E+00	238
	2	3.34E-01	174	2	4.22E-02	294
	3	1.59E-03	175	3	2.94E-04	388
	4	1.29E-05	151	Converged	2.31E-06	
	Converged	8.72E-08	Total 708			Total 1077
			Total 3435			Total 2984

Chapter 5

Dissertation summary and final thoughts

This dissertation presented the development and application of a novel algebraic multigrid based multilevel decomposition method, A-MGARD, for truly unstructured scientific data. The work was motivated by a critical and largely unaddressed gap in the scientific computing landscape: while computational simulations of complex engineering problems generate immense volumes of data on highly unstructured meshes, the majority of existing data compression and machine learning methods are fundamentally limited to structured or block-structured discretizations. A-MGARD was developed to bridge this gap, and its utility was demonstrated across two distinct and practically significant application domains.

Chapter 2 presented the theoretical derivation and practical implementation of A-MGARD. By amalgamating the multilevel decomposition framework of Ainsworth et al. [1, 4, 3, 2] with algebraic multigrid coarsening methods [53, 10], A-MGARD determines the required hierarchy of function spaces, prolongation operators, and mass matrices directly from a mesh’s weighted graph Laplacian, without any assumptions on how the mesh was originally created. This distinguishes A-MGARD from its geometric counterpart MGARD, which is restricted to meshes generated through congruent element refinements. The resulting decomposition produces multilevel coefficients that carry a rich multiscale representation of the original data, occupy no more storage than the original solution vector, and recover the original solution within machine precision when no modifications are made.

Chapter 3 demonstrated A-MGARD’s application as a substitute decomposition method

within a standard lossy data compression pipeline, benchmarked against state-of-the-art compressors MGARD and SZ3 on a series of increasingly complex and unstructured mesh configurations derived from turbulent channel flow data. On meshes where both methods were applicable, A-MGARD demonstrated performance parity with G-MGARD while showing greater robustness on highly anisotropic mesh configurations. As mesh complexity was increased through nodal perturbations and Delaunay triangulation to configurations where G-MGARD was no longer applicable, A-MGARD maintained substantial advantages over SZ3, achieving PSNR gains of 20–23 dB in the worst-case unstructured configurations. Compression factor improvements of 1.8x to 4.6x over SZ3 on randomly permuted data confirmed that A-MGARD’s mesh-informed decomposition produces a significantly more compressible representation of unstructured scientific data than simply processing it as a 1D array with SZ3.

Chapter 4 extended A-MGARD beyond data compression entirely, integrating it as the core of the Multilevel Neural Operator (MNO) for learning surrogate models of complex PDEs on unstructured domains. By replacing the Fourier and wavelet decompositions used in FNO and WNO with A-MGARD, the MNO inherits the expressive multiscale function space learning approach of those methods while relieving their structured grid restrictions. Validation on structured 1D advection wave and 2D Navier-Stokes problems demonstrated that MNO architectures approach WNO performance on structured domains despite being designed for the more general unstructured setting. Applied to the physically meaningful unstructured engineering problem of vortex shedding cylinder in cross flow, the MNO successfully learned surrogate models for velocity magnitude and pressure, capturing dominant flow physics with low and consistent errors, and yielding lift and drag force predictions with relative error norms of 3.5% and 1.15% respectively, on a problem where current state-of-the-art neural operators are not applicable.

Taken together, the contributions of this dissertation tell a unified story: a decomposition method developed out of necessity for unstructured data compression proves to be a

broadly applicable mathematical tool. A-MGARD’s ability to construct meaningful hierarchical function spaces from any unstructured mesh, without geometric assumptions, is the key capability that enables its utility across both application domains. In data compression, this hierarchy produces a multiscale coefficient representation that is more amenable to quantization and encoding than treating original unstructured data as a flat 1D array. In machine learning, this same hierarchy provides an enriched transformed function space in which a neural network can learn the action of an integral kernel operator, capturing multiscale and localized features of complex PDE solutions on geometries that existing neural operators cannot practically handle.

The broader implication of this work is that the structured grid assumption embedded in many existing scientific computing methods is not a fundamental requirement, but rather a practical implementation convenience that comes at the cost of generality. A-MGARD demonstrates that algebraic approaches to constructing function space hierarchies can substitute geometric ones without sacrificing performance, opening the door to a greater class of practically relevant problems in both data compression and machine learning for engineering applications.

Bibliography

- [1] Mark Ainsworth, Ozan Tugluk, Ben Whitney, and Scott Klasky. Multilevel techniques for compression and reduction of scientific data—the univariate case. Computing and Visualization in Science, 19(5):65–76, 2018.
- [2] Mark Ainsworth, Ozan Tugluk, Ben Whitney, and Scott Klasky. Multilevel techniques for compression and reduction of scientific data—the multivariate case. SIAM Journal on Scientific Computing, 41(2):A1278–A1303, 2019.
- [3] Mark Ainsworth, Ozan Tugluk, Ben Whitney, and Scott Klasky. Multilevel techniques for compression and reduction of scientific data—quantitative control of accuracy in derived quantities. SIAM Journal on Scientific Computing, 41(4):A2146–A2171, 2019.
- [4] Mark Ainsworth, Ozan Tugluk, Ben Whitney, and Scott Klasky. Multilevel techniques for compression and reduction of scientific data—the unstructured case. SIAM Journal on Scientific Computing, 42(2):A1402–A1427, 2020.
- [5] Satish Balay, Kris Buschelman, William D Gropp, Dinesh Kaushik, Matthew G Knepley, L Curfman McInnes, Barry F Smith, and Hong Zhang. *Petsc*, 2001.
- [6] Nathan Bell, Luke N Olson, Jacob Schroder, and Ben Southworth. Pyamg: algebraic multigrid solvers in python. Journal of Open Source Software, 8(87):5495, 2023.
- [7] Kaushik Bhattacharya, Bamdad Hosseini, Nikola B Kovachki, and Andrew M Stuart. Model reduction and neural networks for parametric pdes. The SMAI journal of computational mathematics, 7:121–157, 2021.
- [8] Blosc Development Team. A fast, compressed and persistent data store library, 2009-2023. <https://blosc.org>.
- [9] James Brannick, Marian Brezina, S MacLachlan, T Manteuffel, S McCormick, and J Ruge. An energy-based amg coarsening strategy. Numerical linear algebra with applications, 13(2-3):133–148, 2006.
- [10] William L Briggs, Van Emden Henson, and Steve F McCormick. A multigrid tutorial. SIAM, 2000.

- [11] Jed Brown, Ahmad Abdelfattah, Valeria Barra, Natalie Beams, Jean-Sylvain Camier, Veselin Dobrev, Yohann Dudouit, Leila Ghaffari, Tzanio Kolev, David Medina, et al. libceed: Fast algebra for high-order element-based discretizations. Journal of Open Source Software, 6(63):2945, 2021.
- [12] Martin Burtcher and Paruj Ratanaworabhan. Fpc: A high-speed compressor for double-precision floating-point data. IEEE Transactions on Computers, 58(1):18–31, 2009.
- [13] Tony F Chan, Jinchao Xu, and Ludmil Zikatanov. An agglomeration multigrid method for unstructured grids. Contemporary Mathematics, 218:67–81, 1998.
- [14] Tianping Chen and Hong Chen. Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. IEEE transactions on neural networks, 6(4):911–917, 1995.
- [15] Yifan Chen, Bamdad Hosseini, Houman Owhadi, and Andrew M Stuart. Solving and learning nonlinear pdes with gaussian processes. Journal of Computational Physics, 447:110668, 2021.
- [16] Steven Claggett, Sahar Azimi, and Martin Burtcher. Spdp: An automatically synthesized lossless compression algorithm for floating-point data. 2018 Data Compression Conference, pages 335–344, 2018.
- [17] Andrew J Cleary, Robert D Falgout, Van Emden Henson, and Jim E Jones. Coarse-grid selection for parallel algebraic multigrid. In International Symposium on Solving Irregularly Structured Problems in Parallel, pages 104–115. Springer, 1998.
- [18] Y. Collet and M. Kucherauw. Rfc 8878: Zstandard compression and the 'application/zstd' media type, 2021.
- [19] Hans De Sterck, Robert D Falgout, Joshua W Nolting, and Ulrike Meier Yang. Distance-two interpolation for parallel algebraic multigrid. Numerical Linear Algebra with Applications, 15(2-3):115–139, 2008.
- [20] Peter Deutsch. Gzip file format specification version 4.3. Technical report, 1996.
- [21] Sheng Di and Franck Cappello. Fast error-bounded lossy hpc data compression with sz. In 2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pages 730–739. IEEE, 2016.
- [22] Qian Gong, Jieyang Chen, Ben Whitney, Xin Liang, Viktor Reshniak, Tania Banerjee, Jaemoon Lee, Anand Rangarajan, Lipeng Wan, Nicolas Vidal, et al. Mgard: A multi-grid framework for high-performance, error-controlled data compression and refactoring. SoftwareX, 24:101590, 2023.
- [23] Jason Graham, MyoungKyu Lee, Nicholas Malaya, Gregory Eyink, Charles Meneveau, Kalin Kanov, Randal Burns, Alex Szalay, et al. Turbulent channel flow, 2017.

- [24] Leslie Greengard and Vladimir Rokhlin. A new version of the fast multipole method for the laplace equation in three dimensions. *Acta numerica*, 6:229–269, 1997.
- [25] Jeffrey T Hadley, Kenneth E Jansen, and John A Evans. Spectrally safe neural operator warm-starts for large-scale newton solvers. *arXiv preprint arXiv:2607.xxxxx*, 2026.
- [26] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural networks*, 2(5):359–366, 1989.
- [27] Thomas JR Hughes, Guglielmo Scovazzi, and Tayfun E Tezduyar. Stabilized methods for compressible flows. *Journal of Scientific Computing*, 43(3):343–368, 2010.
- [28] *hypre*: High performance preconditioners. <https://llnl.gov/casc/hypre>, <https://github.com/hypre-space/hypre>
- [29] James R Wright III, Valeria Barra, Jed Brown, Leila Ghaffari, Kenneth Jansen, Jeremy L. Thompson. Honee: High-order navier-stokes equation evaluator.
- [30] Kenneth E Jansen, Christian H Whiting, and Gregory M Hulbert. A generalized- α method for integrating the filtered navier–stokes equations with a stabilized finite element method. *Computer methods in applied mechanics and engineering*, 190(3-4):305–319, 2000.
- [31] Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhat-tacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces with applications to pdes. *Journal of Machine Learning Research*, 24(89):1–97, 2023.
- [32] Youngkyu Lee, Shanqing Liu, Jerome Darbon, and George Em Karniadakis. A neural-operator preconditioned newton method for accelerated nonlinear solvers. *arXiv preprint arXiv:2511.08811*, 2025.
- [33] Yi Li, Eric Perlman, Minping Wan, Yunke Yang, Charles Meneveau, Randal Burns, Shiyi Chen, Alexander Szalay, and Gregory Eyink. A public turbulence database cluster and applications to study lagrangian evolution of velocity increments in turbulence. *Journal of Turbulence*, (9):N31, 2008.
- [34] Zhihao Li, Zhilu Lai, Xiaobo Zhang, and Wei Wang. M2no: An efficient multi-resolution operator framework for dynamic multi-scale pde solvers. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*, pages 771–782, 2026.
- [35] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhat-tacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. *arXiv preprint arXiv:2010.08895*, 2020.
- [36] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhat-tacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Graph kernel network for partial differential equations. *arXiv preprint arXiv:2003.03485*, 2020.

- [37] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Andrew Stuart, Kaushik Bhattacharya, and Anima Anandkumar. Multipole graph neural operator for parametric partial differential equations. Advances in Neural Information Processing Systems, 33:6755–6766, 2020.
- [38] Xin Liang, Ben Whitney, Jieyang Chen, Lipeng Wan, Qing Liu, Dingwen Tao, James Kress, David Pugmire, Matthew Wolf, Norbert Podhorszki, et al. Mgard+: Optimizing multilevel methods for error-bounded scientific data reduction. IEEE Transactions on Computers, 71(7):1522–1536, 2021.
- [39] Xin Liang, Kai Zhao, Sheng Di, Sihuan Li, Robert Underwood, Ali M Gok, Jiannan Tian, Junjing Deng, Jon C Calhoun, Dingwen Tao, et al. Sz3: A modular framework for composing prediction-based error-bounded lossy compressors. IEEE Transactions on Big Data, 9(2):485–498, 2022.
- [40] P Lindstrom. Error distributions of lossy floating-point compressors. 10 2017.
- [41] Peter Lindstrom. Fixed-rate compressed floating-point arrays. IEEE transactions on visualization and computer graphics, 20(12):2674–2683, 2014.
- [42] Peter G Lindstrom. Fpzip. Technical report, Lawrence Livermore National Laboratory (LLNL), Livermore, CA (United States), 2017.
- [43] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In International Conference on Learning Representations, 2019.
- [44] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. Nature machine intelligence, 3(3):218–229, 2021.
- [45] Lu Lu, Xuhui Meng, Shengze Cai, Zhiping Mao, Somdatta Goswami, Zhongqiang Zhang, and George Em Karniadakis. A comprehensive and fair comparison of two neural operators (with practical extensions) based on fair data. Computer Methods in Applied Mechanics and Engineering, 393:114778, 2022.
- [46] Tao Lu, Qing Liu, Xubin He, Huizhang Luo, Eric Suchyta, Jong Choi, Norbert Podhorszki, Scott Klasky, Mathew Wolf, Tong Liu, et al. Understanding and modeling lossy compression schemes on hpc scientific data. In 2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pages 348–357. IEEE, 2018.
- [47] Jan Mandel, Marian Brezina, and Petr Vaněk. Energy optimization of algebraic multigrid bases. Computing, 62:205–228, 1999.
- [48] Grant Norman, Conor Rowan, Kurt Maute, and Alireza Doostan. Learning gradient flow: Using equation discovery to accelerate engineering optimization. arXiv preprint arXiv:2602.13513, 2026.

- [49] Jaemin Oh, Youngkyu Lee, Jerome Darbon, and George Em Karniadakis. Spectrally safe neural operator warm-starts for large-scale newton solvers. arXiv preprint arXiv:2606.21828, 2026.
- [50] Luke N Olson, Jacob B Schroder, and Raymond S Tuminaro. A general interpolation strategy for algebraic multigrid using energy minimization. SIAM Journal on Scientific Computing, 33(2):966–991, 2011.
- [51] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. Advances in neural information processing systems, 32, 2019.
- [52] Eric Perlman, Randal Burns, Yi Li, and Charles Meneveau. Data exploration of turbulence simulations using a database cluster. In Proceedings of the 2007 ACM/IEEE Conference on Supercomputing, pages 1–11, 2007.
- [53] JW Ruge and K Stuben. Algebraic multigrid. multigrid methods. Frontiers in Applied Mathematics, 3:73–130, 1987.
- [54] Kianoosh Taghikhani, Yusuke Yamazaki, Jerry Paul Varghese, Markus Apel, Reza Najian Asl, and Shahed Rezaei. Neural-initialized newton: Accelerating nonlinear finite elements via operator learning. arXiv preprint arXiv:2511.06802, 2025.
- [55] Dingwen Tao, Sheng Di, Zizhong Chen, and Franck Cappello. Significantly improving lossy compression for scientific data sets based on multidimensional prediction and error-controlled quantization. In 2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pages 1129–1139. IEEE, 2017.
- [56] Tapas Tripura and Souvik Chakraborty. Wavelet neural operator: a neural operator for parametric partial differential equations. arXiv preprint arXiv:2205.02191, 2022.
- [57] U. Trottenberg, C.W. Oosterlee, and A. Schuller. Multigrid Methods. Elsevier Science, 2001.
- [58] Petr Vanek, Jan Mandel, and Marian Brezina. Algebraic multigrid by smoothed aggregation for second and fourth order elliptic problems. Computing, 56(3):179–196, 1996.
- [59] Wing Lok Wan. An energy-minimizing interpolation for multigrid methods. Citeseer, 1997.
- [60] Wing Lok Wan, Tony F Chan, and Barry Smith. An energy-minimizing interpolation for robust multigrid methods. SIAM Journal on Scientific Computing, 21(4):1632–1649, 1999.

- [61] Yizheng Wang, Zhongkai Hao, Mohammad Sadegh Eshaghi, Cosmin Anitescu, Xiaoying Zhuang, Timon Rabczuk, and Yinghua Liu. Pretrain finite element method: A pretraining and warm-start framework for pdes via physics-informed neural operators. arXiv preprint arXiv:2601.03086, 2026.
- [62] Yizheng Wang, Xiang Li, Ziming Yan, Shuaifeng Ma, Jinshuai Bai, Bokai Liu, Xiaoying Zhuang, Timon Rabczuk, and Yinghua Liu. A pretraining-finetuning computational framework for material homogenization. International Journal of Mechanical Sciences, 314:111388, 2026.
- [63] I. II Wright, James Ray. Improving the Efficiency and Performance of Scale-Resolving Simulations Using Stabilized Continuous-Galerkin Finite Elements. PhD thesis, 2025. Copyright - Database copyright ProQuest LLC; ProQuest does not claim copyright in the individual underlying works; Last updated - 2025-10-09.
- [64] Ulrike Meier Yang et al. Boomerang: A parallel algebraic multigrid solver and preconditioner. Applied Numerical Mathematics, 41(1):155–177, 2002.
- [65] Kai Zhao, Sheng Di, Maxim Dmitriev, Thierry-Laurent D Tonellot, Zizhong Chen, and Franck Cappello. Optimizing error-bounded lossy compression for scientific data by dynamic spline interpolation. In 2021 IEEE 37th International Conference on Data Engineering (ICDE), pages 1643–1654. IEEE, 2021.
- [66] Xu-Hui Zhou, Jiequn Han, Muhammad I Zafar, Eric M Wolf, Christopher R Schrock, Christopher J Roy, and Heng Xiao. Neural operator-based super-fidelity: A warm-start approach for accelerating steady-state simulations. Journal of Computational Physics, 529:113871, 2025.